TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP HCM
KHOA ĐIỆN - ĐIỆN TỬ

ĐỒ ÁN 1
ĐỀ TÀI: THIẾT KẾ BỘ LỌC FIR TRÊN FPGA

Giảng viên hướng dẫn: Ths. Nguyễn Văn Phúc
Sinh viên thực hiện:
Đinh Thị Kiều My 23161297
Phạm Văn Nhiệm 23161305

Tp. Hồ Chí Minh, năm 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài, nhóm em đã nhận được sự hướng dẫn và hỗ trợ từ quý thầy cô, gia đình và bạn bè.

Nhóm em xin chân thành cảm ơn thầy hướng dẫn đã tận tình định hướng, góp ý và hỗ trợ nhóm trong suốt quá trình thực hiện đề tài. Những góp ý của thầy đã giúp nhóm hoàn thiện nội dung thiết kế và báo cáo tốt hơn.

Do kiến thức và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi thiếu sót. Nhóm em rất mong nhận được ý kiến đóng góp từ quý thầy cô để đề tài được hoàn thiện hơn.

Nhóm em xin chân thành cảm ơn.

LỜI CAM ĐOAN

Nhóm em xin cam đoan nội dung báo cáo là kết quả nghiên cứu và thực hiện của nhóm dưới sự hướng dẫn của giảng viên hướng dẫn. Các tài liệu, số liệu và hình ảnh tham khảo sử dụng trong báo cáo đều được trích dẫn rõ ràng theo đúng quy định.

Nhóm em xin chịu trách nhiệm về tính trung thực và chính xác của nội dung báo cáo này.

TÓM TẮT NỘI DUNG

Đề tài tập trung thiết kế và kiểm chứng hệ thống lọc tín hiệu số trên FPGA Sipeed Tang Nano 9K, sử dụng bộ lọc FIR thông thấp 200-tap kết hợp giao tiếp UART với máy tính. Hệ thống cho phép máy tính truyền hệ số FIR xuống FPGA, FPGA tiếp nhận hệ số, đọc dữ liệu đầu vào từ ROM, thực hiện lọc tín hiệu và gửi kết quả sau xử lý trở lại máy tính để kiểm tra và so sánh.

Trong quá trình thực hiện, nhóm xây dựng các khối chức năng chính gồm UART Receiver, UART Transmitter, Byte Packer, Unbyte Packer, bộ nạp hệ số, ROM dữ liệu đầu vào, FIFO lưu kết quả và bộ lọc FIR. Bộ lọc FIR được hiện thực theo kiến trúc MAC tuần tự nhằm giảm số lượng phần cứng sử dụng so với kiến trúc nhân song song.

Thông qua mô phỏng và thực nghiệm, đề tài giúp kiểm chứng luồng truyền nhận dữ liệu giữa máy tính và FPGA, hoạt động của bộ lọc FIR trên phần cứng, cũng như khả năng so sánh kết quả giữa Python/Web và FPGA. Kết quả thực hiện là cơ sở để đánh giá tính đúng đắn của hệ thống và khả năng nạp lại hệ số FIR thông qua UART mà không cần nạp lại bitstream cho FPGA.

MỤC LỤC

TÓM TẮT NỘI DUNG I
MỤC LỤC II
DANH SÁCH HÌNH ẢNH V
DANH SÁCH BẢNG VII
CHƯƠNG 1 GIỚI THIỆU 1
1.1 Tổng quan tình hình nghiên cứu 1
1.1.1 Trong nước 1
1.1.2 Ngoài nước 2
1.2 Lý do chọn đề tài 4
1.3 Mục tiêu nghiên cứu 5
1.4 Đối tượng và phạm vi nghiên cứu 5
1.5 Nội dung nghiên cứu 5
1.6 Giới hạn đề tài 5
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT 7
2.1 Cơ sở lý thuyết về bộ lọc FIR thông thấp 7
2.1.1 Khái niệm bộ lọc FIR thông thấp 7
2.1.2 Phương trình toán học của bộ lọc FIR thông thấp 7
2.1.3 Đáp ứng xung, đáp ứng tần số và pha tuyến tính 8
2.2 Phương pháp thiết kế bộ lọc FIR thông thấp 10
2.2.1 Phương pháp cửa sổ 10
2.2.2 Cửa sổ chữ nhật và hiện tượng Gibbs 10
2.2.3 Các hàm cửa sổ cải tiến 11
2.2.4 Cửa sổ Kaiser 11
2.3 Biểu diễn fixed-point trong bộ lọc FIR 12
2.3.1 Biểu diễn số bù hai 12
2.3.2 Định dạng Q15 12
2.3.3 Sai số lượng tử hóa hệ số 12
2.4 Tổng quan về FPGA trong xử lý tín hiệu số 13
2.5 Tổng quan giao tiếp UART 13
2.5.1 Khái niệm UART 13
2.5.2 Khung dữ liệu UART 14
2.5.3 Nguyên lý hoạt động UART 14
2.5.4 Baudrate và thời điểm lấy mẫu 15
2.5.5 Mạch đồng bộ trong UART 15
2.6 Giao tiếp UART giữa FPGA và máy tính 16
CHƯƠNG 3 THIẾT KẾ HỆ THỐNG 17

3.1 Yêu cầu thiết kế hệ thống 17
3.2 Phần cứng và công cụ sử dụng 17
3.2.1 Board FPGA Sipeed Tang Nano 9K 17
3.2.2 Kết nối phần cứng sử dụng trong hệ thống 18
3.2.3 Công cụ phần mềm sử dụng 19
3.3 Kiến trúc tổng thể hệ thống 19
3.3.1 Sơ đồ khối tổng thể 19
3.3.2 Luồng dữ liệu FPGA - PC 20
3.4 Thiết kế khối UART Receiver 21
3.4.1 Thiết kế mạch đồng bộ tín hiệu RX 22
3.4.2 Máy trạng thái FSM của UART Receiver 23
3.5 Thiết kế khối UART Transmitter 25
3.6 Thiết kế khối rom và rom_controller 26
3.7 Thiết kế khối FIFO 28
3.8 Thiết kế bộ lọc FIR thông thấp 30
3.8.1 Thông số thiết kế 30
3.8.2 Kiến trúc FIR trên FPGA 30
3.9 Thiết kế phần mềm PC sử dụng Streamlit 32
3.9.1 Cấu trúc giao diện phần mềm 32
3.9.2 Quy trình tính toán và lượng tử hóa hệ số 33
3.9.3 Tạo dữ liệu cấu hình và tín hiệu kiểm chứng 34
3.10 Tích hợp hệ thống 35
CHƯƠNG 4 KẾT QUẢ THỰC NGHIỆM 36
4.1 Kiểm chứng thiết kế bằng mô phỏng RTL trên Xilinx ISE/ISim 36
4.1.1 Mục đích, thiết lập mô phỏng và dữ liệu kiểm thử 36
4.1.2 Kịch bản kiểm thử và luồng dữ liệu trong testbench 37
4.1.3 Kết quả mô phỏng, nhận xét và đánh giá 39
4.2 Kiểm chứng mức hệ thống thông qua Python/Streamlit 49
4.2.1 Mục đích kiểm chứng mức hệ thống 49
4.2.2 Quy trình kiểm chứng trên Streamlit 50
4.2.3 Kết quả và nhận xét 53
4.3.1 Làm rõ vai trò xử lý của FPGA trong hệ thống .. **Error! Bookmark not defined.**
4.3.2 Đánh giá định lượng sai số giữa FPGA và mô phỏng Python ..... **Error! Bookmark not defined.**
4.3.3 Đánh giá thời gian xử lý và băng thông truyền dữ liệu ..... **Error! Bookmark not defined.**

4.3.4 Đánh giá lựa chọn kiến trúc và sử dụng tài nguyên FPGA ..... **Error! Bookmark not defined.**
4.3.5 Nhận xét tổng hợp.....**Error! Bookmark not defined.**
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....58
5.1 Kết luận 58
5.2 Hướng phát triển của đề tài 58
Tài liệu tham khảo 66

DANH SÁCH HÌNH ẢNH

Hình 2.1 Bộ lọc số xử lý chuỗi mẫu rời rạc 7
Hình 2.2 Cấu trúc dạng trực tiếp của bộ lọc FIR với N hệ số 8
Hình 2.3 Đáp ứng xung dạng sinc của bộ lọc thông thấp lý tưởng 9
Hình 2.4 So sánh đáp ứng tần số lý tưởng và đáp ứng tần số thực tế 9
Hình 2.5 So sánh bộ lọc pha tuyến tính và pha phi tuyến 10
Hình 2.6 Minh họa đáp ứng tần số khi dùng cửa sổ hình chữ nhật 11
Hình 2.7 Một số hàm cửa sổ thường dùng trong thiết kế FIR. 12
Hình 2.8 Sơ đồ kết nối UART giữa hai thiết bị 14
Hình 2.9 Cấu trúc khung dữ liệu trong giao tiếp UART 15
Hình 2.10 Minh họa thời điểm lấy mẫu dữ liệu UART tại trung tâm bit 16
Hình 2.11 Mạch đồng bộ hai tầng Flip-Flop cho tín hiệu UART 16
Hình 3.1 Hình ảnh thực tế của board Sipeed Tang Nano 9K 19
Hình 3.2 Sơ đồ khối tổng thể hệ thống lọc FIR trên FPGA 21
Hình 3.3 Luồng dữ liệu giữa FPGA và máy tính 21
Hình 3.4 Sơ đồ khối kết nối của khối UART Receiver trong hệ thống 22
Hình 3.5 Sơ đồ khối bộ tạo xung lấy mẫu UART 23
Hình 3.6 Mạch đồng bộ tín hiệu RX dùng hai Flip-Flop 24
Hình 3.7 Máy trạng thái FSM của khối UART Receiver 25
Hình 3.8 Sơ đồ ghép byte và nạp hệ số FIR 25
Hình 3.9 Sơ đồ khối kết nối của khối UART Transmitter trong hệ thống 27
Hình 3.10 Máy trạng thái FSM của khối UART Transmitter 27
Hình 3.11 Sơ đồ khối ROM 28
Hình 3.12 Sơ đồ khối FIFO buffer 30
Hình 3.13 Kiến trúc FIR 200-tap MAC tuần tự 32
Hình 3.14 Máy trạng thái hữu hạn điều khiển khối FIR Filter 33
Hình 3.15 Sơ đồ timing điều khiển khối FIR 33
Hình 3.16 Quy trình tính toán và lượng tử hóa hệ số 36
Hình 3.17 Sơ đồ tích hợp hệ thống 37
Hình 3.18 Sơ đồ chân vào/ra của module top_system 37
Hình 4.1 Dạng sóng tạo xung enable_16x cho UART 42
Hình 4.2 Dạng sóng đồng bộ tín hiệu UART RX 43
Hình 4.3 Dạng sóng UART Receiver nhận một byte dữ liệu 44

Hình 4.4 Dạng sóng Byte Packer ghép hai byte thành hệ số 16 bit 44
Hình 4.5 Dạng sóng nạp hệ số đầu tiên h[0].....45
Hình 4.6 Dạng sóng nạp hệ số cuối h[199] và kích hoạt coeffs_loaded.....45
Hình 4.7 Dạng sóng ROM Controller cấp mẫu đầu vào cho FIR.....46
Hình 4.8 Dạng sóng FIR Filter xử lý mẫu đầu vào và xuất kết quả y(n) 47
Hình 4.9 Dạng sóng FIFO Buffer lưu và đọc kết quả FIR.....48
Hình 4.10 Dạng sóng Unbyte Packer tách kết quả FIR 41 bit thành các byte 8 bit ..48
Hình 4.11 Dạng sóng UART Transmitter truyền byte dữ liệu qua tx_pin.....49
Hình 4.12 Kết quả tổng hợp cuối mô phỏng trên ISim.....50
Hình 4.13. Giao diện nhập thông số thiết kế bộ lọc 52
Hình 4.14 Giao diện hiển thị đáp ứng tần số mô phỏng Python 53
Hình 4.15 Giao diện hiển thị tín hiệu thử nghiệm 53
Hình 4.16 Giao diện đóng gói các tệp để nạp vào kit 54
Hình 4.17 Giao diện kiểm chứng kết quả 54
Hình 4.18 Thông số thiết kế FIR trong Case 1 55
Hình 4.19 Kết quả case 1.....55
Hình 4.20 Thông số thiết kế FIR trong case 2.....56
Hình 4.21 Kết quả case 2.....57
Hình 4.22 Thông số thiết kế FIR trong Case 3 58
Hình 4.23 Kết quả case 3.....58

DANH SÁCH BẢNG

Bảng 1.1 Một số công trình trong nước liên quan đến các thành phần của đề tài 1
Bảng 1.2 Một số công trình quốc tế liên quan đến thiết kế bộ lọc FIR trên FPGA 3
Bảng 3.1 Thông số phần cứng chính của board Sipeed Tang Nano 9K.....19
Bảng 3.2 Các tín hiệu chính sử dụng trong module top_system.....19
Bảng 3.3 Công cụ phần mềm sử dụng 20
Bảng 3.4. Bảng tín hiệu của khối rom và rom_controller 29
Bảng 3.5. Thành phần bên trong khối FIFO 30
Bảng 3.6 Thông số thiết kế chính của bộ lọc FIR.....32
Bảng 3.7 Các tín hiệu giao tiếp chính của khối FIR Filter 33
Bảng 3.8 Cấu trúc các tab chức năng của phần mềm Streamlit 34
Bảng 3.9. Các tệp cấu hình và kiểm chứng do phần mềm tạo ra 37
Bảng 4.1 Thông số thiết lập mô phỏng 38
Bảng 4.2 Tổng hợp các case kiểm thử trong testbench 39
Bảng 4.3 Tiêu chí đánh giá trong pha kiểm thử tích hợp 41
Bảng 4.4 Tổng hợp kết quả kiểm chứng theo từng giai đoạn 50
Bảng 4.5 Tổng hợp thông số timing trong mô phỏng 51
Bảng 4.6 Các trường hợp kiểm chứng bộ lọc FIR trên Streamlit.....54
Bảng 4.7 Kết quả case 1 55
Bảng 4.8 Kết quả case 2 57
Bảng 4.9 Kết quả case 3 58
Bảng 4.10 Tổng hợp kết quả 59

CHƯƠNG 1 GIỚI THIỆU

1.1 Tổng quan tình hình nghiên cứu

Trong các hệ thống xử lý tín hiệu số, bộ lọc số đóng vai trò quan trọng trong việc loại bỏ nhiễu, tách các thành phần tần số mong muốn và nâng cao chất lượng tín hiệu trước khi đưa vào các khối xử lý tiếp theo. Trong đó, bộ lọc FIR được sử dụng rộng rãi nhờ đáp ứng xung hữu hạn, tính ổn định cao và khả năng thiết kế đáp ứng pha tuyến tính.

Cùng với sự phát triển của các ứng dụng xử lý tín hiệu thời gian thực, FPGA ngày càng được sử dụng phổ biến nhờ khả năng xử lý song song, tốc độ cao, độ trễ thấp và tính linh hoạt trong tái cấu hình phần cứng. Những đặc điểm này giúp FPGA đáp ứng hiệu quả các bài toán xử lý dữ liệu yêu cầu hiệu năng cao và khả năng mở rộng.

Bên cạnh việc thiết kế bộ lọc FIR, một hệ thống xử lý tín hiệu hoàn chỉnh trên FPGA còn phải giải quyết các vấn đề như truyền nhận dữ liệu giữa máy tính và FPGA, nạp hệ số lọc, biểu diễn dữ liệu theo dạng fixed-point và kiểm chứng kết quả giữa mô phỏng với phần cứng thực tế. Do đó, các nghiên cứu về bộ lọc FIR, FPGA, giao tiếp UART và phương pháp kiểm chứng mạch số là cơ sở quan trọng để xây dựng và phát triển đề tài.

1.1.1 Trong nước

Trong nước, các nghiên cứu liên quan chủ yếu tập trung vào từng thành phần riêng của hệ thống như thiết kế bộ lọc số, triển khai thuật toán DSP trên FPGA, xây dựng giao tiếp nối tiếp hoặc phát triển các mô hình thực hành FPGA. Mặc dù chưa có nhiều công trình tích hợp đồng thời các chức năng như nạp hệ số FIR qua UART, xử lý tín hiệu trên FPGA và kiểm chứng kết quả với máy tính, các nghiên cứu này vẫn cung cấp cơ sở tham khảo quan trọng về phương pháp thiết kế, hiện thực và kiểm chứng cho từng khối chức năng của đề tài.

Bảng 1.1 Một số công trình trong nước liên quan đến các thành phần của đề tài

STT	**Tác giả – Năm**	**Nội dung nghiên cứu**	**Phương pháp**	**Kết quả**	**Hạn chế**	**Khả năng kế thừa**
1	Bùi Phương Thảo, Nguyễn Xuân Kiên, Đỗ Văn Định – 2025	So sánh bộ lọc FIR và LMS trong xử lý nhiễu tín hiệu EEG	Áp dụng bộ lọc FIR và LMS để lọc nhiễu tín hiệu EEG	Cho thấy FIR có khả năng ứng dụng trong xử lý tín hiệu thực tế	Chưa kiểm chứng trên phần cứng thực tế	Kế thừa cơ sở ứng dụng của FIR trong lọc tín hiệu
2	Nguyễn Văn Thành Lộc,	Thiết kế bộ tính toán FFT 16 điểm	Sử dụng cấu trúc R2SDF, pipeline, fixed-point 16 bit và testbench	Minh họa quy trình đưa thuật	Chỉ tập trung vào FFT, chưa mở rộng	Kế thừa quy trình thiết kế và kiểm

STT	Tác giả – Năm	Nội dung nghiên cứu	Phương pháp	Kết quả	Hạn chế	Khả năng kế thừa
	Hoàng Trương Hữu Thương, Mai Thị Hoài Nhi, Đỗ Duy Tân – 2022	trên FPGA		toán DSP lên FPGA	sang FIR/IIR	chứng DSP trên FPGA
3	Trần Đức Sang và cộng sự – 2021	Thiết kế và kiểm chứng module SIE chuẩn USB 2.0	Thiết kế khối giao tiếp nối tiếp và mô phỏng kiểm chứng	Cung cấp quy trình kiểm chứng khối giao tiếp trên FPGA	Tập trung vào USB 2.0, chưa xét UART đơn giản	Kế thừa cách thiết kế và kiểm chứng khối truyền nhận dữ liệu
4	Trần Đình Thông, Dư Đình Viên và cộng sự – 2018	Thiết kế bộ lọc CIC và FIR cho module thu SDR	Sử dụng MATLAB/Simulink và System Generator	Cho thấy CIC/FIR có thể triển khai trong tuyến thu SDR	Ngoài giai đoạn 2021–2025; chưa phân tích cập nhật hệ số động	Kế thừa hướng thiết kế bộ lọc số và triển khai xuống FPGA
5	Nguyễn Đình Phú, Trương Thị Bích Ngà, Phan Văn Hoan – 2024	Nghiên cứu, thiết kế kit thực hành FPGA	Khảo sát FPGA, I/O, ngoại vi và khả năng mở rộng	Cung cấp nền tảng phần cứng phục vụ thực nghiệm FPGA	Chưa triển khai bài toán xử lý tín hiệu số cụ thể	Kế thừa nền tảng thực nghiệm cho FIR, UART và FIFO

1.1.2 Ngoài nước

Ở ngoài nước, các nghiên cứu về bộ lọc FIR trên FPGA được phát triển theo hướng chuyên sâu hơn, không chỉ tập trung vào hiện thực thuật toán mà còn chú trọng tối ưu kiến

trúc phần cứng, giảm tài nguyên, giảm công suất tiêu thụ và hỗ trợ tái cấu hình trong quá trình hoạt động. Nhiều công trình khai thác hiệu quả các khối MAC, RAM hai cổng và các kiến trúc pipeline nhằm nâng cao hiệu năng xử lý thời gian thực. Những kết quả này là cơ sở tham khảo quan trọng để lựa chọn kiến trúc, phương pháp hiện thực và định hướng mở rộng hệ thống trong đề tài.

Bảng 1.2 Một số công trình quốc tế liên quan đến thiết kế bộ lọc FIR trên FPGA

STT	Tác giả – Năm	Nội dung nghiên cứu	Phương pháp	Kết quả	Hạn chế	Khả năng kế thừa
1	Goldsmith, Crockett, Stewart – 2022	Thiết kế bộ lọc FIR fixed-point có khả năng cấu hình lại trong thời gian chạy trên FPGA	Xây dựng phương pháp thiết kế FIR fixed-point và cho phép thay đổi một số tham số lọc trong runtime	Cho phép cấu hình lại tần số cắt của bộ lọc FIR trong quá trình vận hành	Một số tham số như độ dài bộ lọc, loại cửa sổ và độ rộng từ vẫn được cấu hình trước khi tổng hợp phần cứng	Kế thừa ý tưởng cập nhật đặc tính lọc hoặc hệ số FIR mà không cần nạp lại toàn bộ thiết kế phần cứng
2	Al-Dulaimi, Wahhab, Amer – 2023	Thiết kế và triển khai bộ lọc FIR cho tín hiệu audio trên nền tảng FPGA	Thiết kế bộ lọc FIR, mô phỏng và triển khai trên board FPGA Altera DE1	Chứng minh bộ lọc FIR có thể xử lý tín hiệu audio trên FPGA	Tập trung vào ứng dụng audio, chưa phân tích sâu cơ chế cập nhật hệ số FIR trong quá trình vận hành	Kế thừa cách tổ chức thực nghiệm tín hiệu trước và sau lọc trên FPGA
3	Aljumaili và cộng sự – 2025	Triển khai bộ lọc FIR thông thấp trên FPGA sử dụng một khối MAC và RAM hai cổng	Sử dụng cấu trúc FIR đối xứng, một khối Multiply-Accumulate và dual-port RAM	Giảm kích thước phần cứng khi triển khai bộ lọc FIR bậc cao	Kiến trúc dùng một khối MAC giúp giảm tài nguyên nhưng có thể làm tăng số chu kỳ xử lý cho	Kế thừa ý tưởng sử dụng bộ nhớ hệ số, bộ nhớ mẫu và khối MAC để tối ưu tài nguyên FPGA

STT	Tác giả – Năm	Nội dung nghiên cứu	Phương pháp	Kết quả	Hạn chế	Khả năng kế thừa
					mỗi mẫu dữ liệu	
4	Soni, Kumar, Sharma, Panda – 2025	Thiết kế bộ lọc FIR không dùng bộ nhân trên FPGA và ASIC	Sử dụng phương pháp Sparse PSO để tối ưu bộ lọc FIR multiplierless	Giảm sự phụ thuộc vào các phần tử nhân phần cứng, phù hợp với hướng tối ưu tài nguyên	Phương pháp tối ưu multiplierless có độ phức tạp thiết kế cao và phụ thuộc vào thuật toán tối ưu hệ số	Kế thừa định hướng giảm tài nguyên phần cứng khi triển khai bộ lọc FIR trên FPGA
5	AMD/Xilinx – 2020	Mô tả kiến trúc n-tap Dual Port Memory MAC FIR Filter	Sử dụng cấu trúc Multiply-Accumulate, dual-port block RAM để lưu hệ số và mẫu dữ liệu	Cung cấp kiến trúc tham khảo cho thiết kế bộ lọc FIR dùng MAC và bộ nhớ trên FPGA	Là tài liệu kỹ thuật mô tả kiến trúc tham khảo, chưa trình bày quy trình thực nghiệm hoàn chỉnh cho một hệ thống cụ thể	Kế thừa cách tổ chức bộ nhớ hệ số, bộ nhớ mẫu và khối MAC trong thiết kế FIR trên FPGA

1.2 Lý do chọn đề tài

Từ tổng quan các nghiên cứu có thể thấy bộ lọc FIR và FPGA là hai thành phần quan trọng trong các hệ thống xử lý tín hiệu số hiện đại. Tuy nhiên, để xây dựng một hệ thống hoàn chỉnh, ngoài việc hiện thực bộ lọc trên phần cứng còn cần giải quyết các vấn đề như truyền nhận dữ liệu, nạp hệ số lọc, xử lý tín hiệu và kiểm chứng kết quả giữa phần mềm với phần cứng.

Xuất phát từ yêu cầu đó, đề tài "Thiết kế bộ lọc FIR trên FPGA" được thực hiện nhằm tích hợp kiến thức về xử lý tín hiệu số, thiết kế phần cứng FPGA và giao tiếp UART trong một hệ thống thực nghiệm thống nhất. Thông qua đề tài, nhóm có cơ hội nghiên cứu quy trình thiết kế bộ lọc FIR, triển khai thuật toán trên FPGA, truyền nhận dữ liệu với máy tính và đánh giá tính đúng đắn của kết quả xử lý giữa phần mềm và phần cứng.

1.3 Mục tiêu nghiên cứu

Mục tiêu của đề tài là thiết kế và kiểm chứng một hệ thống xử lý tín hiệu số trên FPGA, sử dụng bộ lọc FIR thông thấp có khả năng cấu hình lại hệ số lọc thông qua giao tiếp UART với máy tính. Hệ thống cho phép nạp hệ số lọc mới từ máy tính xuống FPGA mà không cần tổng hợp và nạp lại bitstream, qua đó xử lý dữ liệu đầu vào trực tiếp trên phần cứng và truyền kết quả đầu ra trở lại máy tính để phục vụ quá trình kiểm chứng.

1.4 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống xử lý tín hiệu số trên FPGA sử dụng bộ lọc FIR thông thấp có khả năng cấu hình lại hệ số thông qua giao tiếp UART với máy tính. Các thành phần chính được nghiên cứu bao gồm: bộ lọc FIR thông thấp, hệ số bộ lọc lượng tử hóa theo định dạng Q15, giao tiếp UART giữa máy tính và FPGA, các module phần cứng hiện thực trên FPGA, và giao diện Streamlit dùng để điều khiển, quan sát và kiểm chứng kết quả.

Phạm vi nghiên cứu của đề tài tập trung vào việc thiết kế và kiểm chứng bộ lọc FIR thông thấp trên FPGA theo quy trình: hệ số bộ lọc được tính toán trên máy tính, lượng tử hóa sang định dạng Q15, sau đó truyền xuống FPGA thông qua giao tiếp UART. Trên FPGA, bộ lọc thực hiện xử lý dữ liệu đầu vào và truyền kết quả đầu ra trở lại máy tính để quan sát, phân tích và so sánh với kết quả mô phỏng trên Python.

1.5 Nội dung nghiên cứu

Để đạt được mục tiêu trên, đề tài thực hiện các nội dung sau:

- Xây dựng cơ chế truyền nhận dữ liệu qua giao tiếp UART giữa máy tính và FPGA, phục vụ việc nạp hệ số lọc và truyền-nhận tín hiệu.
- Thiết kế và hiện thực bộ lọc FIR thông thấp trên FPGA, hỗ trợ cập nhật hệ số lọc linh hoạt mà không cần tổng hợp lại thiết kế.
- Xử lý tín hiệu đầu vào trực tiếp trên phần cứng FPGA và truyền kết quả lọc trở lại máy tính.
- Xây dựng giao diện giám sát và điều khiển bằng Streamlit, hỗ trợ nạp hệ số, hiển thị tín hiệu và kết quả xử lý.
- Kiểm chứng thiết kế ở hai mức: thứ nhất là mức mô phỏng phần cứng để xác minh logic thiết kế. Thứ hai là mức hệ thống thông qua so sánh kết quả xử lý trên FPGA với kết quả mô phỏng tương ứng trên Python, nhằm đánh giá tính đúng đắn và độ chính xác của hệ thống

1.6 Giới hạn đề tài

Đề tài giới hạn ở bộ lọc FIR thông thấp với dữ liệu và hệ số 16 bit, kiểm chứng bằng các tín hiệu Dirac và Chirp. Hệ thống tập trung đánh giá luồng truyền nhận UART, quá trình nạp hệ số, hoạt động của FIR, cơ chế FIFO và việc truyền kết quả về máy tính, chưa mở rộng

sang bộ lọc thông cao, thông dải, chắn dải, xử lý tín hiệu thời gian thực từ ADC hoặc các chuẩn giao tiếp tốc độ cao như USB, Ethernet, PCIe và SPI.

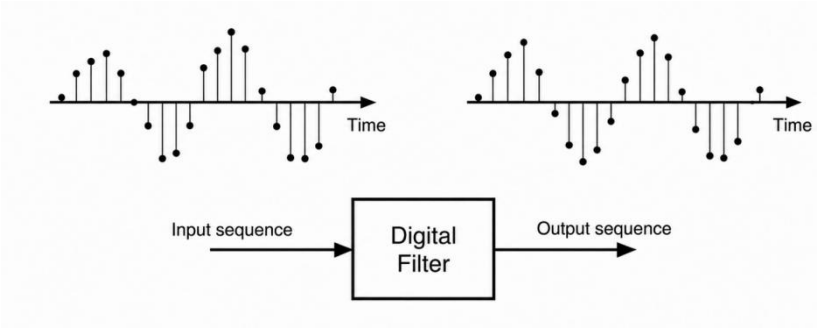
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 Cơ sở lý thuyết về bộ lọc FIR thông thấp

2.1.1 Khái niệm bộ lọc FIR thông thấp

Bộ lọc FIR (Finite Impulse Response) là bộ lọc số có đáp ứng xung hữu hạn, trong đó đầu ra chỉ phụ thuộc vào tín hiệu đầu vào hiện tại và một số mẫu trước đó. Do không sử dụng nhánh hồi tiếp nên bộ lọc FIR luôn ổn định và có thể đạt đặc tính pha tuyến tính khi các hệ số đối xứng [7].

Nhờ đặc tính không đệ quy, bộ lọc FIR hạn chế sự tích lũy sai số trong quá trình tính toán, phù hợp với các ứng dụng yêu cầu độ chính xác và độ tin cậy cao [8].



Hình 2.1 Bộ lọc số xử lý chuỗi mẫu rời rạc

2.1.2 Phương trình toán học của bộ lọc FIR thông thấp

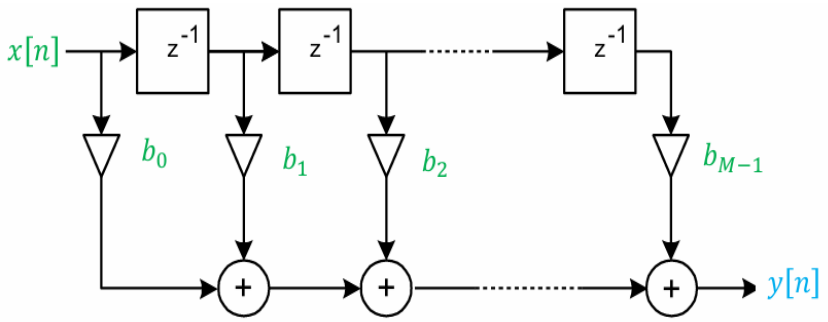
Quan hệ vào – ra của bộ lọc FIR được mô tả bởi phương trình tích chập rời rạc như sau [8]:

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n] \quad (2.1)$$

Trong đó, x[n] là tín hiệu đầu vào tại thời điểm rời rạc n, y[n] là tín hiệu đầu ra, h[k] là hệ số thứ k của bộ lọc hay còn gọi là đáp ứng xung, và N là số tap của bộ lọc FIR.

Theo (2.1), mỗi mẫu đầu ra được tính bằng tổng có trọng số của N mẫu đầu vào gần nhất, trong đó các trọng số chính là hệ số của bộ lọc. Đây là cơ sở để hiện thực FIR bằng các phần tử trễ, bộ nhân và bộ cộng.

Trong cấu trúc dạng trực tiếp của bộ lọc FIR, các mẫu đầu vào được đưa qua chuỗi phần tử trễ. Mỗi mẫu được nhân với một hệ số tương ứng, sau đó các kết quả nhân được cộng lại để tạo thành đầu ra. Cấu trúc này thể hiện rõ bản chất của phương trình tích chập rời rạc và là cơ sở cho nhiều kiến trúc phần cứng của bộ lọc FIR.



Hình 2.2 Cấu trúc dạng trực tiếp của bộ lọc FIR với N hệ số

2.1.3 Đáp ứng xung, đáp ứng tần số và pha tuyến tính

Đáp ứng tần số của bộ lọc FIR được xác định bằng biến đổi Fourier rời rạc theo thời gian của đáp ứng xung. Với bộ lọc FIR có N hệ số, đáp ứng tần số được viết như sau [7]:

$$H(e^{j\omega}) = \sum_{k=0}^{N-1} h\,[k]e^{-j\omega k} \tag{2.2}$$

Trong đó, H(e^{jω}) là đáp ứng tần số của bộ lọc, h[n] là đáp ứng xung hay dãy hệ số của bộ lọc, và ω là tần số góc chuẩn hóa theo radian trên mẫu. Đáp ứng tần số cho biết bộ lọc tác động như thế nào đến từng thành phần tần số của tín hiệu đầu vào.

Đối với bộ lọc thông thấp lý tưởng, đáp ứng biên độ có giá trị bằng 1 trong dải thông và bằng 0 trong dải chặn. Điều này có nghĩa là các thành phần tần số thấp hơn tần số cắt được giữ lại, còn các thành phần tần số cao hơn tần số cắt bị loại bỏ hoàn toàn. Tuy nhiên, bộ lọc thông thấp lý tưởng không thể hiện thực trực tiếp trong thực tế vì đáp ứng xung của nó có dạng hàm sinc kéo dài vô hạn về cả hai phía của trục thời gian rời rạc.

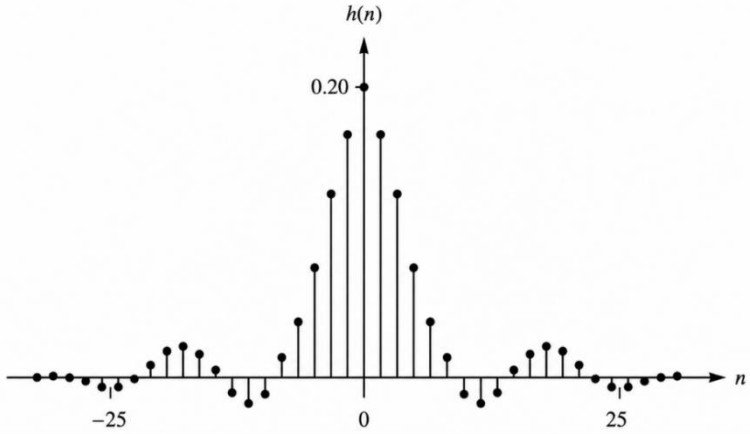
Đáp ứng xung lý tưởng của bộ lọc thông thấp có thể được biểu diễn như sau [7]:

$$h_d(n) = \begin{cases} \frac{\omega_c}{\pi}, & n = 0 \\ \frac{\sin(\omega_c n)}{\pi n}, & n \neq 0 \end{cases} \tag{2.3}$$

Trong đó, h_d[n] là đáp ứng xung lý tưởng của bộ lọc thông thấp, ω_c là tần số cắt chuẩn hóa theo radian trên mẫu, n là chỉ số mẫu trong miền thời gian rời rạc và α là vị trí tâm của đáp ứng xung. Với bộ lọc FIR có chiều dài N, vị trí tâm thường được chọn là:

$$\alpha = (N - 1) / 2 \tag{2.4}$$

Do hàm sinc kéo dài vô hạn, bộ lọc thông thấp lý tưởng là không nhân quả và không thể hiện thực trực tiếp trong hệ thống thời gian thực. Vì vậy, trong thực tế cần xấp xỉ đáp ứng lý tưởng bằng một bộ lọc FIR có số hệ số hữu hạn. Khi đó, đáp ứng tần số thực tế không còn dạng lý tưởng tuyệt đối mà xuất hiện độ gợn trong dải thông, độ gợn trong dải chặn và dải chuyển tiếp giữa hai vùng này.



Hình 2.3 Đáp ứng xung dạng sinc của bộ lọc thông thấp lý tưởng

Trong dải thông, đáp ứng biên độ của bộ lọc thực tế thường được mô tả bởi điều kiện:

$$1 - \delta_1 \leq \left|H\left(e^{j\omega}\right)\right| \leq 1 + \delta_1 \tag{2.5}$$

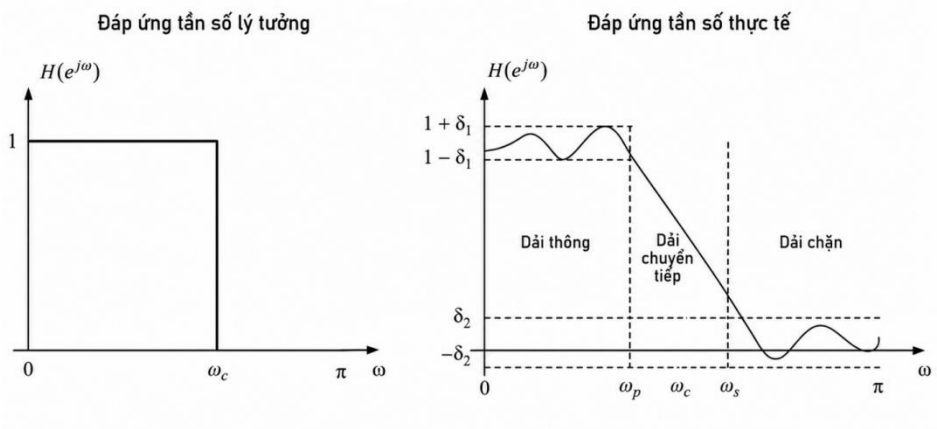
Trong dải chặn, đáp ứng biên độ được giới hạn bởi:

$$\left|H\left(e^{j\omega}\right)\right| \leq \delta_2 \tag{2.6}$$

Độ suy giảm dải chặn được xác định theo công thức [2]:

$$A_{stop} = -20\log_{10}(\delta_2)\ \text{dB} \tag{2.7}$$

Trong đó, δ_p là độ gợn dải thông, δ_s là độ gợn dải chặn, còn A_s là độ suy giảm dải chặn tính theo đơn vị dB. Các tham số này thường được dùng để đánh giá chất lượng của bộ lọc FIR sau khi thiết kế.



Hình 2.4 So sánh đáp ứng tần số lý tưởng và đáp ứng tần số thực tế

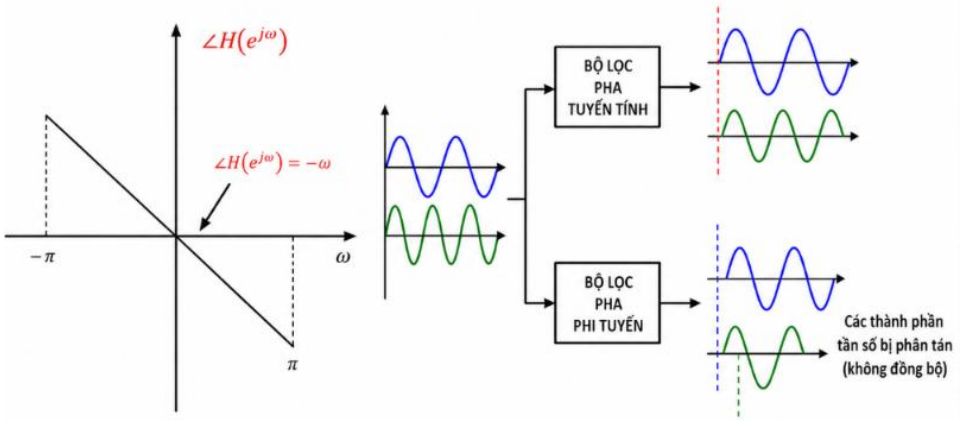
Một ưu điểm quan trọng của bộ lọc FIR là khả năng đạt pha tuyến tính. Nếu các hệ số của bộ lọc có tính đối xứng:

$$h[n] = h[N-1-n] \tag{2.8}$$

thì các thành phần tần số trong dải thông sẽ bị trễ cùng một lượng thời gian. Độ trễ nhóm khi đó được xác định bởi:

$$\tau_g = \frac{N-1}{2} \tag{2.9}$$

Đặc điểm pha tuyến tính giúp hạn chế méo dạng sóng, vì các thành phần tần số của tín hiệu không bị trễ lệch nhau. Đây là lý do bộ lọc FIR thường được ưu tiên trong các ứng dụng cần bảo toàn dạng tín hiệu, chẳng hạn như xử lý âm thanh, xử lý ảnh, truyền thông số và đo lường tín hiệu [7].



Hình 2.5 So sánh bộ lọc pha tuyến tính và pha phi tuyến

2.2 Phương pháp thiết kế bộ lọc FIR thông thấp

2.2.1 Phương pháp cửa sổ

Phương pháp cửa sổ là phương pháp thiết kế FIR dựa trên việc nhân đáp ứng xung lý tưởng với một hàm cửa sổ hữu hạn. Mục tiêu của phương pháp này là cắt ngắn đáp ứng xung lý tưởng để thu được bộ lọc thực tế có chiều dài N [7].

Trong miền thời gian, đáp ứng xung thực tế của bộ lọc được xác định bởi:

$$h[n] = h_d[n] \cdot w[n], 0 \leq n \leq N-1 \tag{2.10}$$

Trong đó, h[n] là đáp ứng xung thực tế, h_d[n] là đáp ứng xung lý tưởng và w[n] là hàm cửa sổ.

Khi chuyển sang miền tần số, phép nhân trong miền thời gian tương đương với phép tích chập trong miền tần số:

$$H(e^{j\omega}) = \frac{1}{2\pi}\int_{-\pi}^{\pi} H_d\,(e^{j\theta}) \cdot W(e^{j(\omega-\theta)})\,d\theta \tag{2.11}$$

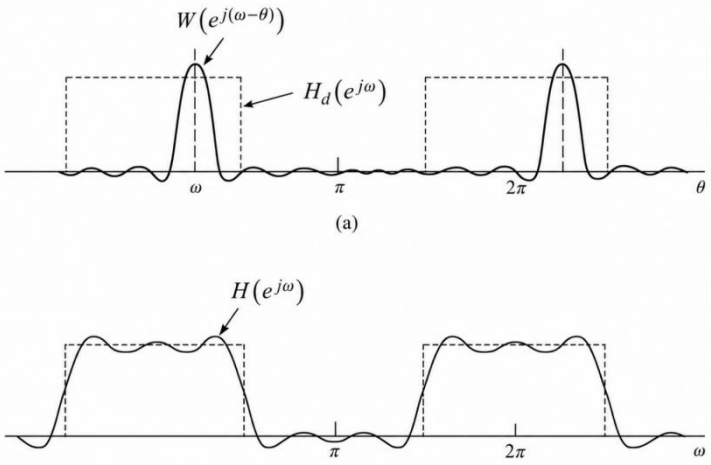
Công thức trên cho thấy đặc tính phổ của hàm cửa sổ có ảnh hưởng trực tiếp đến đáp ứng tần số của bộ lọc sau thiết kế. Vì vậy, việc lựa chọn loại cửa sổ là một bước quan trọng trong quá trình thiết kế bộ lọc FIR.

2.2.2 Cửa sổ chữ nhật và hiện tượng Gibbs

Cửa sổ chữ nhật là dạng cửa sổ đơn giản nhất. Hàm cửa sổ chữ nhật được định nghĩa như sau:

$$w_R[n] = \begin{cases} 1, & 0 \leq n \leq N-1 \\ 0, & \text{ngược lại} \end{cases} \tag{2.12}$$

Khi sử dụng cửa sổ chữ nhật, việc cắt ngắn đáp ứng xung lý tưởng làm xuất hiện các dao động trong đáp ứng tần số. Hiện tượng này thường được gọi là hiện tượng Gibbs. Các dao động này tạo ra độ gợn trong dải thông và dải chặn, khiến đáp ứng thực tế không đạt được dạng lý tưởng tuyệt đối.



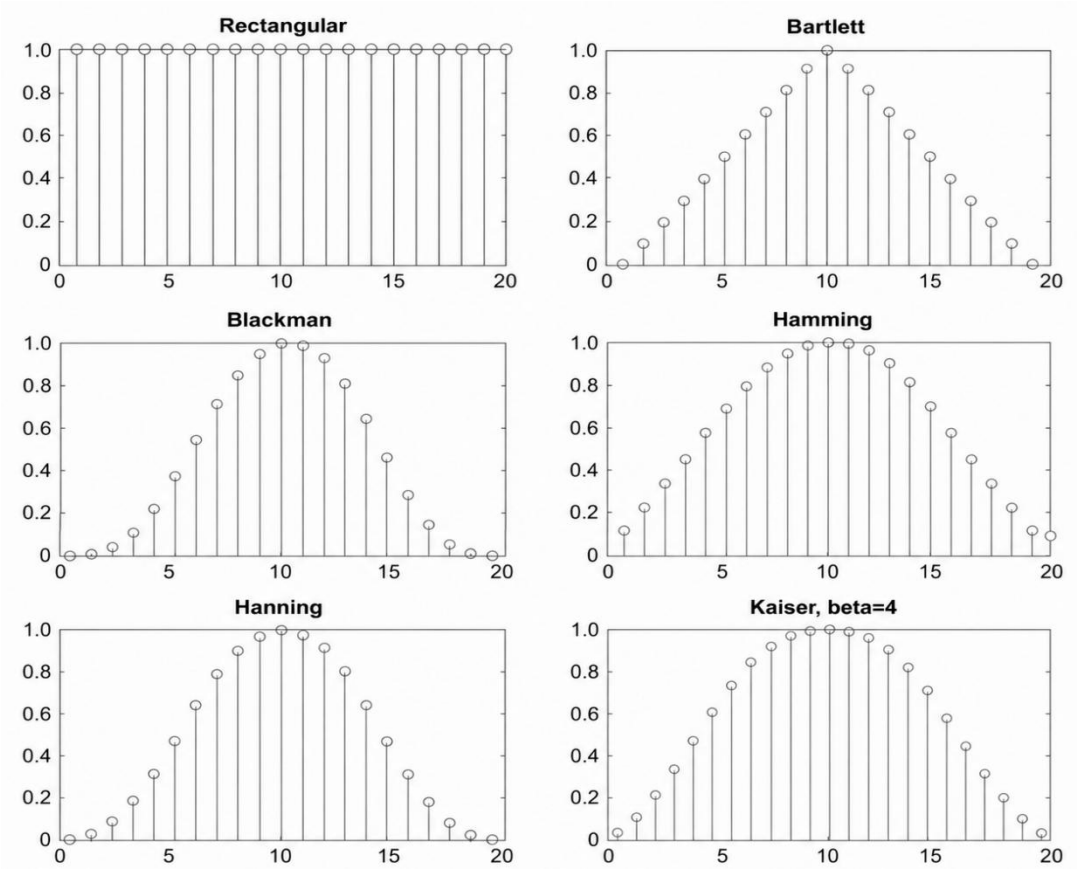
Hình 2.6 Minh họa đáp ứng tần số khi dùng cửa sổ hình chữ nhật

Quan sát Hình 2.6 có thể thấy chất lượng bộ lọc phụ thuộc trực tiếp vào đặc tính phổ của hàm cửa sổ, đặc biệt là độ rộng múi chính và biên độ các múi phụ. Múi chính càng rộng

thì dải chuyển tiếp của bộ lọc càng lớn. Ngược lại, múi phụ càng cao thì độ gợn trong dải thông và dải chặn càng lớn. Do đó, trong nhiều trường hợp cần thay cửa sổ chữ nhật bằng các hàm cửa sổ cải tiến có múi phụ thấp hơn.

2.2.3 Các hàm cửa sổ cải tiến

Các hàm cửa sổ như Hanning, Hamming, Blackman và Kaiser được sử dụng nhằm cải thiện đáp ứng tần số của bộ lọc FIR. Các cửa sổ này có thể làm giảm biên độ múi phụ, từ đó giảm độ gợn trong dải thông và dải chặn. Tuy nhiên, sự cải thiện này thường phải đánh đổi bằng việc mở rộng múi chính, làm cho dải chuyển tiếp của bộ lọc rộng hơn.



Hình 2.7 Một số hàm cửa sổ thường dùng trong thiết kế FIR.

Trong số các hàm cửa sổ, Kaiser là cửa sổ linh hoạt vì tham số β cho phép người thiết kế điều chỉnh liên tục sự đánh đổi giữa độ suy giảm dải chặn và độ rộng dải chuyển tiếp. Nhờ đó, cửa sổ Kaiser phù hợp với các bài toán cần kiểm soát đồng thời chất lượng đáp ứng tần số và chiều dài bộ lọc.

2.2.4 Cửa sổ Kaiser

Hàm cửa sổ Kaiser được định nghĩa như sau [7]:

$$w[n] = \frac{I_0\left(\beta\sqrt{1-\left(\frac{2n}{M}-1\right)^2}\right)}{I_0(\beta)}, \quad 0 \le n \le M \tag{2.13}$$

Trong đó, I_0(.) là hàm Bessel loại một bậc không, β là tham số điều chỉnh cửa sổ và α = (N − 1)/2 là vị trí tâm của cửa sổ.

Tham số β thường được xác định từ độ suy giảm dải chặn yêu cầu A_s theo công thức:

$$\beta = \begin{cases} 0.1102(A-8.7), & A > 50 \\ 0.5842(A-21)^{0.4} + 0.07886(A-21), & 21 \le A \le 50 \\ 0, & A < 21 \end{cases} \quad (2.14)$$

Chiều dài bộ lọc có thể được ước lượng dựa trên độ suy giảm dải chặn và độ rộng dải chuyển tiếp theo công thức:

$$N \approx (A_s - 8)/(2{,}285\Delta\omega) + 1 \quad (2.15)$$

Trong đó, Δω là độ rộng dải chuyển tiếp chuẩn hóa theo radian trên mẫu. Cửa sổ Kaiser được sử dụng phổ biến trong thiết kế FIR vì cho phép điều chỉnh mức suy giảm dải chặn thông qua tham số β, đồng thời hỗ trợ ước lượng chiều dài bộ lọc dựa trên yêu cầu thiết kế. Điều này giúp quá trình thiết kế bộ lọc trở nên linh hoạt và có cơ sở định lượng rõ ràng.

2.3 Biểu diễn fixed-point trong bộ lọc FIR

Khi triển khai bộ lọc FIR trên hệ thống số, tín hiệu và hệ số bộ lọc cần được biểu diễn bằng số nhị phân có độ rộng bit hữu hạn. So với biểu diễn floating-point, biểu diễn fixed-point có chi phí phần cứng thấp hơn, tốc độ xử lý cao hơn và phù hợp với các hệ thống xử lý tín hiệu số có tài nguyên giới hạn [4].

2.3.1 Biểu diễn số bù hai

Số bù hai là cách biểu diễn số nguyên có dấu phổ biến trong các hệ thống số. Với N bit, số bù hai biểu diễn giá trị trong khoảng:

$$-2\text{^}\{N-1\} \le x \le 2\text{^}\{N-1\} - 1 \quad (2.16)$$

Bit có trọng số cao nhất đóng vai trò bit dấu. Giá trị 0 biểu thị số dương hoặc bằng 0, còn giá trị 1 biểu thị số âm. Biểu diễn bù hai phù hợp cho các phép cộng, trừ và nhân trong chuỗi tính toán của bộ lọc FIR.

2.3.2 Định dạng Q15

Định dạng Q15 sử dụng 16 bit, gồm 1 bit dấu và 15 bit phần thập phân. Định dạng này biểu diễn các giá trị thực trong khoảng:

$$-1 \le x < 1 \quad (2.17)$$

với độ phân giải:

$$\Delta = 2^{-15} \quad (2.18)$$

Quá trình lượng tử hóa hệ số FIR từ số thực sang Q15 được thực hiện theo công thức [3]:

$$h_{Q15}[n] = \text{round}(h[n] \cdot 2^{-15}) \quad (2.19)$$

Trong đó, h[n] là hệ số thực và h_Q15[n] là hệ số sau lượng tử hóa sang định dạng Q15. Định dạng Q15 thường được sử dụng trong các hệ thống fixed-point vì vừa đảm bảo độ phân giải tương đối cao, vừa phù hợp với dữ liệu 16 bit trong phần cứng số.

2.3.3 Sai số lượng tử hóa hệ số

Do fixed-point chỉ có số bit hữu hạn, quá trình lượng tử hóa sẽ tạo ra sai số giữa hệ số thực và hệ số biểu diễn trên phần cứng. Sai số lượng tử hóa hệ số được xác định bởi:

$$e_{q[n]} = h[n] - \frac{h_{Q15}[n]}{2^{-15}} \quad (2.20)$$

Sai số này có thể làm đáp ứng tần số thực tế lệch so với đáp ứng thiết kế ban đầu. Vì vậy, cần đánh giá lại đáp ứng tần số sau khi lượng tử hóa hệ số trước khi triển khai bộ lọc trên phần cứng số. Việc lựa chọn độ rộng bit phù hợp là yếu tố quan trọng để cân bằng giữa độ chính xác tính toán và tài nguyên phần cứng.

2.4 Tổng quan về FPGA trong xử lý tín hiệu số

FPGA (Field Programmable Gate Array) là vi mạch tích hợp có khả năng tái cấu hình, được xây dựng từ các khối logic lập trình được và mạng liên kết có thể cấu hình. Sau khi được nạp file cấu hình, FPGA hoạt động như một mạch số chuyên dụng để thực hiện các chức năng như xử lý tín hiệu, điều khiển, truyền thông và giao tiếp ngoại vi.

Đúng như tên gọi *Field Programmable*, FPGA có thể được lập trình hoặc cập nhật chức năng ngay sau khi sản xuất. Điều này khác với ASIC, vốn có cấu trúc phần cứng cố định sau khi chế tạo. Khi cần thay đổi chức năng, ASIC phải trải qua quy trình thiết kế và sản xuất mới, trong khi FPGA chỉ cần nạp lại file cấu hình mà không phải thay đổi phần cứng.

Khác với CPU thực hiện lệnh theo trình tự, FPGA hiện thực trực tiếp các khối logic bằng phần cứng. Nhờ đó, nhiều phép toán và nhiều khối chức năng có thể hoạt động đồng thời, giúp FPGA đặc biệt phù hợp với các bài toán xử lý song song, xử lý thời gian thực và giao tiếp với nhiều tín hiệu ngoại vi.

Trong lĩnh vực xử lý tín hiệu số, FPGA có nhiều ưu điểm nổi bật. Khả năng xử lý song song cho phép các phép toán như nhân, cộng, dịch dữ liệu và cộng tích lũy được thực hiện đồng thời, từ đó nâng cao hiệu năng xử lý. Bên cạnh đó, FPGA có thể được thiết kế theo kiến trúc pipeline, giúp dữ liệu được xử lý liên tục với độ trễ thấp và tốc độ ổn định, đáp ứng yêu cầu của các ứng dụng thời gian thực.

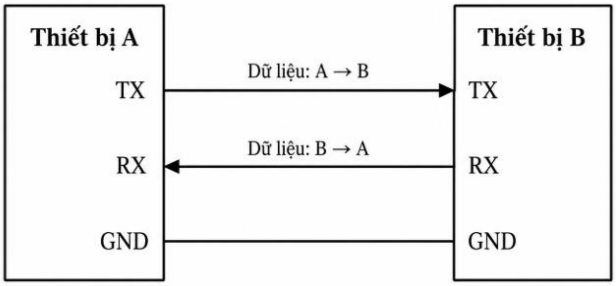
Ngoài hiệu năng xử lý, khả năng tái cấu hình cũng là một ưu điểm quan trọng của FPGA. Người thiết kế có thể dễ dàng sửa đổi thuật toán, cấu trúc phần cứng hoặc các tham số xử lý bằng cách cập nhật file cấu hình. Điều này giúp rút ngắn thời gian phát triển, thuận lợi cho quá trình nghiên cứu, kiểm chứng và nâng cấp hệ thống.

2.5 Tổng quan giao tiếp UART

2.5.1 Khái niệm UART

UART (Universal Asynchronous Receiver/Transmitter) là phương thức truyền thông nối tiếp không đồng bộ, được dùng phổ biến trong vi điều khiển, FPGA và các hệ thống nhúng. Ở phía truyền, dữ liệu song song được đóng gói và truyền từng bit qua đường TX; ở phía nhận, chuỗi bit trên đường RX được giải mã về dữ liệu song song. Hai phía phải thống nhất tốc độ baud, số bit dữ liệu, bit chẵn lẻ và số bit dừng [5].

Trong kết nối cơ bản, chân TX của thiết bị này nối với chân RX của thiết bị kia và ngược lại; hai thiết bị dùng chung GND để thống nhất mức logic. UART có thể hoạt động ở chế độ song công toàn phần, bán song công hoặc đơn công tùy theo yêu cầu hệ thống.



Hình 2.8 Sơ đồ kết nối UART giữa hai thiết bị

UART là bộ thu phát không đồng bộ đa năng, có chức năng chuyển đổi dữ liệu giữa dạng song song và dạng nối tiếp. Ở phía truyền, dữ liệu song song được đóng gói thành khung UART và truyền từng bit qua đường TX. Ở phía nhận, chuỗi bit nối tiếp trên đường RX được giải mã và chuyển trở lại thành dữ liệu song song để cung cấp cho khối xử lý.

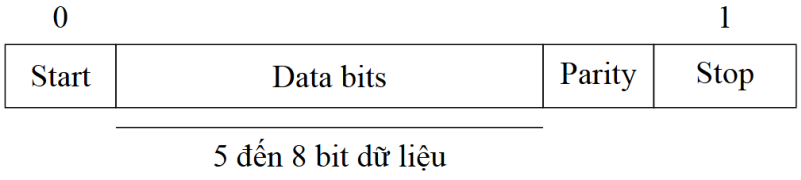
Khác với các giao thức đồng bộ, UART không sử dụng đường clock chung giữa hai thiết bị. Vì vậy, bên phát và bên thu phải được cấu hình thống nhất các thông số truyền, đặc biệt là tốc độ baud.

Trong kết nối UART, chân TX của thiết bị này được nối với chân RX của thiết bị kia và ngược lại. UART có thể hoạt động ở chế độ song công toàn phần, bán song công hoặc đơn công, tùy theo yêu cầu truyền thông của hệ thống.

2.5.2 Khung dữ liệu UART

Dữ liệu UART được truyền theo từng khung truyền, gồm bit bắt đầu, các bit dữ liệu, bit chẵn lẻ (tùy chọn) và bit dừng. Trong các hệ thống nhúng, cấu hình 8N1 được sử dụng phổ biến, bao gồm 1 bit bắt đầu, 8 bit dữ liệu, không sử dụng bit chẵn lẻ và 1 bit dừng.

Khi đường truyền ở trạng thái nghỉ, tín hiệu ở mức logic cao. Quá trình truyền bắt đầu khi bên phát kéo đường truyền xuống mức thấp để tạo bit bắt đầu. Sau đó, các bit dữ liệu được truyền theo thứ tự từ bit có trọng số thấp đến bit có trọng số cao (LSB first). Nếu được sử dụng, bit chẵn lẻ giúp phát hiện một số lỗi truyền đơn giản. Cuối cùng, một hoặc hai bit dừng ở mức logic cao đưa đường truyền trở về trạng thái nghỉ và chuẩn bị cho khung truyền tiếp theo [2].



Hình 2.9 Cấu trúc khung dữ liệu trong giao tiếp UART

2.5.3 Nguyên lý hoạt động UART

Quá trình truyền bắt đầu khi UART Transmitter nhận dữ liệu song song, bổ sung bit bắt đầu, bit chẵn lẻ (nếu có) và bit dừng để tạo thành khung UART hoàn chỉnh. Khung dữ liệu sau đó được truyền nối tiếp từng bit qua đường TX theo tốc độ baud đã cấu hình.

Ở phía nhận, UART Receiver theo dõi đường RX để phát hiện bit bắt đầu, sau đó lấy mẫu các bit dữ liệu tại những thời điểm xác định và kiểm tra bit dừng nhằm xác nhận tính hợp lệ của khung truyền. Khi quá trình nhận hoàn tất, dữ liệu được chuyển về dạng song song và xuất đến khối xử lý bên trong hệ thống.

Trong thực tế, UART Receiver và UART Transmitter thường được thiết kế bằng máy trạng thái hữu hạn (FSM) để điều khiển tuần tự các bước truyền và nhận dữ liệu, giúp hệ thống hoạt động ổn định và dễ kiểm chứng.

2.5.4 Baudrate và thời điểm lấy mẫu

Baudrate là tốc độ truyền dữ liệu của UART, thường được tính bằng số bit truyền trong một giây. Vì UART là giao tiếp không đồng bộ, baudrate là thông số quan trọng để bên phát và bên thu xác định đúng thời gian của từng bit dữ liệu.

Thời gian của một bit trong UART được xác định theo công thức:

$$T_{bit} = \frac{1}{\text{Baudrate}} \tag{2.21}$$

Trong đó, T_bit là thời gian truyền một bit và Baudrate là tốc độ truyền dữ liệu.

Trong các hệ thống số sử dụng clock hệ thống, thời gian một bit thường được quy đổi thành số chu kỳ clock theo công thức:

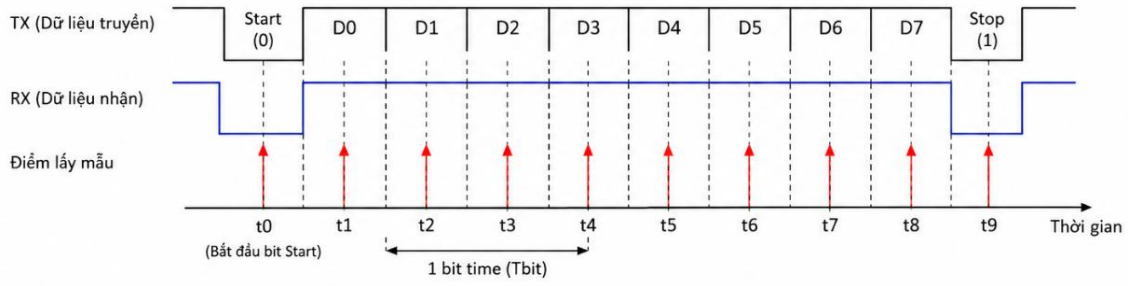
$$\text{DELAY_FRAMES} = \frac{f_{clk}}{\text{Baudrate}} \tag{2.22}$$

Trong đó, f_clk là tần số clock hệ thống. Giá trị DELAY_FRAMES cho biết số chu kỳ clock tương ứng với thời gian truyền một bit UART.

Để lấy mẫu tại trung tâm bit, bộ nhận thường chờ một nửa thời gian bit sau khi phát hiện bit bắt đầu. Giá trị này được xác định theo công thức:

$$\text{HALF_DELAY} = \text{DELAY_FRAMES} / 2 \tag{2.23}$$

Sau khi phát hiện cạnh xuống của bit bắt đầu, bộ nhận đợi HALF_DELAY chu kỳ clock để lấy mẫu tại trung tâm bit đầu tiên. Sau đó, các bit dữ liệu tiếp theo được lấy mẫu sau mỗi DELAY_FRAMES chu kỳ clock. Cách lấy mẫu tại trung tâm bit giúp tăng độ ổn định của quá trình nhận dữ liệu, đặc biệt khi có sai lệch nhỏ giữa baudrate của bên phát và bên thu.

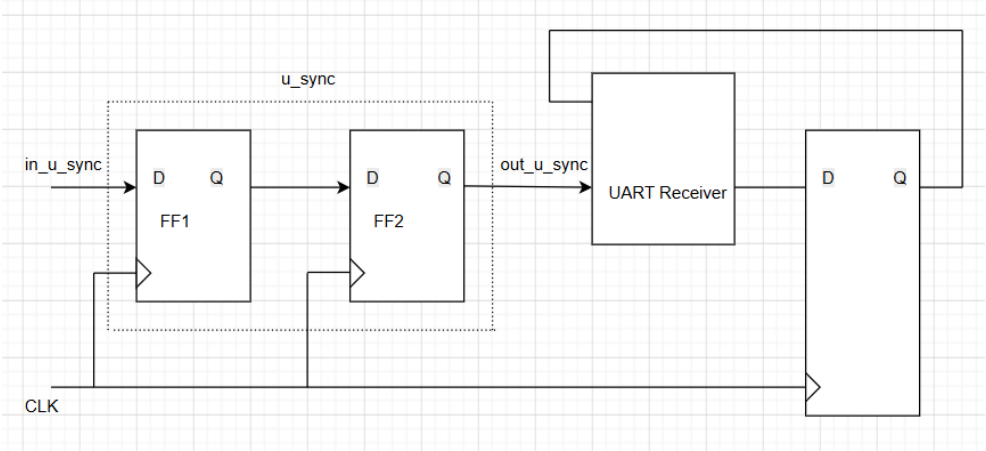


Hình 2.10 Minh họa thời điểm lấy mẫu dữ liệu UART tại trung tâm bit

2.5.5 Mạch đồng bộ trong UART

Trong các hệ thống FPGA, tín hiệu UART RX từ bên ngoài không đồng bộ với clock hệ thống nên có thể gây ra hiện tượng metastability nếu đưa trực tiếp vào các mạch tuần tự.

Để giảm nguy cơ này, tín hiệu RX thường được đưa qua hai tầng Flip-Flop đồng bộ trước khi vào UART Receiver. Flip-Flop thứ nhất lấy mẫu tín hiệu bất đồng bộ từ bên ngoài, trong khi Flip-Flop thứ hai giúp ổn định tín hiệu trước khi đưa vào phần logic xử lý. Mạch đồng bộ không làm thay đổi dữ liệu UART mà chỉ đưa tín hiệu RX về cùng miền clock với hệ thống, giúp quá trình nhận dữ liệu ổn định hơn.



Hình 2.11 Mạch đồng bộ hai tầng Flip-Flop cho tín hiệu UART

2.6 Giao tiếp UART giữa FPGA và máy tính

Trong các hệ thống FPGA, UART thường được sử dụng để trao đổi dữ liệu với máy tính thông qua bộ chuyển đổi USB-UART. Bộ chuyển đổi này thực hiện chuyển đổi dữ liệu từ chuẩn USB sang tín hiệu UART ở mức logic phù hợp để FPGA có thể truyền và nhận dữ liệu.

Giao tiếp UART giữa FPGA và máy tính là kết nối điểm – điểm, trong đó chân TX của thiết bị này được nối với chân RX của thiết bị kia và ngược lại, đồng thời hai thiết bị sử dụng chung đường GND để đảm bảo thống nhất mức logic. Ở chiều truyền từ máy tính đến FPGA, dữ liệu UART được đồng bộ trước khi đưa vào UART Receiver để giải mã. Ngược lại, dữ liệu xử lý trên FPGA được UART Transmitter đóng gói thành khung UART và truyền về máy tính thông qua bộ chuyển đổi USB-UART.

Để truyền nhận chính xác, hai phía phải được cấu hình cùng tốc độ baud, số bit dữ liệu, bit chẵn lẻ và số bit dừng. Trong các hệ thống FPGA, FIFO thường được sử dụng để cân bằng tốc độ giữa khối xử lý và giao tiếp UART, giúp hạn chế mất dữ liệu và đảm bảo đúng thứ tự truyền.

Trong đề tài này, UART được sử dụng để truyền 200 hệ số FIR từ máy tính xuống FPGA và nhận kết quả lọc từ FPGA để phục vụ quá trình kiểm chứng. Giao tiếp này đáp ứng tốt yêu cầu của hệ thống nhờ cấu trúc đơn giản, dễ triển khai và độ tin cậy cao.

CHƯƠNG 3 THIẾT KẾ HỆ THỐNG

3.1 Yêu cầu thiết kế hệ thống

Từ mục tiêu thiết kế bộ lọc FIR thông thấp trên FPGA có khả năng cấu hình lại hệ số qua UART, hệ thống cần đáp ứng các yêu cầu sau:

Về dữ liệu, hệ số bộ lọc và dữ liệu đầu vào được biểu diễn dưới dạng số nguyên có dấu 16 bit. Do UART truyền dữ liệu theo từng byte 8 bit, hệ thống cần ghép hai byte liên tiếp thành dữ liệu 16 bit khi nạp hệ số. Kết quả sau lọc có độ rộng 41 bit, được mở rộng dấu lên 48 bit và tách thành sáu byte 8 bit trước khi truyền về máy tính.

Về kiến trúc phần cứng, bộ lọc FIR được thiết kế với tối đa 200 tap và hiện thực theo kiến trúc nhân – cộng tích lũy tuần tự nhằm tiết kiệm tài nguyên FPGA. FIFO được sử dụng làm bộ đệm giữa khối FIR và khối truyền UART để lưu tạm kết quả và đảm bảo dữ liệu được truyền đúng thứ tự.

Về giao tiếp, hệ thống sử dụng UART để truyền hệ số từ máy tính xuống FPGA và truyền kết quả xử lý từ FPGA trở lại máy tính. Quá trình truyền nhận phải đảm bảo đúng thứ tự byte, đúng định dạng dữ liệu và bảo toàn giá trị số có dấu.

Về kiểm chứng, kết quả xử lý trên FPGA được so sánh với kết quả mô phỏng trên Python/Streamlit. Sai số giữa hai kết quả được đánh giá trong phạm vi ảnh hưởng của quá trình lượng tử hóa hệ số và dữ liệu.

Về môi trường triển khai, hệ thống được hiện thực trên kit FPGA Tang Nano 9K, sử dụng ngôn ngữ Verilog HDL và được kiểm tra bằng mô phỏng RTL trước khi đánh giá ở mức hệ thống.

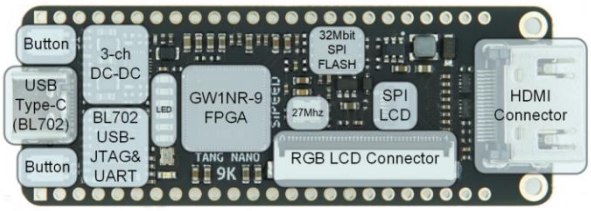
3.2 Phần cứng và công cụ sử dụng

Để triển khai hệ thống lọc FIR thông thấp trên FPGA và giao tiếp với máy tính qua UART, đề tài sử dụng board Sipeed Tang Nano 9K làm nền tảng phần cứng chính. Board tích hợp chip FPGA Gowin GW1NR-9 cùng các tài nguyên như bộ nhớ, khối DSP, clock 27 MHz, USB-JTAG và USB-UART, đáp ứng yêu cầu thiết kế và kiểm chứng hệ thống.

Trong hệ thống, FPGA thực hiện các khối chức năng gồm UART Receiver, UART Transmitter, Byte Packer, Coefficient Loader, ROM Controller, FIR Filter, FIFO Buffer và Unbyte Packer. Máy tính được sử dụng để tạo hệ số FIR, truyền hệ số xuống FPGA, nhận kết quả xử lý và so sánh với dữ liệu tham chiếu.

3.2.1 Board FPGA Sipeed Tang Nano 9K

Board Sipeed Tang Nano 9K sử dụng FPGA Gowin GW1NR-9, phù hợp cho các ứng dụng thiết kế số, xử lý tín hiệu và giao tiếp ngoại vi. Board tích hợp sẵn clock 27 MHz, USB-JTAG, USB-UART, GPIO và các ngoại vi cơ bản, thuận tiện cho quá trình phát triển và kiểm thử.



Hình 3.1 Hình ảnh thực tế của board Sipeed Tang Nano 9K

Với kiến trúc FIR MAC tuần tự được sử dụng trong đề tài, tài nguyên logic, bộ nhớ và các khối DSP của FPGA đáp ứng tốt yêu cầu triển khai mà không cần sử dụng quá nhiều phần tử nhân song song.

Bảng 3.1 *Thông số phần cứng chính của board Sipeed Tang Nano 9K*

Hạng mục	Thông số
Tên board	Sipeed Tang Nano 9K
FPGA	Gowin GW1NR-9
Logic element	8640 LUT4
Flip-Flop	6480 FF
Block SRAM	468 Kbits
Shadow SRAM	17,280 bits
User Flash	608 Kbits
PSRAM	64 Mbits
SPI Flash	32 Mbits
Multiplier	20 khối nhân 18 × 18
PLL	2
Clock onboard	27 MHz
Giao tiếp nạp chương trình	USB-JTAG thông qua chip BL702
Giao tiếp UART	USB-UART thông qua chip BL702
Ngoại vi hỗ trợ	LED, nút nhấn, GPIO, header mở rộng

Clock 27 MHz trên board được sử dụng làm clock hệ thống cho toàn bộ thiết kế, đồng thời là cơ sở tạo các xung điều khiển phục vụ khối UART và bộ lọc FIR.

3.2.2 Kết nối phần cứng sử dụng trong hệ thống

Các tín hiệu chính của hệ thống gồm clock, reset, UART RX/TX, nút nhấn và LED trạng thái. Các tín hiệu này được khai báo trong module top và ánh xạ tới các chân vật lý của FPGA thông qua file ràng buộc chân.

Bảng 3.2 *Các tín hiệu chính sử dụng trong module top_system*

Tín hiệu	Loại tín hiệu	Chức năng
clk_27mhz	Input	Clock hệ thống 27 MHz cấp cho toàn bộ thiết kế.
rst_n	Input	Tín hiệu reset mức thấp, dùng để khởi tạo lại hệ thống.
rx_pin	Input	Chân nhận dữ liệu UART từ máy tính xuống FPGA.
tx_pin	Output	Chân truyền dữ liệu UART từ FPGA về máy tính.

Tín hiệu	Loại tín hiệu	Chức năng
key2	Input	Nút nhấn dùng để chọn hoặc thay đổi chế độ kiểm thử dữ liệu.
led_don	Output	LED báo trạng thái hoàn tất một quá trình trong hệ thống.
led_mode	Output	LED báo chế độ dữ liệu hoặc trạng thái hoạt động đang được chọn.

Trong đó, rx_pin và tx_pin đảm nhiệm truyền nhận dữ liệu giữa FPGA và máy tính thông qua UART. Các LED được sử dụng để quan sát nhanh trạng thái hoạt động của hệ thống trong quá trình thực nghiệm.

3.2.3 Công cụ phần mềm sử dụng

Đề tài sử dụng Verilog HDL để mô tả phần cứng và Gowin IDE để tổng hợp, ánh xạ chân, định tuyến và nạp thiết kế xuống FPGA. Quá trình mô phỏng được thực hiện trên Xilinx ISE/ISim nhằm kiểm tra chức năng trước khi triển khai trên phần cứng.

Ở phía máy tính, Python và giao diện Web được sử dụng để thiết kế hệ số FIR, tạo dữ liệu kiểm thử, giao tiếp UART và hiển thị kết quả, phục vụ quá trình đối chiếu giữa mô phỏng và thực nghiệm.

Bảng 3.3 *Công cụ phần mềm sử dụng*

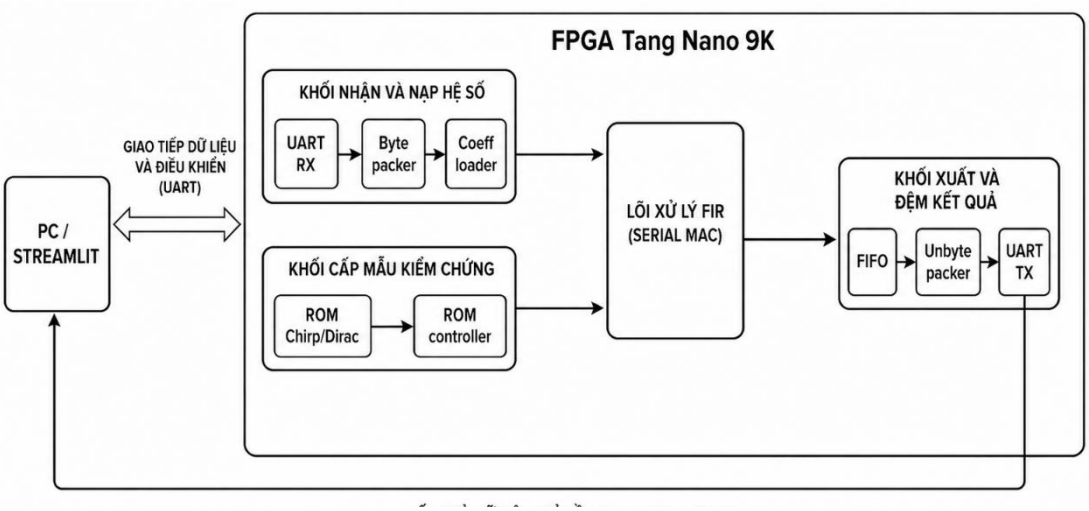
Công cụ	Vai trò
Verilog HDL	Mô tả các khối phần cứng như UART RX/TX, FIR, ROM, FIFO và các khối điều khiển.
Gowin IDE	Tổng hợp, ánh xạ chân, định tuyến và nạp thiết kế xuống FPGA Tang Nano 9K.
Xilinx ISE/ISim	Mô phỏng HDL và kiểm tra chức năng của hệ thống ở mức mô phỏng.
Python	Tạo hệ số FIR, tạo dữ liệu kiểm thử, giao tiếp UART và xử lý dữ liệu nhận về.
Giao diện Web	Hỗ trợ nhập thông số, hiển thị kết quả tham chiếu và kết quả nhận từ FPGA.
USB-JTAG	Nạp bitstream xuống FPGA.
USB-UART	Truyền nhận dữ liệu giữa máy tính và FPGA.
File HEX	Lưu dữ liệu kiểm thử hoặc dữ liệu khởi tạo cho ROM trong quá trình mô phỏng và kiểm chứng.

Nhờ sự kết hợp giữa phần cứng Tang Nano 9K và các công cụ phát triển, hệ thống đáp ứng đầy đủ các yêu cầu về thiết kế, mô phỏng, triển khai và kiểm chứng bộ lọc FIR trên FPGA.

3.3 Kiến trúc tổng thể hệ thống

3.3.1 Sơ đồ khối tổng thể

Hệ thống gồm hai phần chính là PC/Streamlit và FPGA Tang Nano 9K. PC/Streamlit thực hiện thiết kế hệ số FIR, truyền hệ số xuống FPGA qua UART và nhận kết quả sau xử lý để kiểm chứng. FPGA đảm nhiệm việc nhận hệ số, cấp dữ liệu đầu vào, thực hiện phép lọc FIR và truyền kết quả trở lại máy tính.

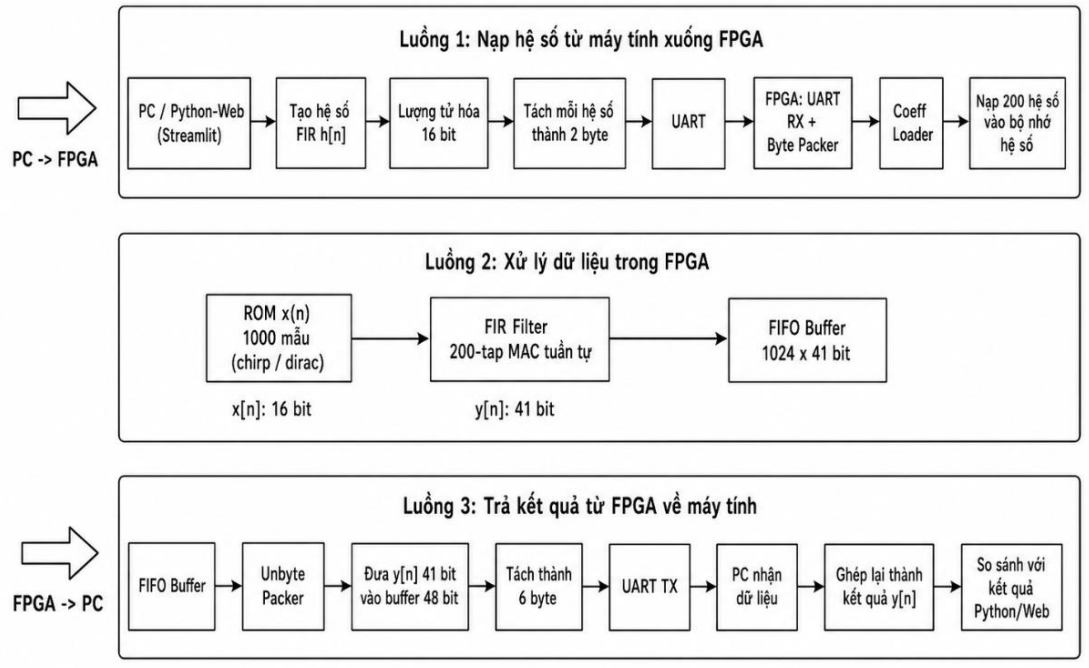


Hình 3.2 Sơ đồ khối tổng thể hệ thống lọc FIR trên FPGA

Bên trong FPGA, hệ thống được chia thành bốn nhóm khối chính. Khối nhận và nạp hệ số nhận dữ liệu từ PC qua UART, ghép byte và ghi hệ số vào FIR core. Khối cấp mẫu kiểm chứng dùng ROM để cung cấp tín hiệu Chirp hoặc Dirac cho FIR. Lõi xử lý FIR thực hiện phép lọc theo kiến trúc MAC tuần tự 200 tap. Khối xuất và đệm kết quả lưu tạm kết quả bằng FIFO, tách dữ liệu thành byte và truyền về PC qua UART TX.

3.3.2 Luồng dữ liệu FPGA - PC

Luồng dữ liệu giữa máy tính và FPGA gồm hai hướng: truyền hệ số từ máy tính xuống FPGA và truyền kết quả xử lý từ FPGA về máy tính.



Hình 3.3 Luồng dữ liệu giữa FPGA và máy tính

Ở chiều truyền xuống, Streamlit tạo các hệ số FIR 16 bit và gửi qua UART theo từng byte. FPGA nhận dữ liệu thông qua UART Receiver, Byte Packer ghép hai byte thành một hệ số 16 bit và Coefficient Loader ghi tuần tự vào bộ nhớ hệ số. Khi nạp đủ 200 hệ số, tín hiệu `coeffs_loaded` được kích hoạt để bắt đầu quá trình xử lý.

Tiếp theo, ROM Controller cấp tuần tự các mẫu dữ liệu đầu vào từ ROM cho FIR Filter. Bộ lọc thực hiện phép nhân – cộng tích lũy theo kiến trúc MAC tuần tự và tạo ra kết quả 41 bit kèm tín hiệu `fir_y_valid`.

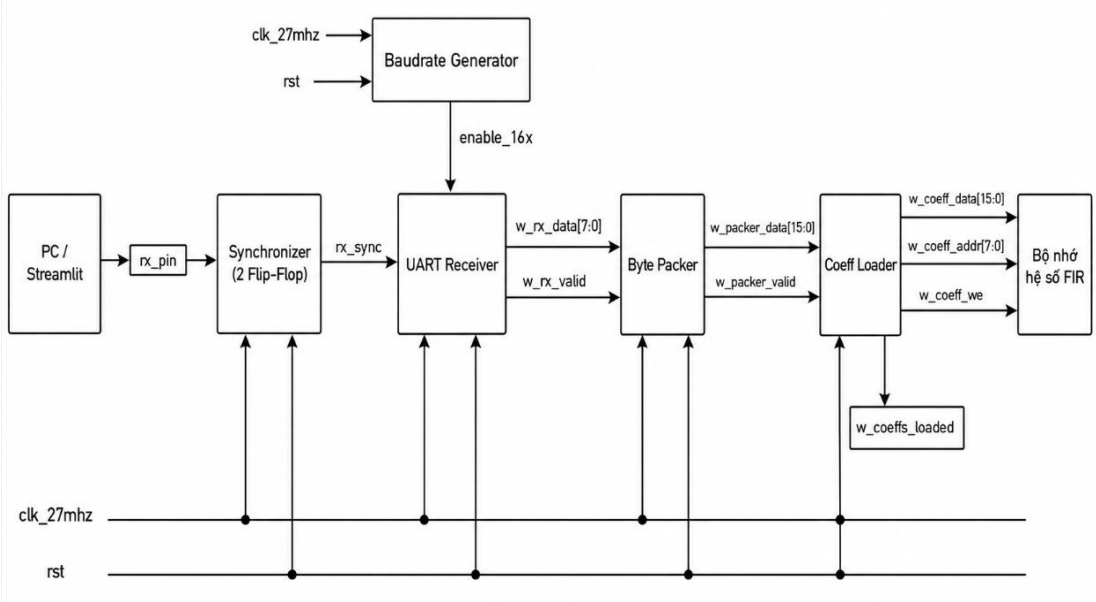
Kết quả sau lọc được lưu vào FIFO Buffer. Khi khối truyền sẵn sàng, Unbyte Packer đọc dữ liệu từ FIFO, tách thành sáu byte và chuyển sang UART Transmitter để truyền về máy tính.

Tại máy tính, các byte nhận được được ghép lại thành giá trị đầu ra của bộ lọc FIR và so sánh với kết quả tham chiếu trên Python/Web. Quá trình này cho phép kiểm chứng toàn bộ hệ thống, từ truyền nhận UART, nạp hệ số, xử lý FIR đến truyền dữ liệu đầu ra.

3.4 Thiết kế khối UART Receiver

Khối UART Receiver được thiết kế nhằm tiếp nhận dữ liệu hệ số FIR từ máy tính thông qua giao tiếp UART và chuyển đổi dữ liệu nối tiếp thành dữ liệu song song 8 bit để các khối phía sau xử lý. Trong hệ thống, mỗi hệ số FIR có độ rộng 16 bit nên hai byte liên tiếp sẽ được ghép thành một hệ số hoàn chỉnh trước khi ghi vào bộ nhớ hệ số. Đồng thời, tín hiệu RX từ bên ngoài FPGA được đồng bộ với clock hệ thống nhằm đảm bảo quá trình nhận dữ liệu ổn định và tin cậy..

Cụm UART Receiver gồm các khối Baudrate Generator, Synchronizer, UART Receiver, Byte Packer và Coeff Loader.



Hình 3.4 Sơ đồ khối kết nối của khối UART Receiver trong hệ thống

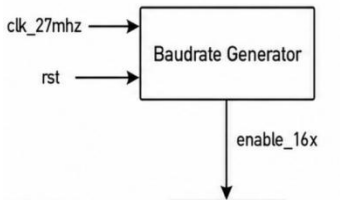
Luồng dữ liệu bắt đầu từ máy tính và được truyền vào FPGA qua chân rx_pin. Tín hiệu này trước tiên được đưa qua Synchronizer để đồng bộ với clock hệ thống, sau đó chuyển đến UART Receiver để nhận dữ liệu nối tiếp và chuyển thành các byte song song 8 bit.

Các byte hợp lệ được đưa sang Byte Packer để ghép hai byte liên tiếp thành một hệ số FIR 16 bit. Sau đó, Coeff Loader ghi tuần tự các hệ số vào bộ nhớ của bộ lọc FIR. Khi đủ 200 hệ số, tín hiệu coeffs_loaded được kích hoạt để báo hoàn tất quá trình nạp hệ số.

Bên cạnh luồng dữ liệu chính, Baudrate Generator tạo tín hiệu enable_16x cung cấp cho UART Receiver nhằm xác định thời điểm lấy mẫu dữ liệu theo chuẩn UART.

Thiết kế bộ tạo xung lấy mẫu UART

Khối Baudrate Generator được thiết kế nhằm tạo tín hiệu enable_16x cho khối UART Receiver. Đây là tín hiệu cho phép lấy mẫu dữ liệu UART với tần số gấp 16 lần tốc độ baud, giúp bộ nhận xác định đúng thời điểm kiểm tra bit bắt đầu và lấy mẫu các bit dữ liệu trong khung truyền.



Hình 3.5 Sơ đồ khối bộ tạo xung lấy mẫu UART

Trong hệ thống, clock chính là 27 MHz và tốc độ truyền UART được chọn là 9600 baud. Tần số lấy mẫu được xác định: f_enable = 9600 × 16 = 153600 Hz

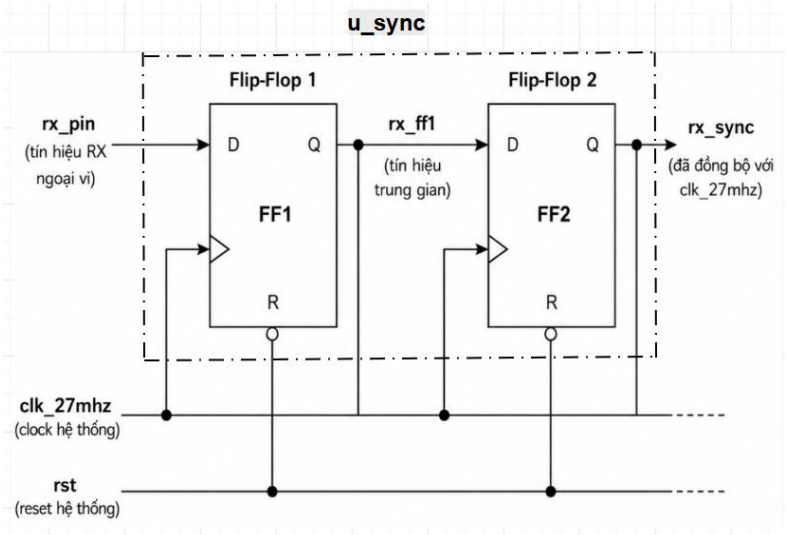
Số chu kỳ clock tương ứng: N = 27000000 / 153600 ≈ 175.78

Do đó, bộ đếm được thiết kế đếm từ 0 đến 175. Khi đạt giá trị 175, tín hiệu enable_16x được tạo trong một chu kỳ clock rồi bộ đếm trở về 0 để bắt đầu chu kỳ mới.

Khối Baudrate Generator có hai tín hiệu vào là clk_27mhz và rst, một tín hiệu ra là enable_16x. Khi rst được kích hoạt, bộ đếm được khởi tạo về 0; trong chế độ hoạt động bình thường, bộ đếm tăng theo từng chu kỳ clock và phát xung enable_16x định kỳ cho UART Receiver.

3.4.1 Thiết kế mạch đồng bộ tín hiệu RX

Tín hiệu UART RX từ máy tính được đưa vào FPGA qua chân rx_pin. Do đây là tín hiệu không đồng bộ với clock hệ thống clk_27mhz, nếu đưa trực tiếp vào UART Receiver có thể gây hiện tượng metastability, làm ảnh hưởng đến quá trình lấy mẫu dữ liệu. Vì vậy, tín hiệu rx_pin được đưa qua mạch đồng bộ hai tầng Flip-Flop trước khi xử lý.



Hình 3.6 Mạch đồng bộ tín hiệu RX dùng hai Flip-Flop

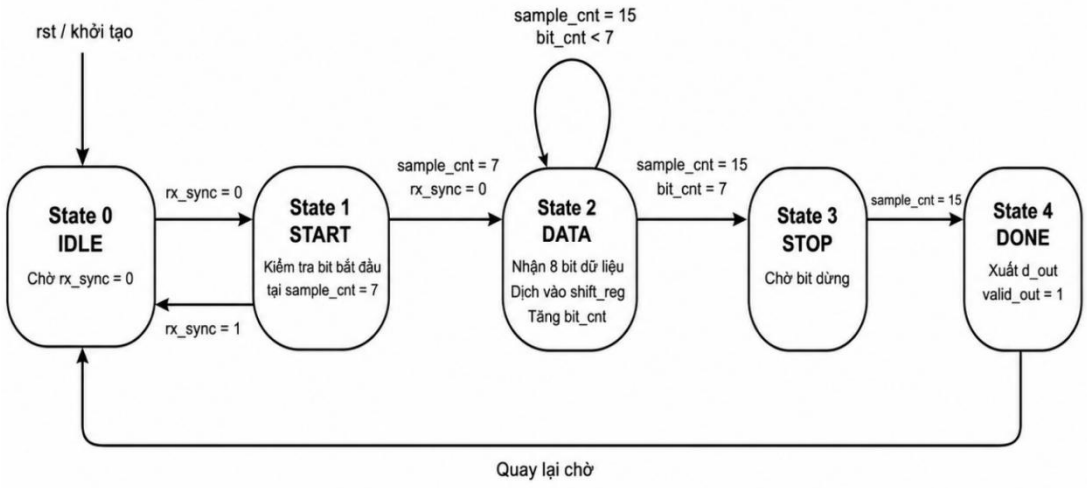
Mạch đồng bộ gồm hai Flip-Flop mắc nối tiếp và hoạt động theo cùng clock hệ thống. Flip-Flop thứ nhất lấy mẫu tín hiệu rx_pin, sau đó Flip-Flop thứ hai lấy lại dữ liệu từ tầng thứ nhất để tạo tín hiệu rx_sync ổn định trước khi đưa vào UART Receiver.

Khi rst được kích hoạt, hai thanh ghi được khởi tạo ở mức logic 1, tương ứng với trạng thái nghỉ của đường truyền UART. Trong quá trình hoạt động, tín hiệu RX được đồng bộ qua hai tầng Flip-Flop theo từng cạnh lên của clock.

Mạch đồng bộ không làm thay đổi dữ liệu UART mà chỉ đưa tín hiệu RX về cùng miền clock với hệ thống, giúp giảm nguy cơ metastability và nâng cao độ tin cậy của quá trình nhận dữ liệu.

3.4.2 Máy trạng thái FSM của UART Receiver

Máy trạng thái FSM của khối UART Receiver được thiết kế để điều khiển quá trình nhận một byte dữ liệu từ đường truyền UART. Trong module Verilog, FSM sử dụng tín hiệu enable_16x để xác định thời điểm lấy mẫu, đồng thời dùng hai bộ đếm sample_cnt và bit_cnt để quản lý vị trí lấy mẫu trong từng bit và số bit dữ liệu đã nhận.



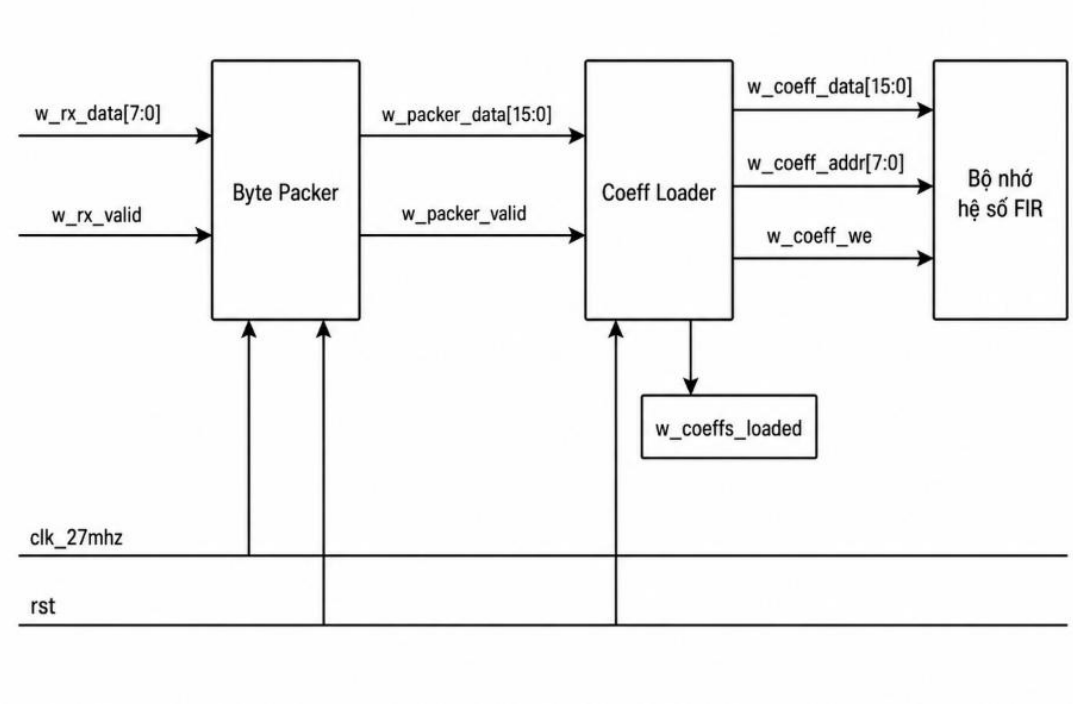
Hình 3.7 Máy trạng thái FSM của khối UART Receiver

FSM gồm năm trạng thái. State 0: Chờ đường truyền ở trạng thái nghỉ. Khi phát hiện RX chuyển từ mức cao xuống mức thấp, FSM chuyển sang trạng thái kiểm tra bit bắt đầu. State 1: Chờ đến giữa bit bắt đầu. Khi sample_cnt = 7, tín hiệu RX được kiểm tra lại. Nếu vẫn ở mức 0 thì bit bắt đầu hợp lệ và FSM chuyển sang nhận dữ liệu; ngược lại quay về trạng thái chờ. State 2: Nhận lần lượt 8 bit dữ liệu. Cứ mỗi 16 xung enable_16x, một bit được lấy mẫu và dịch vào thanh ghi shift_reg. Sau mỗi lần lấy mẫu, bit_cnt tăng thêm một đơn vị. State 3: Chờ hết thời gian của bit dừng. Sau đó dữ liệu trong shift_reg được đưa ra d_out và tín hiệu valid_out được kích hoạt để báo một byte đã nhận hoàn tất. State 4: Kết thúc quá trình nhận và quay trở lại trạng thái chờ để tiếp tục nhận byte kế tiếp.

Cấu trúc FSM này giúp việc nhận dữ liệu UART diễn ra tuần tự, đúng định dạng khung truyền và đồng bộ với tốc độ baudrate đã thiết lập.

Ghép byte và nạp hệ số FIR

Sau khi UART Receiver nhận được từng byte dữ liệu 8 bit, dữ liệu này chưa thể ghi trực tiếp thành hệ số FIR vì mỗi hệ số trong hệ thống có độ rộng 16 bit. Vì vậy, hai khối Byte Packer và Coeff Loader được sử dụng để ghép byte và nạp hệ số vào bộ nhớ hệ số của bộ lọc FIR.



Hình 3.8 Sơ đồ ghép byte và nạp hệ số FIR

Byte Packer lưu byte đầu tiên vào thanh ghi tạm. Khi nhận được byte thứ hai, hai byte được ghép thành một từ dữ liệu 16 bit theo dạng {temp_high, d_in}. Sau khi ghép hoàn tất, tín hiệu pack_valid được kích hoạt để báo hệ số đã sẵn sàng.

Coeff Loader nhận dữ liệu 16 bit từ Byte Packer và ghi tuần tự vào bộ nhớ hệ số thông qua các tín hiệu coeff_data, coeff_addr và coeff_we. Sau mỗi lần ghi, địa chỉ ghi được tăng lên để chuẩn bị cho hệ số tiếp theo.

Trong hệ thống, bộ lọc FIR sử dụng 200 hệ số, tương ứng với các địa chỉ từ 0 đến 199. Sau khi hệ số cuối cùng được ghi, tín hiệu coeffs_loaded được kích hoạt để báo hoàn tất quá trình nạp hệ số và cho phép hệ thống chuyển sang giai đoạn xử lý dữ liệu.

Nguyên lý hoạt động của khối UART Receiver

Quá trình hoạt động bắt đầu khi tín hiệu rx_pin được đồng bộ qua Synchronizer để tạo tín hiệu rx_sync. Baudrate Generator tạo xung enable_16x làm tín hiệu điều khiển quá trình lấy mẫu trong UART Receiver. Sau khi phát hiện bit bắt đầu, UART Receiver nhận đủ 8 bit dữ liệu và xuất ra một byte song song kèm tín hiệu valid_out.

Các byte nhận được được chuyển đến Byte Packer để ghép thành hệ số FIR 16 bit, sau đó Coeff Loader ghi tuần tự các hệ số vào bộ nhớ của bộ lọc FIR. Khi toàn bộ 200 hệ số đã

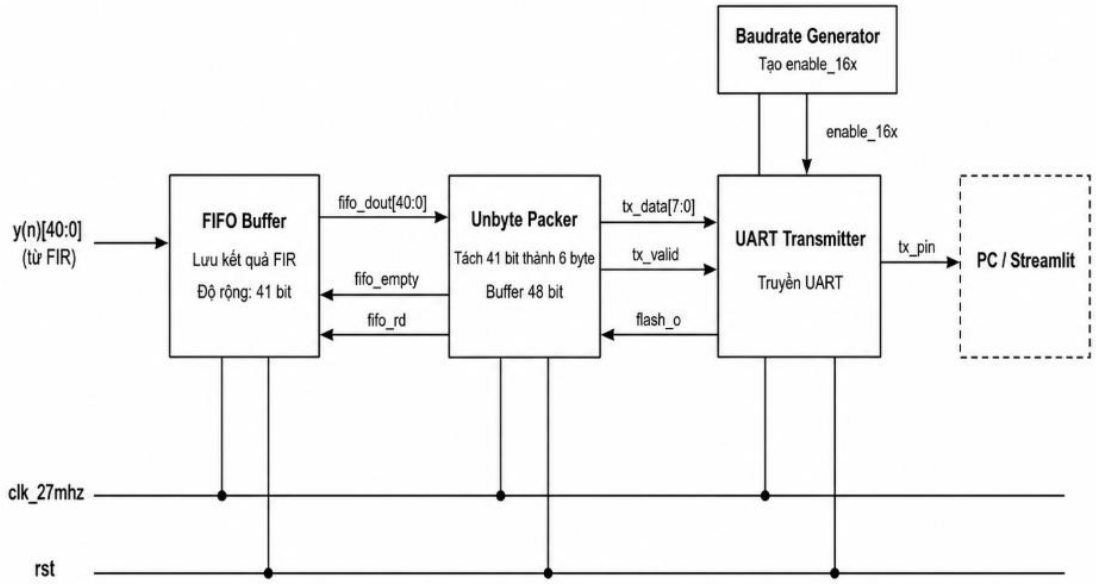
được nạp, tín hiệu coeffs_loaded được kích hoạt, cho phép hệ thống bắt đầu đọc dữ liệu từ ROM và thực hiện quá trình lọc FIR.

3.5 Thiết kế khối UART Transmitter

Khối UART Transmitter được thiết kế để truyền kết quả sau khi lọc FIR từ FPGA về máy tính thông qua giao tiếp UART phục vụ quá trình kiểm chứng. Do đầu ra của bộ lọc FIR có độ rộng 41 bit, trong khi UART chỉ truyền từng byte 8 bit, dữ liệu được lưu tạm trong FIFO rồi đưa qua khối Unbyte Packer để mở rộng lên 48 bit và tách thành 6 byte trước khi truyền. Việc truyền dữ liệu được điều khiển theo chuẩn UART 9600 baud bằng tín hiệu `enable_16x`, đảm bảo máy tính nhận đúng thứ tự và định dạng dữ liệu.

Sơ đồ khối kết nối UART Transmitter

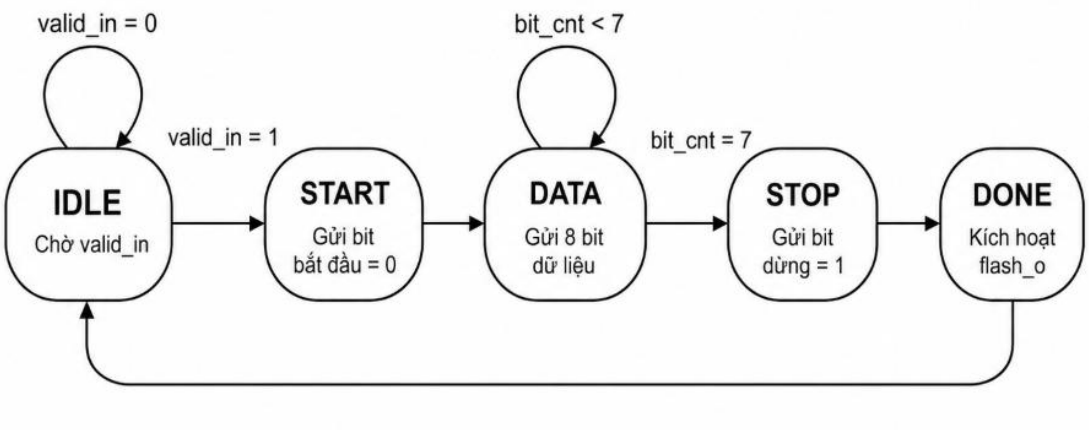
Sơ đồ khối kết nối của UART Transmitter được xây dựng nhằm mô tả luồng truyền dữ liệu kết quả từ FPGA về máy tính. Trong hệ thống, cụm truyền UART gồm các khối chính: FIFO Buffer, Unbyte Packer, UART Transmitter và Baudrate Generator.



Hình 3.9 Sơ đồ khối kết nối của khối UART Transmitter trong hệ thống

Sau khi bộ lọc FIR hoàn thành xử lý một mẫu dữ liệu, kết quả 41 bit được ghi vào FIFO để tách biệt tốc độ xử lý của FIR với tốc độ truyền UART. Khi FIFO có dữ liệu, Unbyte Packer đọc từng mẫu, mở rộng lên 48 bit rồi tách thành sáu byte liên tiếp. Các byte này được truyền tuần tự đến UART Transmitter thông qua các tín hiệu tx_data và tx_valid. UART Transmitter tiếp nhận từng byte, đóng gói theo khung truyền gồm một bit bắt đầu, tám bit dữ liệu và một bit dừng, sau đó truyền ra chân tx_pin. Sau khi truyền xong mỗi byte, tín hiệu flash_o được kích hoạt để báo hoàn tất và cho phép Unbyte Packer chuyển sang truyền byte tiếp theo.

Máy trạng thái FSM của UART Transmitter



Hình 3.10 Máy trạng thái FSM của khối UART Transmitter

Máy trạng thái hữu hạn (FSM) của UART Transmitter điều khiển toàn bộ quá trình truyền một byte dữ liệu qua giao tiếp UART. Ban đầu, khối truyền ở trạng thái chờ với đường tx_pin được giữ ở mức logic 1. Khi tín hiệu valid_in được kích hoạt, byte dữ liệu được nạp vào thanh ghi truyền và FSM bắt đầu quá trình phát khung UART. Trước tiên, tx_pin được đưa xuống mức 0 để tạo bit bắt đầu, tiếp theo tám bit dữ liệu được truyền tuần tự theo từng xung enable_16x, trong khi bộ đếm bit_cnt theo dõi số bit đã phát. Sau khi truyền đủ tám bit dữ liệu, FSM phát bit dừng bằng cách đưa tx_pin trở lại mức logic 1 trong thời gian một bit. Khi hoàn tất, tín hiệu flash_o được kích hoạt trong một chu kỳ clock để báo truyền xong một byte, sau đó FSM quay về trạng thái chờ và sẵn sàng nhận byte tiếp theo.

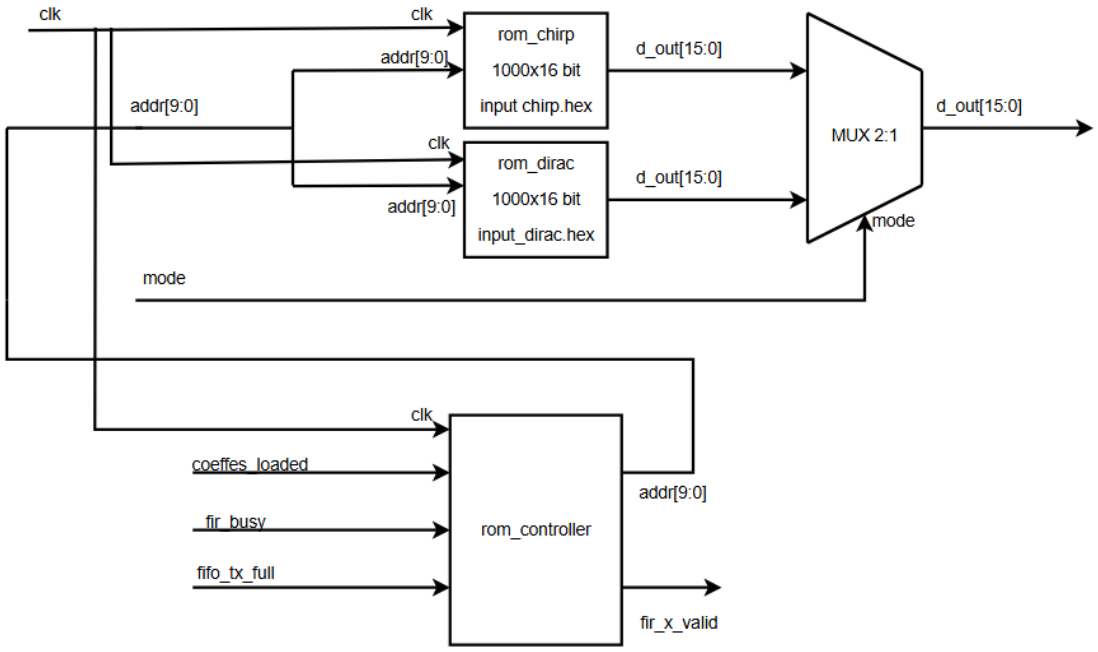
Nguyên lý hoạt động

Trong quá trình hoạt động, mỗi kết quả 41 bit từ bộ lọc FIR được ghi vào FIFO trước khi truyền. Khi FIFO không rỗng, Unbyte Packer đọc dữ liệu, mở rộng lên 48 bit và tách thành sáu byte để truyền tuần tự sang UART Transmitter. Mỗi byte được đóng gói thành một khung UART và phát ra chân **tx_pin** theo tốc độ 9600 baud. Sau khi truyền xong một byte, tín hiệu **flash_o** được tạo để yêu cầu truyền byte kế tiếp. Quá trình này được lặp lại cho đến khi truyền hết sáu byte của một kết quả FIR. Nếu FIFO vẫn còn dữ liệu, hệ thống tiếp tục đọc và truyền các kết quả tiếp theo cho đến khi bộ đệm rỗng, bảo đảm toàn bộ dữ liệu đầu ra của bộ lọc FIR được gửi đầy đủ về máy tính để kiểm chứng.

3.6 Thiết kế khối rom và rom_controller

Khối ROM có nhiệm vụ lưu trữ trước các mẫu tín hiệu kiểm chứng và cung cấp lần lượt từng mẫu cho FIR core trong quá trình kiểm thử. Hai loại tín hiệu được sử dụng gồm tín hiệu Chirp và tín hiệu Dirac, được lưu trong hai bộ nhớ rom độc lập. Việc sử dụng rom giúp bảo đảm dữ liệu đầu vào luôn cố định, thuận tiện cho việc so sánh kết quả giữa mô phỏng Python và thực nghiệm trên FPGA. Và khối rom_controller điều phối quá trình đọc mẫu từ ROM,chỉ cấp mẫu mới khi hệ số đã nạp xong, FIR core không bận và FIFO kết quả chưa đầy.

Khối rom và rom_controller được thiết kế với sơ đồ như Hình 3.11:



Hình 3.11 Sơ đồ khối ROM

Cấu trúc khối ROM cấp mẫu đầu vào gồm bốn thành phần chính: rom_controller, rom_chirp, rom_dirac và bộ chọn MUX 2:1. Trong đó, rom_chirp lưu các mẫu tín hiệu Chirp, rom_dirac lưu các mẫu tín hiệu Dirac, còn rom_controller điều phối quá trình đọc mẫu và cấp tín hiệu hợp lệ cho FIR core. Bộ chọn MUX 2:1 có độ rộng 16 bit được sử dụng để lựa chọn dữ liệu đầu ra giữa hai ROM dựa trên tín hiệu mode.

Mỗi ROM được thiết kế với kích thước 1000x16 bit, tương ứng với 1000 mẫu tín hiệu, mỗi mẫu có độ rộng 16 bit. Do cần truy xuất 1000 mẫu nên địa chỉ đọc ROM có độ rộng 10 bit, cho phép tạo địa chỉ từ 0 đến 999. Dữ liệu trong ROM được khởi tạo bằng lệnh $readmemh, trong đó rom_chirp đọc dữ liệu từ file input_chirp.hex, còn rom_dirac đọc dữ liệu từ file input_dirac.hex.

Bảng 3.4. Bảng tín hiệu của khối rom và rom_controller

Tín hiệu	Chiều	Chức năng
coeffs_loaded	Input	Báo quá trình nạp hệ số FIR đã hoàn tất. Khi tín hiệu này bằng 1, rom_controller mới bắt đầu cấp mẫu cho khối FIR.
fir_busy	Input	Báo FIR đang xử lý mẫu hiện tại. rom_controller chỉ cấp mẫu mới khi fir_busy = 0.
fifo_tx_full	Input	Báo FIFO truyền kết quả đã đầy. Khi FIFO đầy, quá trình cấp mẫu mới được tạm dừng để tránh mất dữ liệu đầu ra.
mode	Input	Chọn loại tín hiệu đầu vào. mode = 0 chọn tín hiệu Chirp, mode = 1 chọn tín hiệu Dirac.
rom_addr[9:0]	Output	Địa chỉ đọc ROM do rom_controller tạo ra, dùng để truy xuất các mẫu từ địa chỉ 0 đến 999.
fir_x_valid	Output	Báo mẫu đầu vào hợp lệ cho FIR . Tín hiệu này được bật trong một chu kỳ clock khi cấp mẫu mới.

d_out[15:0]	Output	Dữ liệu đầu ra sau MUX 2:1, được đưa đến cổng x_data[15:0] của khối FIR .
Clk		Clock hệ thống.

Hai ROM rom_chirp và rom_dirac được đọc đồng thời bằng cùng một địa chỉ rom_addr[9:0]. Dữ liệu đầu ra của hai ROM lần lượt là d_chirp[15:0] và d_dirac[15:0]. Tín hiệu mode quyết định dữ liệu nào được đưa ra ngoài thông qua MUX 2:1. Khi mode = 0, đầu ra d_out[15:0] lấy dữ liệu từ rom_chirp. Khi mode = 1, đầu ra d_out[15:0] lấy dữ liệu từ rom_dirac. Về mặt chức năng, bộ chọn này là một MUX 2:1 có độ rộng 16 bit. Trong sơ đồ RTL sau tổng hợp, MUX 16 bit có thể được biểu diễn thành 16 MUX 1 bit, mỗi MUX tương ứng với một bit của bus dữ liệu.

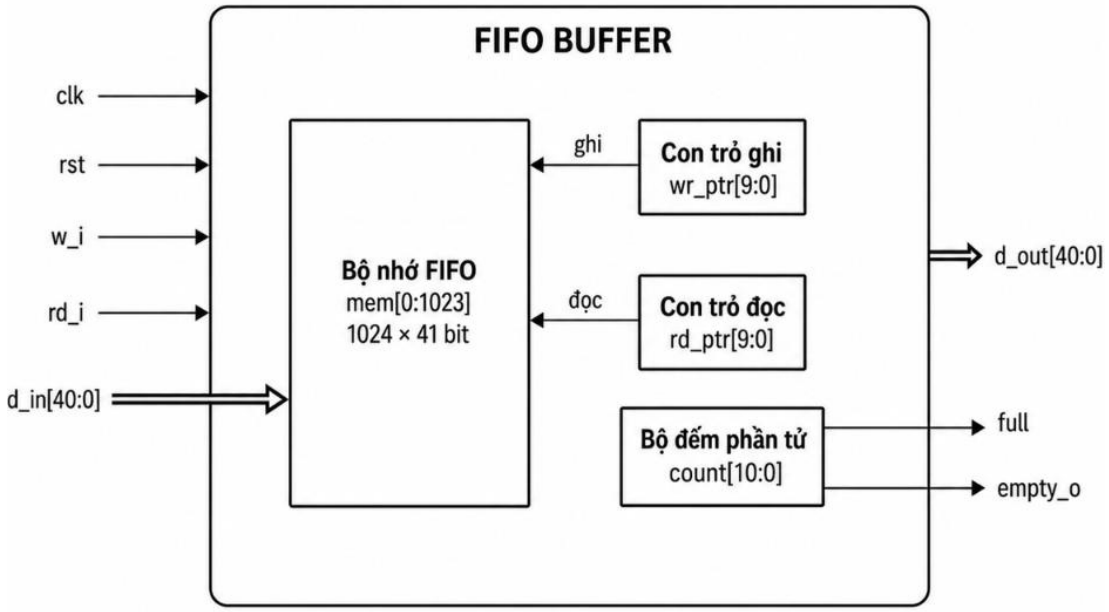
Khối rom_controller có nhiệm vụ tạo địa chỉ đọc ROM và phát tín hiệu fir_x_valid để báo mẫu đầu vào hợp lệ cho FIR core. Khối điều khiển chỉ bắt đầu hoạt động khi hệ số FIR đã được nạp xong, tức tín hiệu coeffs_loaded = 1. Trước khi cấp mẫu mới, rom_controller kiểm tra hai điều kiện: FIR core không bận và FIFO truyền kết quả chưa đầy. Hai điều kiện này được xác định thông qua các tín hiệu fir_busy và fifo_tx_full. Nếu fir_busy = 0 và fifo_tx_full = 0, bộ điều khiển sẽ phát fir_x_valid = 1 trong một chu kỳ clock để FIR core nhận mẫu hiện tại.

Trình tự hoạt động của rom_controller gồm ba trạng thái chính. Ở trạng thái đầu tiên, bộ điều khiển chờ điều kiện cấp mẫu thỏa mãn. Khi FIR core không bận và FIFO truyền chưa đầy, tín hiệu fir_x_valid được bật để báo mẫu hợp lệ. Ở trạng thái tiếp theo, fir_x_valid được hạ xuống và địa chỉ rom_addr được tăng thêm một đơn vị nếu chưa đọc hết 1000 mẫu. Nếu rom_addr đạt giá trị 999, biến done được bật để kết thúc quá trình cấp mẫu. Ở trạng thái cuối, bộ điều khiển chờ FIR core xử lý xong mẫu hiện tại trước khi quay lại trạng thái chờ cấp mẫu mới.

Với cách thiết kế này, dữ liệu kiểm chứng được cấp cho FIR core theo trình tự ổn định và đồng bộ với trạng thái xử lý của hệ thống. Đồng thời, việc sử dụng hai ROM riêng kết hợp với MUX 2:1 giúp hệ thống có thể lựa chọn nhanh giữa tín hiệu Chirp và Dirac mà không cần thay đổi lại cấu trúc phần cứng.

3.7 Thiết kế khối FIFO

Khối FIFO được thiết kế để đệm dữ liệu kết quả từ FIR core trước khi đưa sang khối tách byte và truyền về PC thông qua UART TX. Trong hệ thống, đầu ra của FIR core có độ rộng 41 bit, trong khi UART truyền dữ liệu theo từng byte 8 bit. Vì vậy, FIFO đóng vai trò trung gian giúp lưu tạm kết quả FIR, tránh mất dữ liệu khi tốc độ tạo kết quả và tốc độ truyền UART không hoàn toàn giống nhau.



Hình 3.12 Sơ đồ khối FIFO buffer

Bảng 3.5. Thành phần bên trong khối FIFO

Thành phần	Độ rộng	Chức năng
mem[0:1023]	1024 × 41 bit	Bộ nhớ lưu dữ liệu FIFO.
wr_ptr[9:0]	10 bit	Con trỏ ghi, xác định vị trí sẽ ghi dữ liệu tiếp theo.
rd_ptr[9:0]	10 bit	Con trỏ đọc, xác định vị trí sẽ đọc dữ liệu tiếp theo.
count[10:0]	11 bit	Đếm số phần tử hiện có trong FIFO.

Nguyên lý ghi dữ liệu vào FIFO được thực hiện dựa trên tín hiệu w_i và trạng thái full. Khi w_i = 1 và FIFO chưa đầy, dữ liệu d_in[40:0] được ghi vào vị trí mem[wr_ptr]. Sau khi ghi, con trỏ wr_ptr tăng thêm một đơn vị để trỏ đến vị trí ghi kế tiếp. Nếu wr_ptr đã đạt vị trí cuối cùng của FIFO, con trỏ sẽ quay về 0 để tiếp tục ghi theo cơ chế vòng.

Nguyên lý đọc dữ liệu khỏi FIFO được thực hiện dựa trên tín hiệu rd_i và trạng thái empty_o. Khi rd_i = 1 và FIFO không rỗng, dữ liệu tại vị trí mem[rd_ptr] được đưa ra d_out[40:0]. Sau khi đọc, con trỏ rd_ptr tăng thêm một đơn vị. Tương tự con trỏ ghi, khi rd_ptr đạt đến vị trí cuối cùng của FIFO, nó sẽ quay về 0 để tiếp tục đọc theo cơ chế vòng.

Hai tín hiệu full và empty_o được tạo từ bộ đếm count. Khi count = DEPTH, FIFO được xem là đầy và tín hiệu full được bật. Khi count = 0, FIFO được xem là rỗng và tín hiệu empty_o được bật. Biến count tăng khi chỉ có thao tác ghi hợp lệ, giảm khi chỉ có thao tác đọc hợp lệ. Nếu thao tác ghi và đọc cùng xảy ra hợp lệ trong cùng một chu kỳ clock, số phần tử trong FIFO không thay đổi. Trong hệ thống tổng thể, FIFO nhận dữ liệu từ FIR core khi y_valid = 1. Dữ liệu sau khi lưu trong FIFO sẽ được đọc bởi khối unbyte_packer để chuyển từ dữ liệu 41 bit thành 6 byte trước khi truyền qua UART TX. Ngoài ra, tín hiệu full của FIFO còn được đưa về rom_controller dưới dạng fifo_tx_full. Khi FIFO đầy, rom_controller sẽ tạm dừng cấp mẫu mới cho FIR core, nhờ đó tránh trường hợp FIR tiếp tục tạo kết quả trong khi khối truyền UART chưa xử lý kịp.

Với cách thiết kế này, FIFO giúp tách biệt tốc độ giữa khối xử lý FIR và khối truyền dữ liệu UART, đồng thời bảo đảm dữ liệu kết quả không bị ghi tràn hoặc đọc sai trong quá trình vận hành hệ thống.

3.8 Thiết kế bộ lọc FIR thông thấp

Bộ lọc FIR trong hệ thống được thiết kế theo hướng cấu hình lại được: hệ số lọc không được cố định trong mã nguồn Verilog mà được lưu trong bộ nhớ có thể ghi trên FPGA và có thể thay đổi thông qua giao tiếp UART mà không cần tổng hợp và nạp lại bitstream. Mục này trình bày các thông số thiết kế, phương pháp nạp hệ số động và kiến trúc phần cứng hiện thực bộ lọc.

3.8.1 Thông số thiết kế

Bộ lọc được hiện thực trên phần cứng với số tap cố định là 200 tương ứng với 200 thanh ghi dịch và 200 vị trí bộ nhớ hệ số. Số tap thực tế được xác định bởi phương pháp cửa sổ Kaiser tại phía máy tính. Khi số tap tính toán nhỏ hơn số tap cố định thì ta sẽ đệm bằng 0 trước khi truyền xuống FPGA, qua đó bảo đảm cấu trúc phần cứng không thay đổi dù bộ lọc thiết kế có chiều dài khác nhau. Các thông số chính được tổng hợp trong Bảng 3.6.

Bảng 3.6 Thông số thiết kế chính của bộ lọc FIR

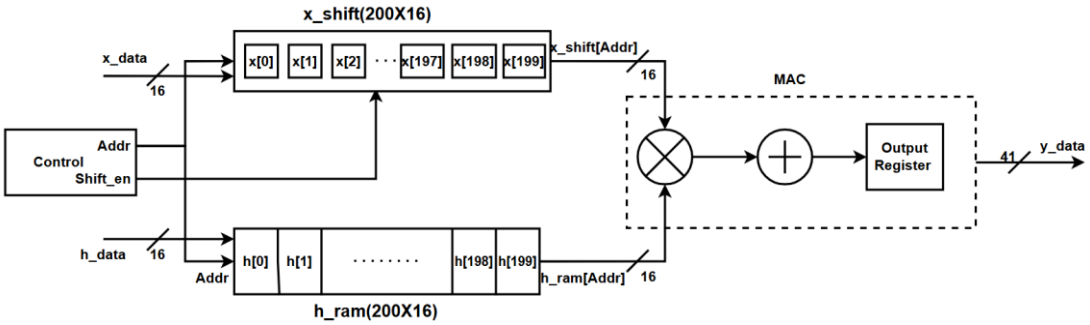
Tham số	Mô tả
Loại bộ lọc	FIR thông thấp
Số tap tối đa	200 (cố định trên phần cứng)
Độ rộng hệ số	16 bit có dấu, định dạng Q15
Độ rộng dữ liệu vào x[n]	16 bit có dấu
Độ rộng đầu ra	41 bit có dấu
Kiến trúc xử lý	MAC tuần tự
Phương pháp thiết kế hệ số	Cửa sổ Kaiser

Việc lựa chọn kiến trúc MAC tuần tự thay vì kiến trúc song song xuất phát từ giới hạn tài nguyên DSP của FPGA Tang Nano 9K, chỉ có 20 khối nhân 18×18, không đủ để hiện thực 200 phép nhân đồng thời. Với kiến trúc tuần tự, mỗi mẫu đầu ra được tính qua 200 chu kỳ MAC liên tiếp, thời gian xử lý một mẫu vẫn nhỏ hơn nhiều so với thời gian truyền một byte qua UART ở baudrate 9600, nên không gây nghẽn dữ liệu trong toàn hệ thống.

3.8.2 Kiến trúc FIR trên FPGA

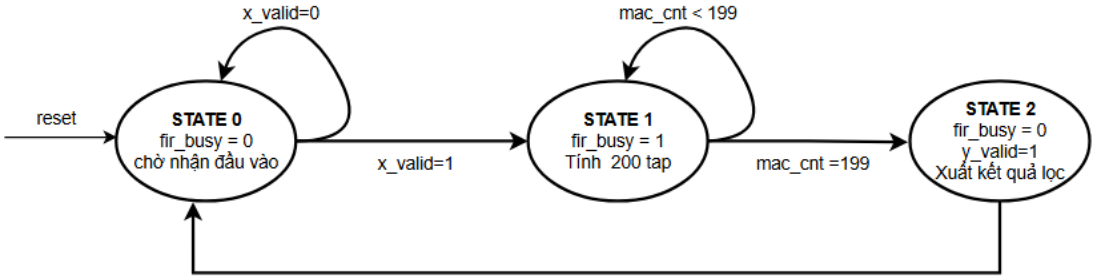
Để phù hợp với tài nguyên phần cứng của FPGA Tang Nano 9K, khối FIR Filter được thiết kế theo kiến trúc MAC tuần tự. Thay vì sử dụng 200 bộ nhân song song cho 200 tap, thiết kế chỉ dùng một đơn vị nhân – cộng tích lũy và thực hiện lần lượt qua từng hệ số. Cách thiết kế này giúp giảm tài nguyên phần cứng nhưng vẫn đáp ứng được tốc độ xử lý của hệ thống.

Về cấu trúc bên trong, khối FIR Filter gồm bốn thành phần chính: bộ nhớ hệ số h_ram, thanh ghi dịch mẫu x_shift, khối nhân – cộng tích lũy MAC và máy trạng thái điều khiển. Hình 3.13 minh họa kiến trúc tổng quát của khối FIR Filter 200-tap dùng MAC tuần tự.

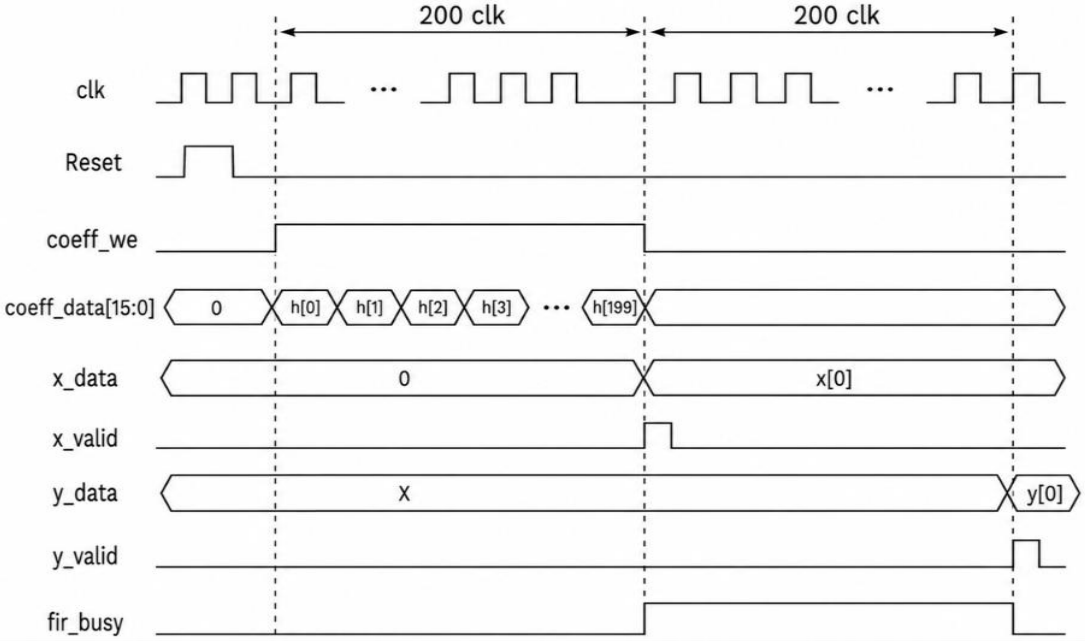


Hình 3.13 Kiến trúc FIR 200-tap MAC tuần tự

Và toàn bộ quá trình tính toán được điều khiển bởi một máy trạng thái ba trạng thái, tương ứng với vòng lặp khởi tạo – tích lũy – xuất kết quả, được minh họa ở Hình 3.14.



Hình 3.14 Máy trạng thái hữu hạn điều khiển khối FIR Filter



Hình 3.15 Sơ đồ timing điều khiển khối FIR

Từ sơ đồ timing có thể thấy quá trình hoạt động gồm hai pha chính. Pha thứ nhất là nạp 200 hệ số vào bộ nhớ hệ số. Pha thứ hai là xử lý mẫu đầu vào. Sau khi hệ số đã nạp xong, khối điều khiển đưa mẫu x[0] vào FIR. Khối FIR giữ trạng thái bận trong quá trình MAC tuần tự và chỉ phát y_valid khi y_data hợp lệ. Cách tổ chức này giúp khối FIR xử lý tuần tự từng mẫu, tiết kiệm tài nguyên phần cứng và vẫn đáp ứng tốc độ truyền nhận của hệ thống.

Bảng 3.7 Các tín hiệu giao tiếp chính của khối FIR Filter

Tín hiệu	Chiều	Ý nghĩa

clk	Input	Clock hệ thống (27 MHz)
rst	Input	Reset toàn khối, mức cao kích hoạt
coeff_data[15:0]	Input	Giá trị hệ số 16 bit từ coeff_loader
coeff_we	Input	Write enable, cho phép ghi hệ số
x_data[15:0]	Input	Mẫu tín hiệu đầu vào 16 bit từ ROM
x_valid	Input	Báo mẫu x_data hợp lệ
y_data[40:0]	Output	Kết quả lọc 41 bit có dấu
y_valid	Output	Báo kết quả y_data hợp lệ
fir_busy	Output	Báo bộ lọc đang bận xử lý hay không

Kết lại mục 3.8 đã trình bày cách thiết kế bộ lọc FIR thông thấp cấu hình lại hệ số trên FPGA. Hệ thống sử dụng số tap phần cứng cố định 200, hệ số 16 bit Q15 và bộ tích lũy 41 bit. Các hệ số được tính ở phía máy tính, đệm đủ 200 phần tử và nạp vào bộ nhớ hệ số h_ram thông qua UART. Nhờ đó, đáp ứng tần số của bộ lọc có thể thay đổi mà không cần nạp lại bitstream.

Về kiến trúc phần cứng, khối FIR dùng MAC tuần tự để tiết kiệm tài nguyên DSP của Tang Nano 9K. Mỗi mẫu được xử lý lần lượt qua 200 tap; tín hiệu x_valid, fir_busy và y_valid đảm nhiệm việc đồng bộ quá trình nhận mẫu, xử lý và xuất kết quả. Cách tổ chức này bảo đảm thiết kế gọn, rõ ràng và phù hợp với mục tiêu kiểm chứng bộ lọc FIR cấu hình lại hệ số qua UART.

3.9 Thiết kế phần mềm PC sử dụng Streamlit

Phần mềm PC được xây dựng bằng Streamlit nhằm hỗ trợ quá trình thiết kế hệ số, mô phỏng đáp ứng, tạo dữ liệu cấu hình và kiểm chứng thực nghiệm cho hệ thống FIR trên FPGA. Trong thiết kế này, các phép tính thiết kế bộ lọc được thực hiện ở phía PC; FPGA chỉ nhận bộ hệ số đã lượng tử hóa và thực hiện phép lọc bằng FIR core. Cách tổ chức này giúp giảm khối lượng tính toán trên FPGA và cho phép thay đổi đặc tính bộ lọc bằng cách tạo lại bộ hệ số ở phía phần mềm.

3.9.1 Cấu trúc giao diện phần mềm

Giao diện phần mềm được tổ chức theo dạng các tab chức năng nhằm tách riêng từng giai đoạn trong quá trình thiết kế và kiểm chứng. Cách chia này giúp người dùng thao tác theo đúng trình tự: nhập thông số, quan sát mô phỏng lý tưởng, tạo tệp cấu hình và kiểm chứng kết quả thực nghiệm từ FPGA.

Bảng 3.8 Cấu trúc các tab chức năng của phần mềm Streamlit

Tab giao diện	Nội dung thiết kế
Thiết kế	Nhập thông số bộ lọc gồm Fs, Fp, Fstop và As. Kiểm tra các điều kiện ràng buộc ban đầu, , tính số tap, beta Kaiser và kích hoạt quá trình thiết kế.

Mô phỏng lý tưởng	Hiển thị đáp ứng tần số của bộ lọc trên thang dB và thang tuyến tính, quan sát dải thông, dải chuyển tiếp, dải chặn và độ gợn.
Tạo tín hiệu thử nghiệm và đóng gói tệp	Tạo tín hiệu Chirp, tín hiệu Dirac, bộ hệ số Q15 và các tệp cấu hình dùng cho FPGA.
Kiểm chứng thực nghiệm	Tải lên kết quả từ FPGA sau đó giải mã và vẽ đồ thị đồng thời đối chiếu với mô phỏng python.

Trong tab thiết kế, phần mềm sử dụng các thành phần nhập liệu như ô nhập số, thanh trượt và các thẻ thông số để hiển thị nhanh số tap, trễ nhóm, định dạng hệ số và tham số beta Kaiser. Phần mềm đồng thời sử dụng trạng thái phiên làm việc để lưu lại thông số đã nhập, kết quả tính toán và trạng thái hoàn thành thiết kế. Nhờ đó, dữ liệu tính toán có thể được dùng tiếp ở các tab mô phỏng, tạo tệp và kiểm chứng mà không cần nhập lại.

3.9.2 Quy trình tính toán và lượng tử hóa hệ số

Quy trình tính toán hệ số bắt đầu từ các thông số do người dùng nhập gồm tần số lấy mẫu Fs, tần số biên dải thông Fp, tần số biên dải chặn Fstop và độ suy hao dải chặn As. Trước khi thiết kế, phần mềm kiểm tra các điều kiện hợp lệ như $Fs > 0$, $Fp > 0$, $Fstop > 0$, $As \geq 21$ dB, $Fp < Fs/2$, $Fstop < Fs/2$ và $Fstop > Fp$. Nếu có thông số không hợp lệ, phần mềm hiển thị thông báo lỗi và không cho phép thực hiện bước tính toán.

Khi các thông số hợp lệ, phần mềm tính độ rộng dải chuyển tiếp chuẩn hóa theo tỉ số giữa Fstop - Fp và Fs/2. Từ giá trị này và yêu cầu suy hao As, hàm kaiserord được sử dụng để xác định số tap N và tham số beta của cửa sổ Kaiser. Nếu N là số chẵn, phần mềm tăng thêm một đơn vị để thu được số tap lẻ, phù hợp với bộ lọc FIR tuyến tính pha loại I. Sau đó, phần mềm kiểm tra N có vượt quá giới hạn phần cứng N_MAX_HW = 200 hay không. Nếu $N > 200$, chương trình báo lỗi để người dùng điều chỉnh lại thông số thiết kế.

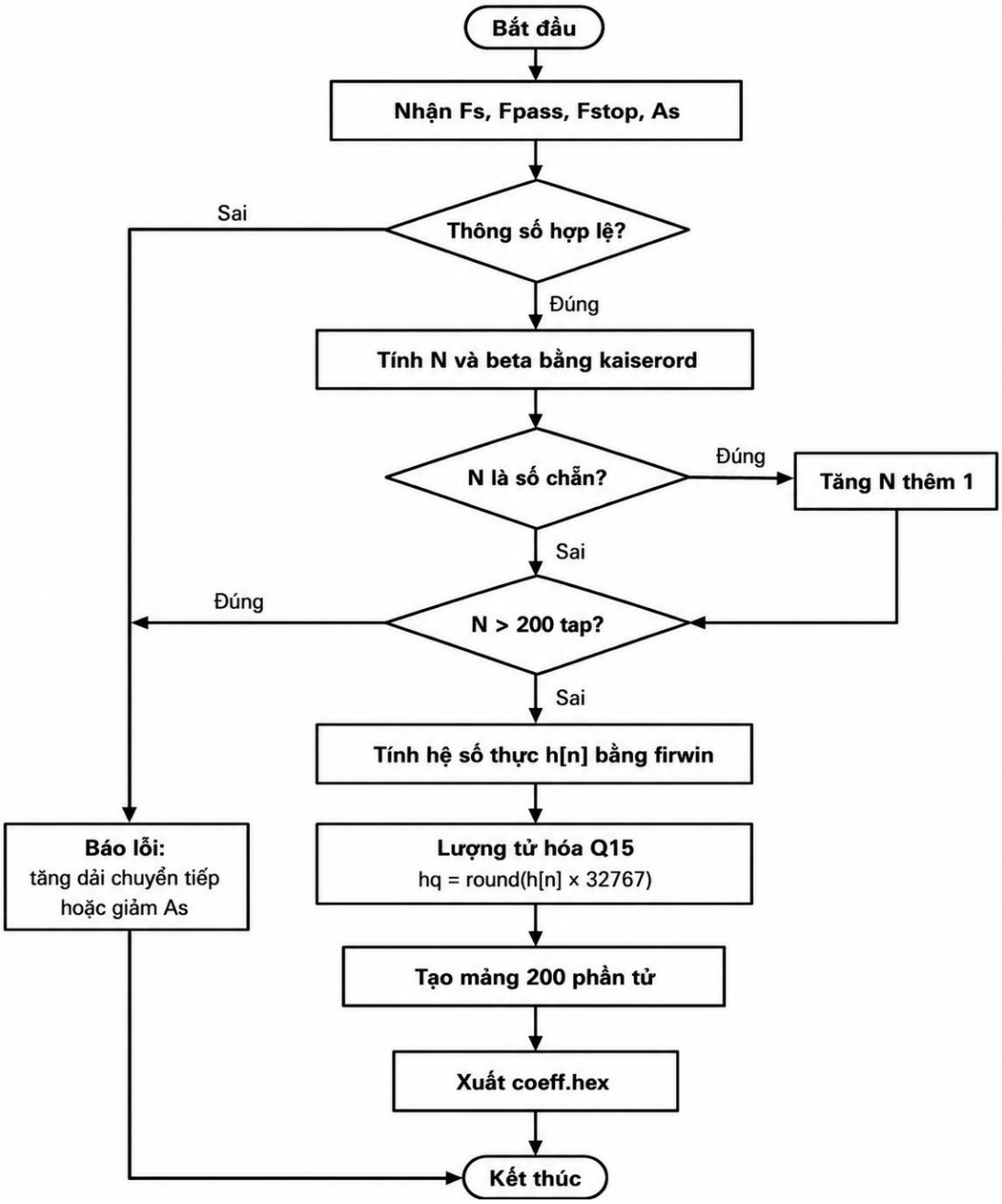
Hệ số FIR dạng số thực được tính bằng hàm firwin với cửa sổ Kaiser. Tần số cắt được chọn tại trung điểm giữa Fp và Fstop:

$$Fc = (Fp + Fstop) / 2$$

Sau khi có hệ số thực h[n], phần mềm lượng tử hóa hệ số sang định dạng Q15 để phù hợp với độ rộng hệ số 16 bit trên FPGA. Hệ số thực được nhân với 32767, làm tròn và giới hạn trong miền biểu diễn cho phép. Công thức lượng tử hóa được sử dụng là:

$$h_Q15[n] = \text{round}(h[n] \times 32767)$$

Sau bước lượng tử hóa, phần mềm tạo một mảng hệ số có độ dài cố định 200 phần tử. Các hệ số thực tế được đặt ở đầu mảng, các vị trí còn lại được đệm bằng 0. Cách chuẩn hóa này bảo đảm dữ liệu gửi xuống FPGA luôn phù hợp với bộ nhớ hệ số h_ram[0:199] trong FIR core.



Hình 3.16 Quy trình tính toán và lượng tử hóa hệ số

3.9.3 Tạo dữ liệu cấu hình và tín hiệu kiểm chứng

Sau khi thiết kế bộ hệ số, phần mềm tạo các tệp phục vụ cấu hình và kiểm chứng hệ thống. Bộ hệ số sau lượng tử hóa được xuất thành tệp coeff.hex, gồm 200 hệ số 16 bit ở dạng hex để truyền xuống FPGA qua UART và ghi vào bộ nhớ hệ số của bộ lọc FIR.

Đồng thời, phần mềm tạo hai tín hiệu kiểm chứng Chirp và Dirac, mỗi tín hiệu gồm 1000 mẫu. Tín hiệu Chirp có tần số biến thiên từ 0 đến Fs/2 để đánh giá đáp ứng thông thấp của bộ lọc, trong khi tín hiệu Dirac gồm một mẫu đầu có biên độ cực đại và các mẫu còn lại bằng 0 để kiểm tra đáp ứng xung.

Sau khi lượng tử hóa sang 16 bit, hai tín hiệu được xuất thành các tệp input_chirp.hex và input_dirac.hex để khởi tạo ROM mẫu đầu vào trên FPGA. Ngoài ra, phần mềm còn tạo

tệp taps_config.vh chứa các tham số thiết kế như số tap, số mẫu, β, As, Fp, Fstop, Fs và số bit lượng tử hóa.

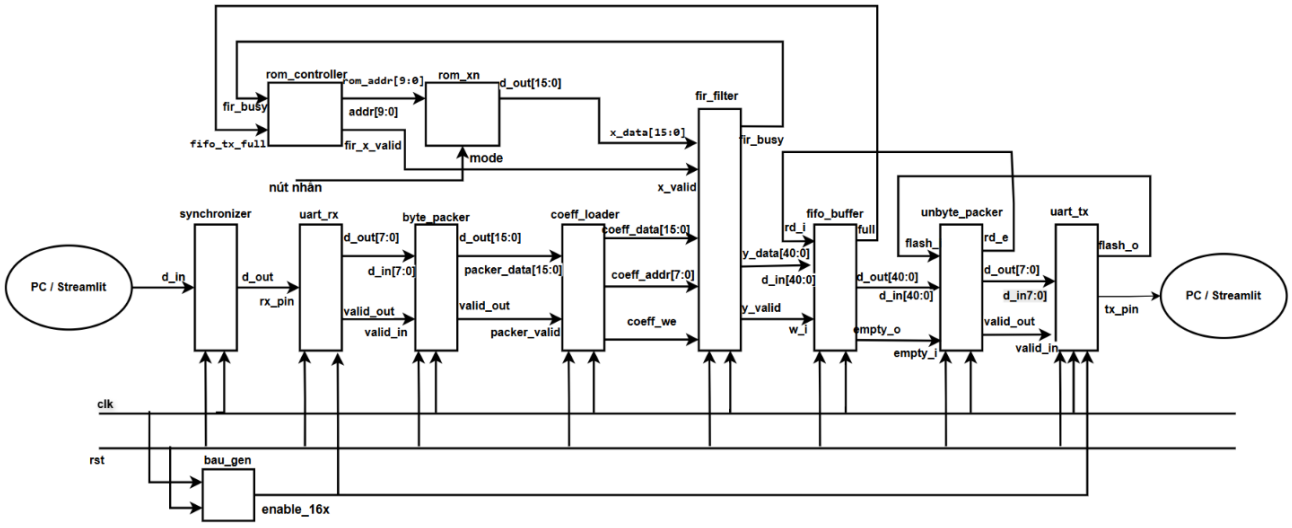
Bảng 3.9. Các tệp cấu hình và kiểm chứng do phần mềm tạo ra

Tệp tạo ra	Vai trò
coeff.hex	Chứa 200 hệ số FIR Q15, dùng để nạp vào RAM hệ số trên FPGA.
input_chirp.hex	Chứa 1000 mẫu tín hiệu Chirp, dùng cho kiểm chứng khả năng lọc thông thấp.
input_dirac.hex	Chứa 1000 mẫu tín hiệu Dirac, dùng cho kiểm chứng đáp ứng xung và đáp ứng tần số.
taps_config.vh	Chứa các tham số thiết kế phục vụ đồng bộ giữa phần mềm và phần cứng.

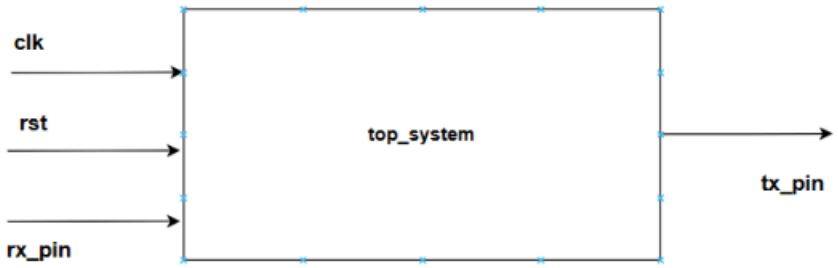
Trong giao diện, các tệp được đóng gói dưới dạng liên kết tải về để người dùng có thể lưu trực tiếp. Việc tạo đồng thời hệ số, tín hiệu kiểm chứng và tệp tham số giúp bảo đảm các dữ liệu dùng cho phần mềm mô phỏng và phần cứng FPGA được tạo từ cùng một bộ thông số thiết kế.

3.10 Tích hợp hệ thống

Sau khi thiết kế riêng từng khối, các module được kết nối tất cả các khối để tạo thành hệ thống hoàn chỉnh. Sơ đồ tích hợp thể hiện rõ các module thật trong hệ thống và các tín hiệu giao tiếp chính giữa chúng.



Hình 3.17 Sơ đồ tích hợp hệ thống



Hình 3.18 Sơ đồ chân vào/ra của module top_system

CHƯƠNG 4 KẾT QUẢ THỰC NGHIỆM

4.1 Kiểm chứng thiết kế bằng mô phỏng RTL trên Xilinx ISE/ISim

4.1.1 Mục đích, thiết lập mô phỏng và dữ liệu kiểm thử

Mô phỏng được thực hiện nhằm kiểm chứng chức năng của hệ thống lọc FIR trước khi triển khai trên FPGA. Testbench được xây dựng để kiểm tra cả từng module và toàn bộ hệ thống. Ở mức module, các khối Baudrate Generator, Synchronizer, UART Receiver, Byte Packer, Coeff Loader, FIR Filter, FIFO Buffer, Unbyte Packer và UART Transmitter được kiểm tra độc lập. Ở mức hệ thống, toàn bộ luồng dữ liệu được kiểm chứng từ quá trình nạp hệ số FIR qua UART, xử lý dữ liệu, lưu kết quả vào FIFO và truyền kết quả trở lại máy tính.

Mục tiêu của quá trình mô phỏng là xác nhận hoạt động đúng của từng khối chức năng, kiểm tra luồng dữ liệu trong toàn hệ thống và đối chiếu kết quả đầu ra của bộ lọc FIR với giá trị tham chiếu.

Mô phỏng được thực hiện trên Xilinx ISE/ISim với clock hệ thống 27 MHz. Khối Baudrate Generator tạo tín hiệu enable_16x phục vụ lấy mẫu UART ở tốc độ 9600 baud theo phương pháp lấy mẫu 16 lần mỗi bit. Với clock 27 MHz, tín hiệu enable_16x được tạo sau mỗi 176 chu kỳ clock, đáp ứng yêu cầu truyền nhận UART.

Bảng 4.1 Thông số thiết lập mô phỏng

Thông số	Giá trị sử dụng trong mô phỏng
Công cụ mô phỏng	Xilinx ISE/ISim
Clock hệ thống	27MHz
Chu kỳ clock	37.037ns
Chuẩn truyền thông	UART
Baudrate UART	9600 baud
Hệ số lấy mẫu UART	16x
Số tap FIR	200 tap
Độ rộng hệ số FIR	16 bit
Độ rộng mẫu vào x(n)	16 bit
Độ rộng kết quả FIR y(n)	41 bit
Số mẫu đầu vào trong ROM	10 mẫu
Số byte truyền cho mỗi kết quả FIR	6 byte

Trong mô phỏng, ROM lưu 10 mẫu dữ liệu gồm cả giá trị dương và âm để kiểm tra khả năng xử lý dữ liệu có dấu của hệ thống:

x(n) = {1231, -2345, -3457, -4569, 5671, -6783, 7995, -8907, 9019, -10131}

Bộ lọc FIR sử dụng 200 hệ số, trong đó:

h(n) = {1, 0, 0, ..., 0}

với h[0] = 1 và h[1]–h[199] = 0.

Khi đó phương trình FIR $y(n)=\Sigma h(k)x(n-k)$ được rút gọn thành: y(n)=x(n)

Do đó, kết quả đầu ra kỳ vọng của bộ lọc chính là dữ liệu đầu vào:

y(n) = {1231, -2345, 3457, -4569, 5671, -6783, 7895, -8907, 9019, -10131}

Bộ dữ liệu kiểm thử được lựa chọn nhằm kiểm tra đầy đủ quá trình nhận dữ liệu UART, nạp hệ số, xử lý FIR và truyền kết quả, đồng thời giúp việc đối chiếu giữa đầu vào và đầu ra trở nên trực quan.

4.1.2 Kịch bản kiểm thử và luồng dữ liệu trong testbench

Testbench được xây dựng theo hai giai đoạn.

Giai đoạn đầu kiểm tra độc lập các module gồm Baudrate Generator, Synchronizer, UART Receiver, Byte Packer, Coeff Loader, FIR Filter, FIFO Buffer, Unbyte Packer và UART Transmitter.

Sau khi từng module hoạt động đúng, testbench thực hiện kiểm thử tích hợp toàn hệ thống theo luồng:

rx_pin → Synchronizer → UART Receiver → Byte Packer → Coeff Loader → FIR Filter → FIFO Buffer → Unbyte Packer → UART Transmitter → tx_pin

Quá trình nạp hệ số được thực hiện tuần tự từ địa chỉ 0 đến 199. Mỗi hệ số 16 bit được truyền bằng hai byte UART, tương ứng tổng cộng 400 byte. Sau khi nạp đủ hệ số, tín hiệu coeffs_loaded được kích hoạt để ROM bắt đầu cấp dữ liệu cho FIR Filter.

Bộ lọc FIR xử lý 10 mẫu dữ liệu trong ROM và tạo ra 10 kết quả đầu ra 41 bit. Mỗi kết quả được ghi vào FIFO, mở rộng lên 48 bit và tách thành 6 byte trước khi truyền qua UART TX. Tổng số byte truyền về máy tính là 60 byte.

Trong testbench, các giá trị đầu ra kỳ vọng được khai báo trước trong mảng expected_y để so sánh với dữ liệu thu được từ UART TX.

Bảng 4.2 Tổng hợp các case kiểm thử trong testbench

STT	Nội dung kiểm thử	Khối kiểm thử	Tín hiệu quan sát	Điều kiện thực hiện	Kết quả mong muốn
TC01	Kiểm tra tạo xung lấy mẫu UA	Baudrate Generator	enable_16x	Sau khi reset được nhả	enable_16x tạo xung tuần hoàn, độ rộng 1 chu kỳ clock, chu kỳ phù hợp UART 9600 baud (16× Oversampling)
TC02	Kiểm tra đồng bộ tín hiệu RX	Synchronizer	rx_pin_sys, rx_sync	Tín hiệu UART RX thay đổi mức logic	rx_sync bám theo rx_pin_sys sau độ trễ đồng bộ, không xuất hiện chuyển mức bất thường

STT	Nội dung kiểm thử	Khối kiểm thử	Tín hiệu quan sát	Điều kiện thực hiện	Kết quả mong muốn
TC03	Kiểm tra nhận một byte UART	UART Receiver	rx_sync, rx_data, rx_valid, sample_cnt, bit_cnt	Gửi một khung UART hợp lệ	UART Receiver nhận đủ 8 bit dữ liệu, rx_valid được kích hoạt sau khi hoàn thành khung truyền
TC04	Kiểm tra ghép hai byte thành hệ số FIR	Byte Packer	rx_data, pack_data, pack_valid	Nhận hai byte liên tiếp từ UART RX	Hai byte được ghép đúng thành một hệ số 16 bit, pack_valid chỉ bật sau khi nhận đủ hai byte
TC05	Kiểm tra ghi hệ số đầu tiên	Coeff Loader	coeff_addr, coeff_data, coeff_we	Nhận hệ số đầu tiên từ Byte Packer	Hệ số đầu tiên được ghi vào địa chỉ 0 của bộ nhớ hệ số
TC06	Kiểm tra hoàn tất nạp hệ số	Coeff Loader	coeff_addr, coeff_count, coeffs_loaded	Ghi đủ toàn bộ hệ số FIR	coeff_count đạt số lượng hệ số thiết kế và coeffs_loaded được kích hoạt
TC07	Kiểm tra hoàn tất nạp hệ số	ROM Controller	rom_addr, rom_data, fir_x_valid	coeffs_loaded = 1	ROM bắt đầu cấp dữ liệu tuần tự cho FIR và fir_x_valid được kích hoạt đúng thời điểm
TC08	Kiểm tra xử lý FIR	FIR Filter	fir_busy, fir_y_valid, fir_y_data	FIR nhận mẫu đầu vào hợp lệ	FIR hoàn thành phép lọc và xuất kết quả đúng theo bộ hệ số đã nạp
TC09	Kiểm tra lưu và đọc FIFO	FIFO Buffer	fifo_empty, fifo_full, fifo_rd, fifo_dout	FIR xuất dữ liệu hợp lệ	FIFO lưu và đọc đúng dữ liệu, không mất mẫu và không tràn bộ đệm
TC10	Kiểm tra tách dữ liệu 41 bit	Unbyte Packer	tx_data, tx_valid, byte_cnt, flash_o	FIFO có dữ liệu	Mỗi kết quả FIR được tách thành 6 byte theo đúng thứ tự truyền
TC11	Kiểm tra truyền UART	UART Transmitter	tx_pin_sys, flash_o, state	Có byte cần truyền	UART TX truyền đúng khung UART gồm Start bit, 8 Data bit và Stop bit

STT	Nội dung kiểm thử	Khối kiểm thử	Tín hiệu quan sát	Điều kiện thực hiện	Kết quả mong muốn
TC12	Kiểm tra toàn hệ thống	Top Module	coeff_count, fir_count, tx_count, flash_count, system_error_coun	Hoàn thành toàn bộ quá trình mô phỏng	Toàn bộ luồng dữ liệu từ UART RX đến UART TX hoàn thành, các bộ đếm đạt giá trị kỳ vọng và không phát hiện lỗi

Các trường hợp kiểm thử được chia thành hai nhóm:

- **TC01–TC11:** kiểm tra chức năng của từng module.
- **TC12:** kiểm tra toàn bộ hệ thống từ UART RX đến UART TX.

Trong quá trình kiểm thử tích hợp, các bộ đếm coeff_count, fir_count, tx_count và flash_count được sử dụng để theo dõi số lượng dữ liệu đi qua từng giai đoạn, trong khi system_error_count dùng để ghi nhận các sai lệch nếu có.

Bảng 4.3 Tiêu chí đánh giá trong pha kiểm thử tích hợp

Chỉ tiêu đánh giá	Ý nghĩa	Giá trị mong muốn
coeff_count	Số hệ số FIR đã được nạp vào hệ thống	200
fir_count	Số kết quả FIR được tạo ra	10
tx_count	Số byte được Unbyte Packer đưa sang UART TX	60
flash_count	Số byte đã được UART TX truyền xong	60
system_error_count	Số lỗi trong quá trình kiểm thử tích hợp	0
Timeout	Kiểm tra hệ thống có bị treo khi chờ kết quả hay không	Không xảy ra

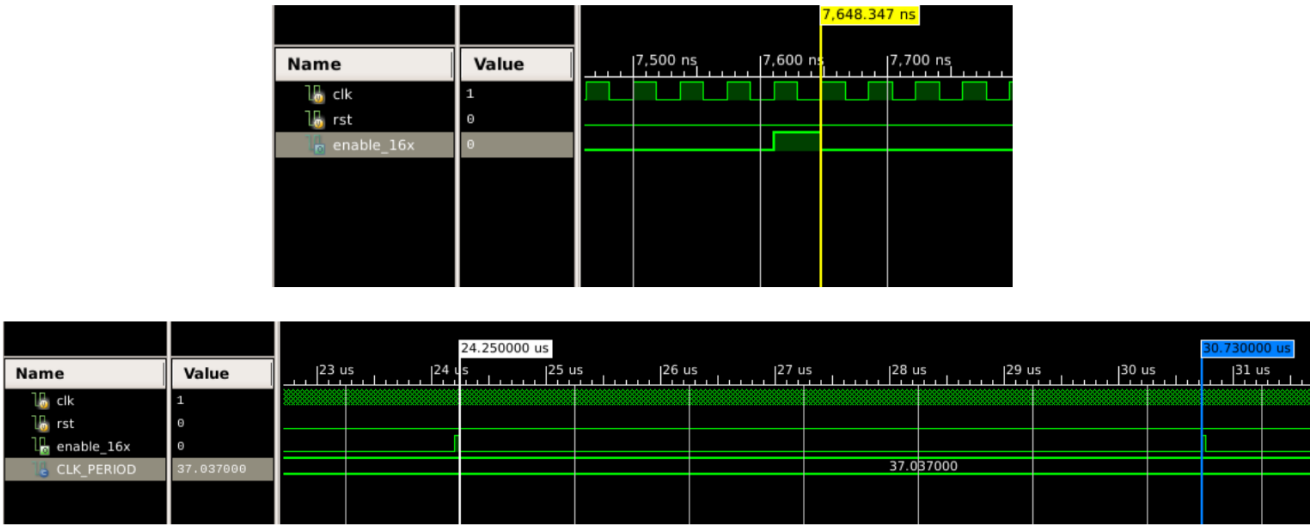
Kịch bản kiểm thử được xây dựng theo hướng từ kiểm thử đơn vị đến kiểm thử tích hợp, giúp đánh giá đầy đủ chức năng của từng module cũng như sự phối hợp giữa các khối trong toàn hệ thống.

4.1.3 Kết quả và nhận xét

Sau khi hoàn thành các kịch bản kiểm thử, hệ thống được mô phỏng trên Xilinx ISE/ISim để đánh giá hoạt động của từng module và kiểm chứng luồng dữ liệu của toàn hệ thống. Kết quả được trình bày theo đúng trình tự xử lý của testbench, từ kiểm thử các module độc lập đến quá trình tích hợp gồm: tạo xung lấy mẫu UART, đồng bộ tín hiệu RX, nhận dữ liệu UART, ghép byte thành hệ số FIR, nạp hệ số, xử lý FIR, lưu kết quả vào FIFO, tách byte và truyền dữ liệu qua UART TX.

A.1 Kết quả tạo xung lấy mẫu UART

Khối Baudrate Generator có nhiệm vụ tạo tín hiệu enable_16x phục vụ cho quá trình truyền nhận UART. Tín hiệu này được sử dụng làm mốc lấy mẫu cho UART Receiver và UART Transmitter, tương ứng với cơ chế lấy mẫu 16 lần trên mỗi bit dữ liệu. Với clock hệ thống 27 MHz và tốc độ truyền UART 9600 baud, xung enable_16x được kỳ vọng lặp lại sau khoảng 176 chu kỳ clock.



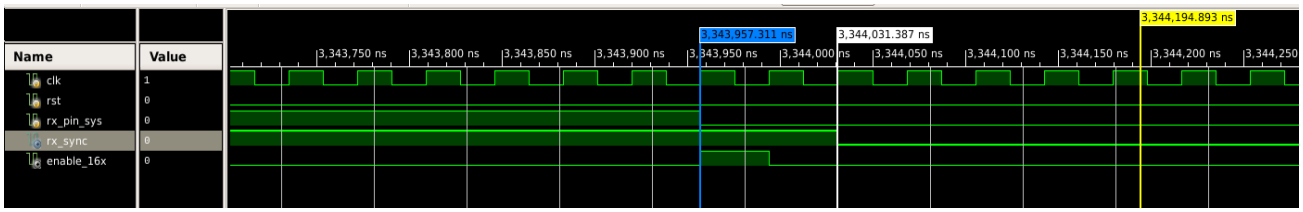
Hình 4.1 Dạng sóng tạo xung enable_16x cho UART

Dạng sóng ở Hình 4.1 cho thấy tín hiệu enable_16x chỉ được kích hoạt trong một chu kỳ clock và sau đó trở về mức 0. Hai xung enable_16x liên tiếp cách nhau xấp xỉ 6,518 µs, tương ứng khoảng 176 chu kỳ clock của hệ thống 27 MHz. Kết quả này phù hợp với thiết kế bộ tạo baudrate cho UART 9600 baud với hệ số lấy mẫu 16x. Việc enable_16x chỉ kéo dài trong một chu kỳ clock cũng đảm bảo các bộ đếm trong UART RX và UART TX chỉ cập nhật tại đúng thời điểm lấy mẫu, tránh hiện tượng đếm sai hoặc lấy mẫu lệch nhịp. Ngoài ra, dạng sóng cũng cho thấy sau khi tín hiệu reset được nhả, enable_16x bắt đầu hoạt động tuần tự và ổn định. Điều này chứng minh khối Baudrate Generator hoạt động đúng chức năng, tạo được xung điều khiển thời gian cho các khối UART phía sau.

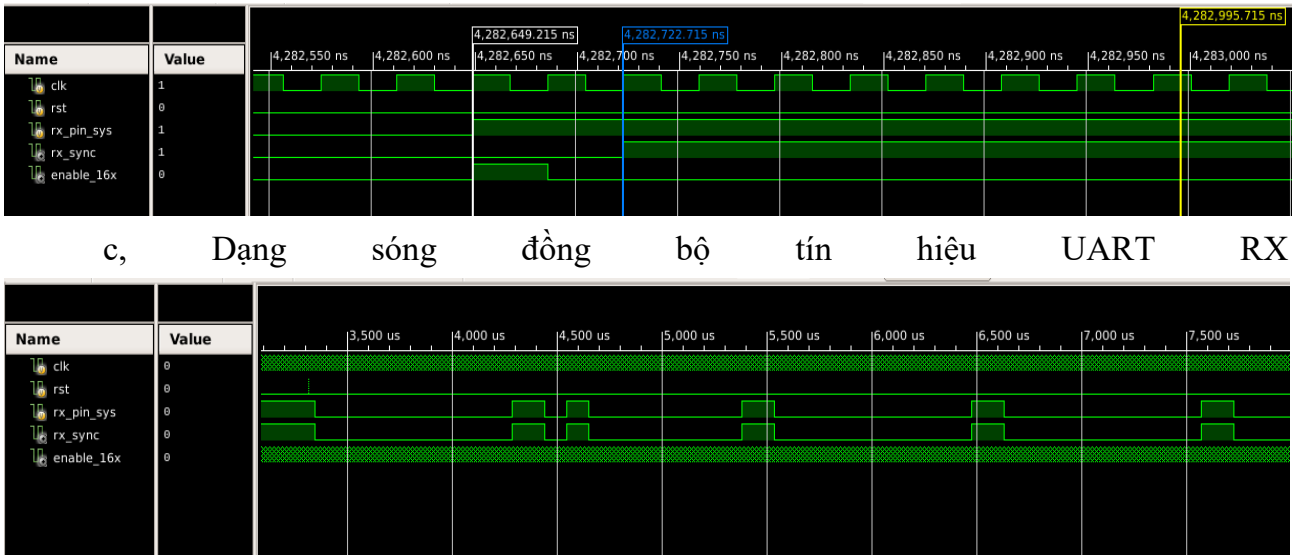
A.2 Kết quả đồng bộ tín hiệu UART RX

Tín hiệu UART RX từ testbench được đưa vào hệ thống qua rx_pin_sys. Đây là tín hiệu có nguồn gốc từ bên ngoài hệ thống nên không đồng bộ trực tiếp với clock FPGA. Vì vậy, trước khi đưa vào UART Receiver, tín hiệu này được đi qua khối Synchronizer để tạo ra tín hiệu rx_sync. Việc đồng bộ tín hiệu RX giúp giảm rủi ro sai lệch khi lấy mẫu dữ liệu nối tiếp.

a, Độ trễ cạnh xuống



b, Độ trễ cạnh lên



Hình 4.2 Dạng sóng đồng bộ tín hiệu UART RX

Dạng sóng ở Hình 4.2 trình bày quá trình đồng bộ tín hiệu UART RX khi tín hiệu đầu vào rx_pin_sys thay đổi mức logic. Hình gồm ba phần: độ trễ tại cạnh xuống, độ trễ tại cạnh lên và dạng sóng tổng quát sau khi đồng bộ. Trong đó, rx_pin_sys là tín hiệu UART từ testbench, còn rx_sync là tín hiệu sau khi đi qua khối Synchronizer trước khi đưa vào UART Receiver.

Ở Hình 4.2(a), khi rx_pin_sys chuyển từ mức logic 1 xuống 0, rx_sync chỉ chuyển mức sau một khoảng trễ nhỏ do tín hiệu phải đi qua hai tầng Flip-Flop đồng bộ. Khoảng trễ này là đặc tính bình thường của mạch đồng bộ, giúp đưa tín hiệu bất đồng bộ về cùng miền clock với hệ thống trước khi xử lý.

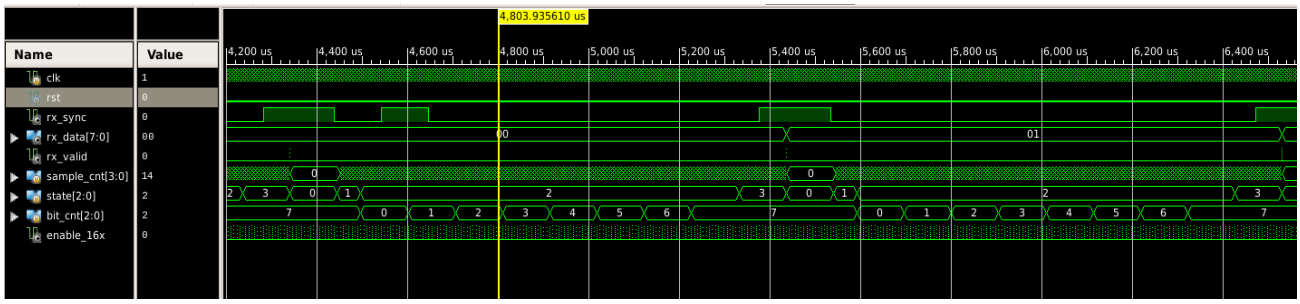
Ở Hình 4.2(b), khi rx_pin_sys chuyển từ mức 0 lên mức 1, rx_sync cũng thay đổi sau khoảng trễ tương tự. Kết quả này cho thấy khối Synchronizer hoạt động đúng ở cả cạnh lên và cạnh xuống, đồng thời bảo đảm tín hiệu đầu ra chỉ thay đổi tại cạnh clock của hệ thống.

Ở Hình 4.2(c), dạng sóng tổng quát cho thấy rx_sync giữ nguyên mức logic của rx_pin_sys, chỉ bị dịch thời gian một khoảng nhỏ. Với baudrate 9600, độ trễ này nhỏ hơn nhiều so với thời gian của một bit UART nên không ảnh hưởng đến quá trình phát hiện bit bắt đầu và lấy mẫu dữ liệu của UART Receiver.

Từ kết quả mô phỏng có thể kết luận khối Synchronizer đã thực hiện đúng chức năng đồng bộ tín hiệu RX trước khi đưa vào UART Receiver, góp phần nâng cao độ tin cậy của quá trình nhận dữ liệu trên FPGA.

A.3 Kết quả UART Receiver nhận một byte dữ liệu

Sau khi tín hiệu RX được đồng bộ, UART Receiver bắt đầu xử lý khung truyền UART. Mỗi khung truyền gồm một bit bắt đầu, 8 bit dữ liệu và một bit dừng. Khi nhận đủ một khung truyền hợp lệ, UART Receiver xuất byte nhận được tại rx_data và kích hoạt rx_valid để báo dữ liệu hợp lệ.



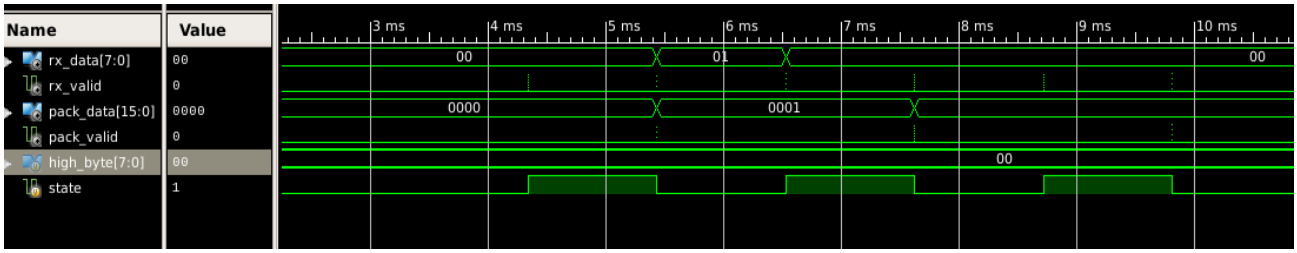
Hình 4.3 Dạng sóng UART Receiver nhận một byte dữ liệu

Quan sát waveform cho thấy khi `rx_sync` xuất hiện bit bắt đầu, FSM chuyển sang trạng thái nhận dữ liệu. Bộ đếm `sample_cnt` tạo thời điểm lấy mẫu, trong khi `bit_cnt` tăng từ 0 đến 7 để nhận đủ 8 bit dữ liệu. Sau khi kết thúc bit dừng, tín hiệu `rx_valid` được kích hoạt trong một chu kỳ clock và giá trị nhận được tại `rx_data` là `0x01`, đúng với byte thấp của hệ số đầu tiên `16'h0001`.

Kết quả mô phỏng cho thấy UART Receiver phát hiện đúng khung truyền, chuyển đổi chính xác dữ liệu nối tiếp thành dữ liệu song song 8 bit và tạo tín hiệu báo dữ liệu hợp lệ để chuyển sang khối Byte Packer.

A.4 Kết quả Byte Packer ghép hai byte thành hệ số 16 bit

Do UART truyền dữ liệu theo từng byte 8 bit, trong khi mỗi hệ số FIR có độ rộng 16 bit, hệ thống cần ghép hai byte liên tiếp thành một hệ số hoàn chỉnh. Trong kịch bản mô phỏng, mỗi hệ số 16 bit được truyền theo thứ tự byte cao trước, byte thấp sau. Vì vậy, với hệ số đầu tiên h[0] = 16'h0001, UART sẽ gửi byte 00 trước và byte 01 sau.



Hình 4.4 Dạng sóng Byte Packer ghép hai byte thành hệ số 16 bit

Dạng sóng ở Hình 4.4 cho thấy Byte Packer nhận dữ liệu từ UART Receiver thông qua rx_data và rx_valid. Ở xung rx_valid đầu tiên, rx_data có giá trị 00. Đây là byte cao của hệ số h[0], nên Byte Packer lưu giá trị này vào thanh ghi high_byte và chuyển trạng thái sang chờ byte thứ hai. Tại thời điểm này, pack_valid chưa được kích hoạt vì khối mới nhận được một byte, chưa đủ dữ liệu để tạo thành một hệ số 16 bit.

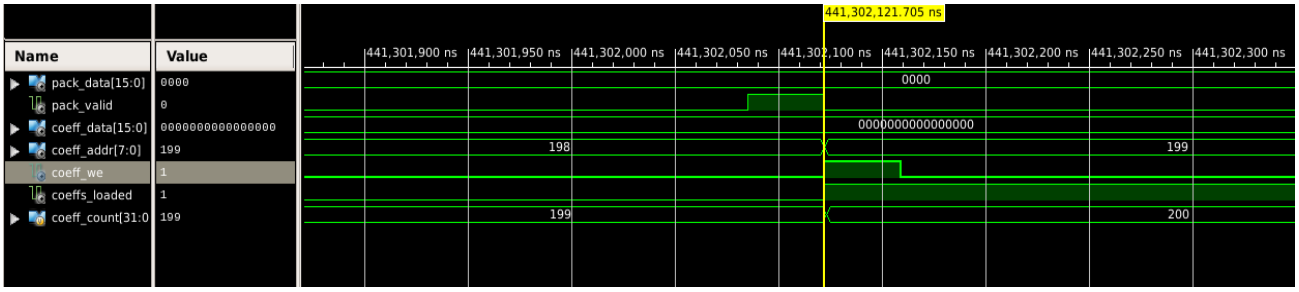
Ở xung rx_valid tiếp theo, rx_data có giá trị 01. Đây là byte thấp của hệ số h[0]. Khi byte thứ hai được nhận, Byte Packer ghép high_byte với rx_data theo dạng {high_byte, rx_data}, tạo ra pack_data = 0001. Đồng thời, pack_valid được kích hoạt để báo rằng một hệ số 16 bit đã sẵn sàng cho khối Coeff Loader.

Sau khi pack_data = 0001 được tạo ra, rx_data tiếp tục chuyển sang 00. Đây không phải là dữ liệu bị sai thứ tự mà là byte cao của hệ số kế tiếp h[1] = 16'h0000. Như vậy, dạng

sóng chứng minh Byte Packer đã ghép đúng byte cao và byte thấp thành hệ số 16 bit, đồng thời chỉ kích hoạt pack_valid sau khi nhận đủ hai byte.

A.5 Kết quả nạp hệ số đầu tiên h[0]

Sau khi Byte Packer tạo ra hệ số 16 bit, Coeff Loader ghi hệ số này vào bộ nhớ hệ số của FIR. Quá trình nạp hệ số được thực hiện tuần tự theo địa chỉ từ 0 đến 199. Hình 4.6 trình bày thời điểm hệ số đầu tiên h[0] được ghi vào bộ nhớ hệ số.



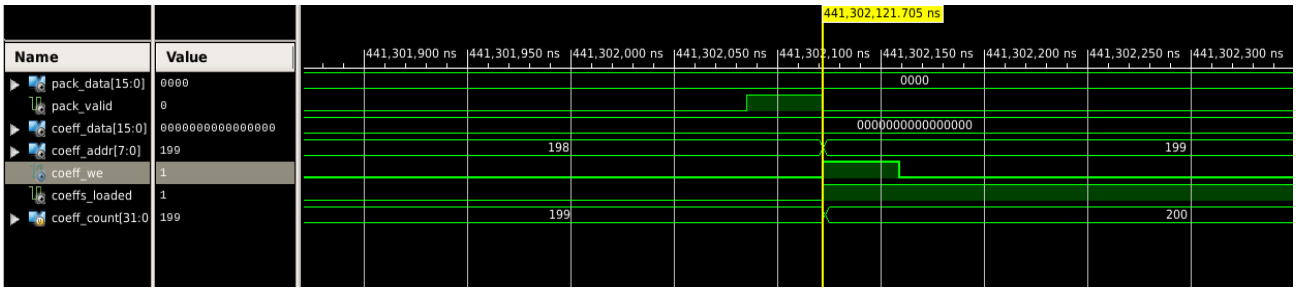
Hình 4.5 Dạng sóng nạp hệ số đầu tiên h[0]

Dạng sóng ở Hình 4.5 cho thấy pack_data có giá trị 0001 và pack_valid được kích hoạt, báo rằng hệ số 16 bit đầu tiên đã được ghép xong từ hai byte UART. Coeff Loader nhận dữ liệu này và đưa coeff_addr về địa chỉ 0. Tại cùng thời điểm, coeff_data có giá trị 1 và coeff_we được kích hoạt để ghi hệ số vào bộ nhớ hệ số FIR.

Tín hiệu coeffs_loaded vẫn ở mức 0 vì tại thời điểm này hệ thống mới chỉ ghi hệ số đầu tiên, chưa nạp đủ 200 hệ số. Bộ đếm coeff_count cũng bắt đầu tăng để ghi nhận số lượng hệ số đã được nạp. Kết quả này đúng với kịch bản kiểm thử, trong đó hệ số đầu tiên được đặt là h[0] = 1. Việc ghi đúng h[0] rất quan trọng vì trong kịch bản kiểm thử chức năng, h[0] = 1 và các hệ số còn lại bằng 0 sẽ làm cho FIR hoạt động như đường truyền thẳng, tức y(n) = x(n).

A.6 Kết quả nạp hệ số cuối h[199] và kích hoạt coeffs_loaded

Sau khi nạp hệ số đầu tiên, hệ thống tiếp tục nhận và ghi các hệ số còn lại. Trong kịch bản mô phỏng, các hệ số từ h[1] đến h[199] đều bằng 0. Hình 4.6 trình bày đoạn cuối của quá trình nạp hệ số, tại thời điểm hệ số h[199] được ghi và tín hiệu coeffs_loaded được kích hoạt.



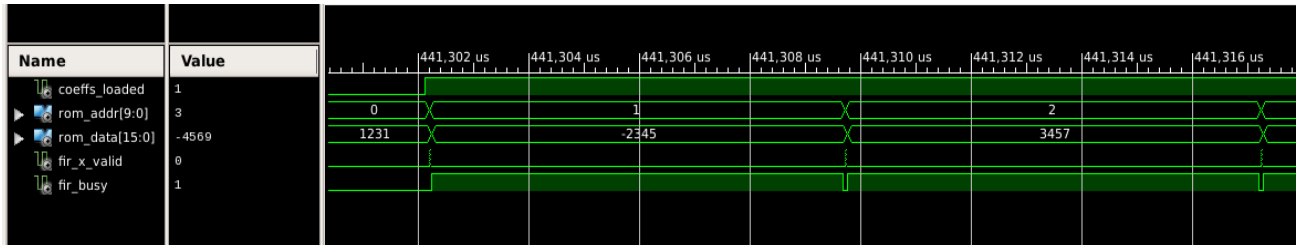
Hình 4.6 Dạng sóng nạp hệ số cuối h[199] và kích hoạt coeffs_loaded

Dạng sóng ở Hình 4.7 cho thấy pack_data có giá trị 0000 và pack_valid được kích hoạt, tương ứng với hệ số cuối cùng được Byte Packer tạo ra. Coeff Loader đưa coeff_addr đến địa chỉ 199 và kích hoạt coeff_we để ghi hệ số này vào bộ nhớ hệ số FIR. Giá trị coeff_data bằng 0, đúng với kịch bản h[1..199] = 0.

Sau khi hệ số ở địa chỉ 199 được ghi, coeffs_loaded được đưa lên mức 1. Đồng thời, bộ đếm coeff_count đạt giá trị 200, chứng minh hệ thống đã nạp đủ 200 hệ số FIR. Đây là tín hiệu quan trọng vì nó cho phép các khối phía sau, đặc biệt là ROM Controller, bắt đầu quá trình cấp dữ liệu đầu vào cho FIR Filter. Kết quả này xác nhận quá trình nạp hệ số qua UART, ghép byte và ghi vào bộ nhớ hệ số đã hoàn tất đúng theo thiết kế.

A.7 Kết quả ROM Controller cấp mẫu đầu vào cho FIR

Sau khi coeffs_loaded được kích hoạt, hệ thống chuyển từ giai đoạn nạp hệ số sang giai đoạn xử lý dữ liệu. ROM Controller bắt đầu cấp địa chỉ đọc mẫu đầu vào từ ROM và tạo tín hiệu fir_x_valid để báo cho FIR Filter nhận mẫu mới.



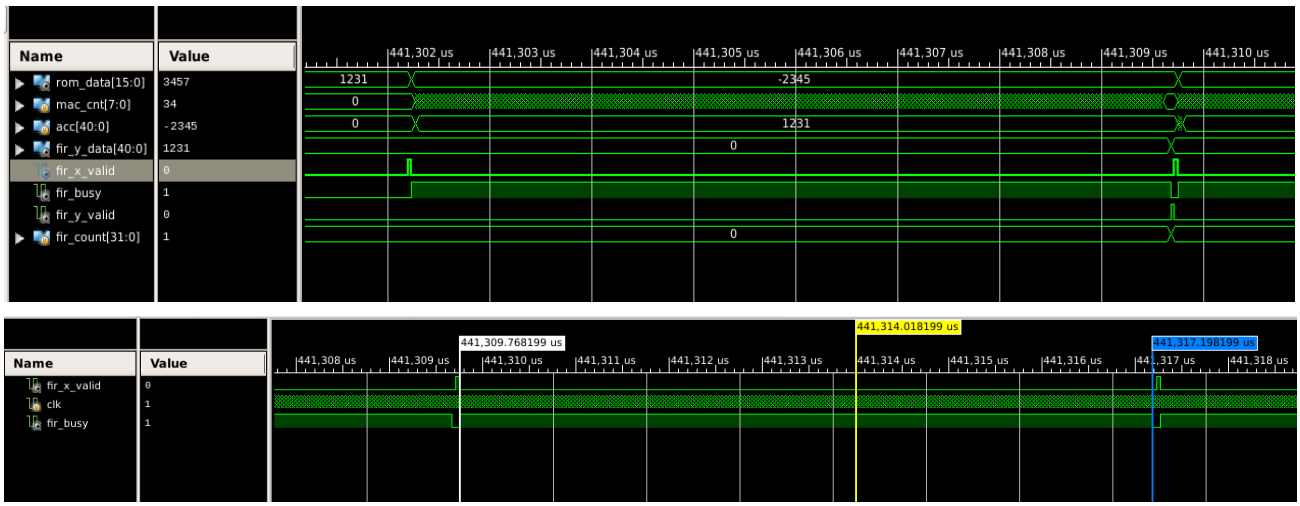
Hình 4.7 Dạng sóng ROM Controller cấp mẫu đầu vào cho FIR

Dạng sóng ở Hình 4.7 cho thấy sau khi coeffs_loaded ở mức 1, ROM Controller bắt đầu cấp địa chỉ rom_addr cho ROM. Địa chỉ rom_addr lần lượt tăng từ 0, 1 đến 2, tương ứng với các mẫu rom_data lần lượt là 1231, -2345 và 3457. Các giá trị này trùng với dữ liệu đã khai báo trong ROM mô phỏng.

Tại mỗi thời điểm mẫu ROM hợp lệ, fir_x_valid được kích hoạt để đưa dữ liệu vào FIR Filter. Sau khi FIR nhận mẫu, fir_busy được đưa lên mức 1, cho thấy bộ lọc bắt đầu thực hiện quá trình tính toán. Kết quả này chứng minh ROM Controller đã hoạt động đúng, chỉ bắt đầu cấp dữ liệu sau khi toàn bộ hệ số FIR đã được nạp xong. Đồng thời, luồng điều khiển giữa Coeff Loader, ROM Controller và FIR Filter phối hợp đúng theo thứ tự thiết kế.

A.8 Kết quả FIR Filter xử lý mẫu và xuất kết quả y(n)

FIR Filter trong hệ thống sử dụng kiến trúc MAC tuần tự để thực hiện phép nhân cộng qua 200 tap. Trong kịch bản kiểm thử chức năng, hệ số được chọn là h[0] = 1 và h[1..199] = 0. Với cấu hình này, bộ lọc không làm thay đổi giá trị mẫu đầu vào mà hoạt động tương đương đường truyền thẳng. Do đó, kết quả kỳ vọng là y(n) = x(n).



Hình 4.8 Dạng sóng FIR Filter xử lý mẫu đầu vào và xuất kết quả y(n)

Quan sát Hình 4.8 cho thấy khi tín hiệu fir_x_valid được kích hoạt, khối FIR bắt đầu nhận mẫu đầu vào từ ROM và đưa fir_busy lên mức 1 để báo đang xử lý. Trong thời gian này, bộ đếm mac_cnt tăng tuần tự để thực hiện phép nhân – cộng tích lũy trên toàn bộ 200 tap, còn tín hiệu acc thể hiện giá trị tích lũy trong quá trình tính toán.

Sau khi hoàn thành toàn bộ chu trình MAC, tín hiệu fir_y_valid được kích hoạt và dữ liệu đầu ra xuất hiện tại fir_y_data. Kết quả đầu tiên thu được là 1231, trùng với giá trị của mẫu đầu vào đầu tiên trong ROM. Điều này phù hợp với bộ hệ số kiểm thử đã lựa chọn, trong đó chỉ có hệ số đầu tiên khác 0 nên bộ lọc không làm thay đổi giá trị tín hiệu.

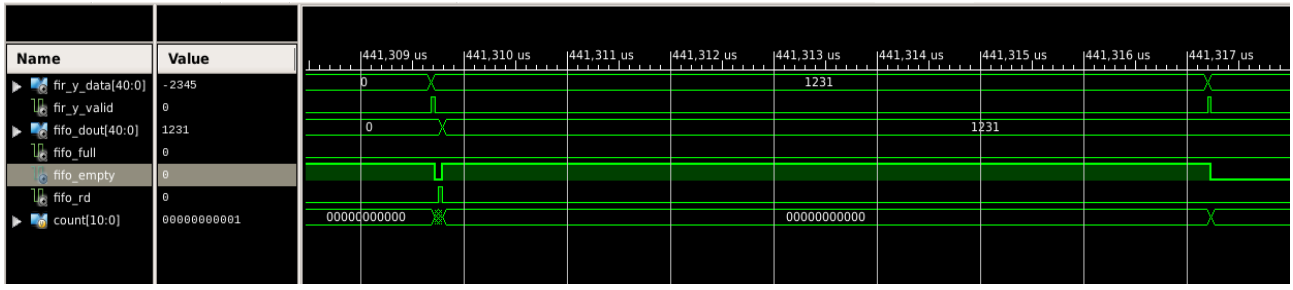
Cần lưu ý rằng tại thời điểm fir_y_data xuất hiện, rom_data có thể đã chuyển sang mẫu tiếp theo do quá trình đọc ROM và xử lý FIR diễn ra theo dạng pipeline. Vì vậy, kết quả đầu ra cần được đối chiếu với mẫu đã được FIR nhận trước đó, thay vì so sánh trực tiếp với giá trị rom_data tại cùng thời điểm quan sát waveform.

Thời gian khối FIR tính toán cho một dữ liệu đầu ra tại fir_y_data là 411.317 198199ns - 411 309.768199ns = 7430ns, xấp xỉ với 201 chù kỳ clk (1 chu kỳ là 37.037ns), với 200 chu kỳ cho việc tính toán và 1 chu kì reset bộ lọc.

Kết quả mô phỏng xác nhận khối FIR hoạt động đúng theo thiết kế, xử lý chính xác dữ liệu có dấu và tạo đầu ra phù hợp với giá trị kỳ vọng.

A.9 Kết quả FIFO Buffer lưu và đọc kết quả FIR

Kết quả FIR sau khi được tạo ra có độ rộng 41 bit và được ghi vào FIFO trước khi truyền qua UART TX. FIFO đóng vai trò đệm dữ liệu giữa khối xử lý FIR và khối truyền UART, giúp tránh mất dữ liệu khi hai khối có tốc độ hoạt động khác nhau.



Hình 4.9 Dạng sóng FIFO Buffer lưu và đọc kết quả FIR

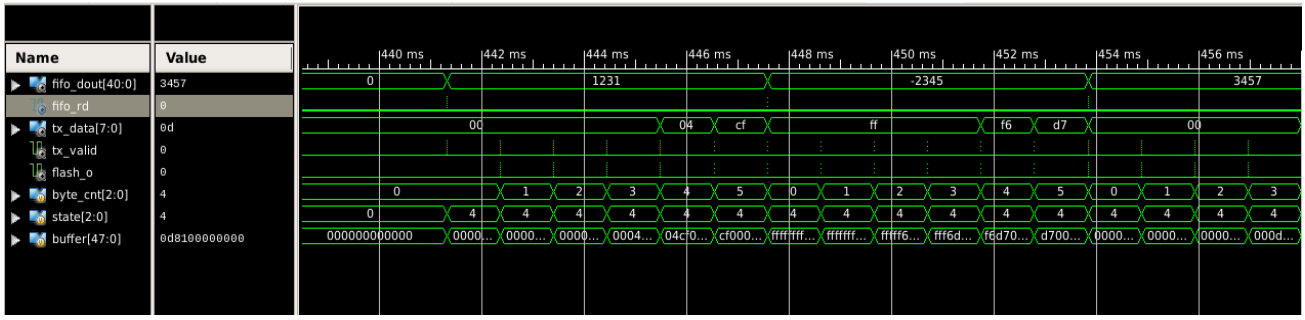
Dạng sóng ở Hình 4.9 cho thấy khi fir_y_valid được kích hoạt, kết quả FIR đầu tiên có giá trị fir_y_data = 1231 được ghi vào FIFO. Sau khi dữ liệu được ghi, fifo_empty chuyển xuống mức 0, thể hiện FIFO không còn rỗng và đã có dữ liệu sẵn sàng để đọc. Tín hiệu fifo_full vẫn ở mức 0, cho thấy FIFO chưa bị đầy trong quá trình mô phỏng.

Khi khối phía sau yêu cầu đọc dữ liệu, fifo_rd được kích hoạt. Sau đó, dữ liệu xuất hiện tại fifo_dout với giá trị 1231, trùng với kết quả FIR đã được ghi trước đó. Điều này chứng minh FIFO đã lưu và đọc dữ liệu đúng thứ tự.

Trong waveform, có thể tại một số thời điểm fir_y_data đã chuyển sang kết quả kế tiếp như -2345, trong khi fifo_dout vẫn đang xuất dữ liệu 1231. Đây là hiện tượng bình thường vì FIFO là bộ đệm, dữ liệu ra có thể trễ hơn dữ liệu vừa được FIR tạo ra. Vì vậy, việc fifo_dout = 1231 trong khi FIR đã chuẩn bị kết quả kế tiếp không phải lỗi, mà phản ánh đúng nguyên lý hoạt động của FIFO.

A.10 Kết quả Unbyte Packer tách kết quả FIR thành 6 byte

Mỗi kết quả FIR có độ rộng 41 bit, trong khi UART TX chỉ truyền dữ liệu theo từng byte 8 bit. Vì vậy, trước khi truyền qua UART, kết quả FIR cần được mở rộng lên 48 bit và tách thành 6 byte. Khối Unbyte Packer thực hiện chức năng này và sử dụng tín hiệu flash_o từ UART TX để biết khi nào byte hiện tại đã truyền xong.



Hình 4.10 Dạng sóng Unbyte Packer tách kết quả FIR 41 bit thành các byte 8 bit

Dạng sóng ở Hình 4.10 cho thấy khi FIFO có dữ liệu, Unbyte Packer kích hoạt fifo_rd để đọc một kết quả FIR từ fifo_dout. Dữ liệu đọc ra sau đó được đưa vào thanh ghi buffer 48 bit. Với dữ liệu dương, các bit mở rộng phía trên có giá trị 0; với dữ liệu âm, các bit mở rộng phía trên có giá trị 1 để giữ đúng dấu của số.

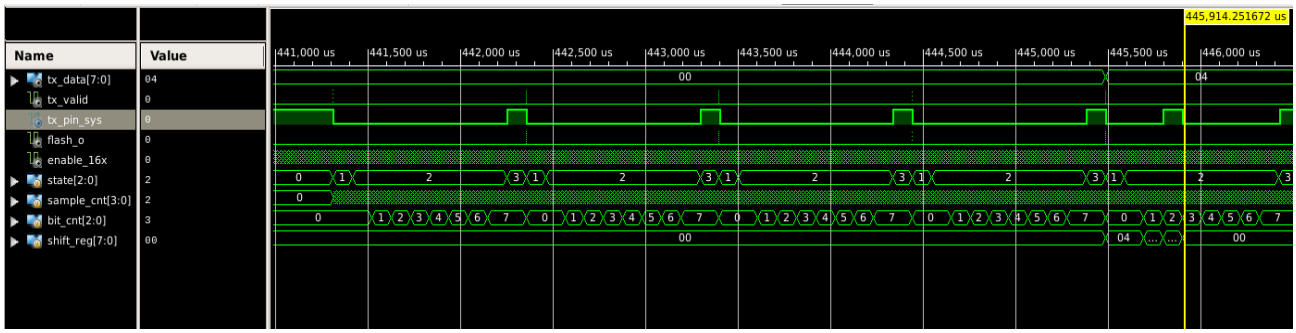
Sau khi dữ liệu được đưa vào buffer, tx_data lần lượt xuất các byte 8 bit và tx_valid được kích hoạt để gửi từng byte sang UART Transmitter. Bộ đếm byte_cnt chạy từ 0 đến 5, tương ứng với 6 byte của một kết quả FIR. Sau mỗi byte được UART TX truyền xong, flash_o được kích hoạt, cho phép Unbyte Packer chuyển sang byte tiếp theo.

Hình 4.10 cũng chứng minh khối Unbyte Packer xử lý đúng cả dữ liệu dương và dữ liệu âm. Với kết quả dương 1231, giá trị hex tương ứng là 0x04CF, khi mở rộng thành 48 bit sẽ có các byte cao là 00. Vì vậy, chuỗi byte truyền ra có dạng 00 00 00 00 04 CF. Với kết quả âm -2345, dữ liệu được mở rộng dấu nên các byte cao có giá trị FF, thể hiện đúng định dạng

số âm bù hai. Kết quả này chứng minh quá trình đọc FIFO, mở rộng dấu và tách dữ liệu 41 bit thành 6 byte đã hoạt động đúng.

A.11 Kết quả UART Transmitter truyền byte dữ liệu

Sau khi Unbyte Packer tạo từng byte dữ liệu tại tx_data và kích hoạt tx_valid, UART Transmitter nhận byte này và truyền ra ngoài qua tx_pin_sys. Một khung UART gồm bit bắt đầu, 8 bit dữ liệu và bit dừng. Dữ liệu trong mỗi byte được truyền theo thứ tự LSB trước.



Hình 4.11 Dạng sóng UART Transmitter truyền byte dữ liệu qua tx_pin

Dạng sóng ở Hình 4.1\1 cho thấy khi tx_valid được kích hoạt, UART TX nhận byte tại tx_data và bắt đầu truyền dữ liệu nối tiếp qua tx_pin_sys. Khối UART TX lần lượt chuyển qua các trạng thái truyền bit bắt đầu, truyền 8 bit dữ liệu và truyền bit dừng. Trong trạng thái truyền dữ liệu, bit_cnt tăng từ 0 đến 7, thể hiện quá trình truyền đủ 8 bit của một byte.

Trong giai đoạn truyền, tx_pin_sys xuống mức 0 ở start bit, sau đó thay đổi theo từng bit dữ liệu và trở lại mức 1 ở stop bit. Sau khi hoàn tất truyền một byte, flash_o được kích hoạt. Tín hiệu này không chỉ báo rằng UART TX đã truyền xong byte hiện tại mà còn được đưa ngược về Unbyte Packer để cho phép xuất byte kế tiếp. Do đó, dạng sóng chứng minh UART TX truyền đúng khung UART và phối hợp đúng với khối Unbyte Packer trong luồng truyền dữ liệu đầu ra.

A.12 Kết quả tổng hợp cuối mô phỏng

Sau khi hoàn thành toàn bộ quá trình nạp hệ số, xử lý FIR và truyền kết quả qua UART TX, testbench in ra phần tổng hợp kết quả. Các biến đếm được sử dụng để xác nhận số lượng dữ liệu đi qua từng giai đoạn, đồng thời kiểm tra xem có lỗi so sánh dữ liệu hay không.

```
INTEGRATION SUMMARY
coeff_count = 200, expected = 200
fir_count   = 10, expected = 10
tx_count    = 60, expected = 60
flash_count = 60, expected = 60
system_error_count = 0
========================================
========================================
TIMING RESULT
Load coefficients time       = 437.983 ms
First FIR result latency     = 7.556 us
Total time to last TX byte   = 500.569 ms
========================================
FULL SYSTEM TEST PASSED - OK
========================================
FINAL SUMMARY
unit_error_count   = 0
system_error_count = 0
total_error_count  = 0
========================================
ALL TESTS PASSED - OK
```

Hình 4.12 Kết quả tổng hợp cuối mô phỏng trên ISim

Kết quả tổng hợp cho thấy coeff_count = 200, fir_count = 10, tx_count = 60 và flash_count = 60, phù hợp với kịch bản kiểm thử gồm 200 hệ số FIR, 10 mẫu đầu vào và mỗi kết quả FIR được truyền thành 6 byte. Đồng thời, unit_error_count = 0 và system_error_count = 0, chứng tỏ không phát hiện sai lệch trong cả kiểm thử đơn vị và kiểm thử tích hợp.

Về thời gian, hệ thống mất khoảng 437,983 ms để nạp 200 hệ số qua UART, 7,556 µs từ khi nạp xong hệ số đến khi xuất kết quả FIR đầu tiên và 500,569 ms để hoàn tất toàn bộ quá trình truyền dữ liệu. Điều này cho thấy thời gian chủ yếu nằm ở quá trình truyền UART, trong khi thời gian xử lý của khối FIR rất nhỏ và không trở thành nút thắt của hệ thống.

Bảng 4.4 Tổng hợp kết quả kiểm chứng theo từng giai đoạn

Giai đoạn kiểm tra	Tín hiệu quan sát chính	Kết quả mô phỏng	Đánh giá
Unit test từng khối	unit_error_count	unit_error_count = 0	Đạt
Tạo xung UART	enable_16x	Lặp lại sau khoảng 176 clock, kéo dài 1 clock	Đạt
Đồng bộ RX	rx_pin_sys, rx_sync	rx_sync thay đổi theo rx_pin_sys sau độ trễ đồng bộ	Đạt
UART RX	rx_data, rx_valid	Nhận được byte 01, rx_valid bật sau khi hoàn tất khung UART	Đạt
Byte Packer	pack_data, pack_valid	00 và 01 được ghép thành pack_data = 0001	Đạt
Nạp hệ số đầu	coeff_addr, coeff_data, coeff_we	coeff_addr = 0, coeff_data = 1, coeff_we = 1	Đạt
Nạp hệ số cuối	coeff_addr, coeff_data, coeffs_loaded	coeff_addr = 199, coeff_data = 0, coeff_count = 200	Đạt
ROM Controller	rom_addr, rom_data, fir_x_valid	rom_addr tăng từ 0; rom_data lần lượt 1231, -2345, 3457	Đạt

Giai đoạn kiểm tra	Tín hiệu quan sát chính	Kết quả mô phỏng	Đánh giá
FIR Filter	fir_y_data, fir_y_valid	fir_y_data đầu tiên = 1231, fir_count = 10 $Timing_{1\ mẫu}$=200 chu kỳ clk	Đạt
FIFO Buffer	fifo_dout, fifo_rd, fifo_empty	fifo_dout = 1231 sau khi đọc kết quả đầu tiên	Đạt
Unbyte Packer	tx_data, tx_valid, byte_cnt	byte_cnt chạy 0 đến 5, dữ liệu dương/âm được mở rộng dấu đúng	Đạt
UART TX	tx_pin_sys, flash_o	| state đi qua start/data/stop, flash_o bật sau mỗi byte	Đạt
Toàn hệ thống	coeff_count, fir_count, tx_count, flash_count, system_error_count	00 hệ số, 10 kết quả, 60 byte, system_error_count = 0	Đạt

Bảng 4.5 Tổng hợp thông số timing trong mô phỏng

Thông số timing	Ý nghĩa	Kết quả mô phỏng
Chu kỳ enable_16x	Thời gian giữa hai xung lấy mẫu UART 16x	xấp xỉ 6518,688 ns
Load coefficients time	Thời gian nạp 200 hệ số FIR qua UART	437,983 ms
First FIR result latency	Độ trễ từ khi nạp xong hệ số đến khi có kết quả FIR đầu tiên	7,556 µs
Total time to last TX byte	Thời gian từ lúc bắt đầu nạp hệ số đến khi truyền xong byte cuối	500,569 ms

Từ các dạng sóng được quan sát theo từng giai đoạn, có thể thấy hệ thống hoạt động đúng theo kịch bản kiểm thử đã xây dựng. Dữ liệu hệ số được truyền vào qua UART, đồng bộ, nhận byte, ghép thành hệ số 16 bit và nạp vào bộ nhớ hệ số FIR. Sau khi nạp đủ 200 hệ số, ROM Controller cấp dữ liệu đầu vào cho FIR Filter. Kết quả FIR được ghi vào FIFO, tách thành các byte 8 bit và truyền ra UART TX. Việc phân tích từng nhóm tín hiệu chứng minh không chỉ kết quả cuối cùng đúng, mà luồng dữ liệu giữa các khối cũng phối hợp đúng theo thiết kế.

4.2 Kiểm chứng mức hệ thống thông qua Python/Streamlit

4.2.1 Mục đích kiểm chứng mức hệ thống

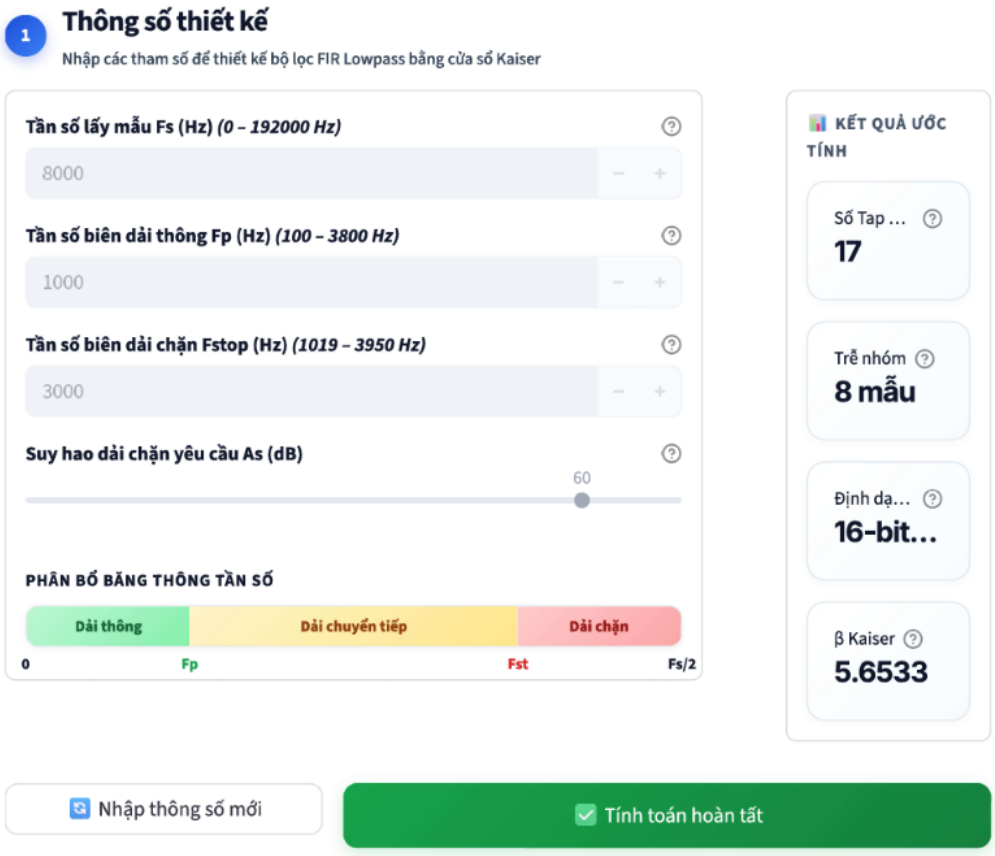
Phần kiểm chứng mức hệ thống được thực hiện nhằm đánh giá kết quả hoạt động của bộ lọc FIR sau khi triển khai trên FPGA. Khác với mô phỏng Vivado tập trung kiểm tra tín hiệu mức RTL, phần kiểm chứng trên Streamlit tập trung vào kết quả đầu ra sau lọc, khả năng so sánh với mô hình Python và đặc tính đáp ứng tần số của bộ lọc.

4.2.2 Quy trình kiểm chứng trên Streamlit

Trong quá trình kiểm chứng, hai tín hiệu thử chính được sử dụng là tín hiệu Dirac và tín hiệu Chirp. Tín hiệu Dirac dùng để quan sát đáp ứng xung và đáp ứng tần số của bộ lọc. Tín hiệu Chirp dùng để đánh giá khả năng lọc tín hiệu trên miền thời gian và kiểm tra độ trễ nhóm.

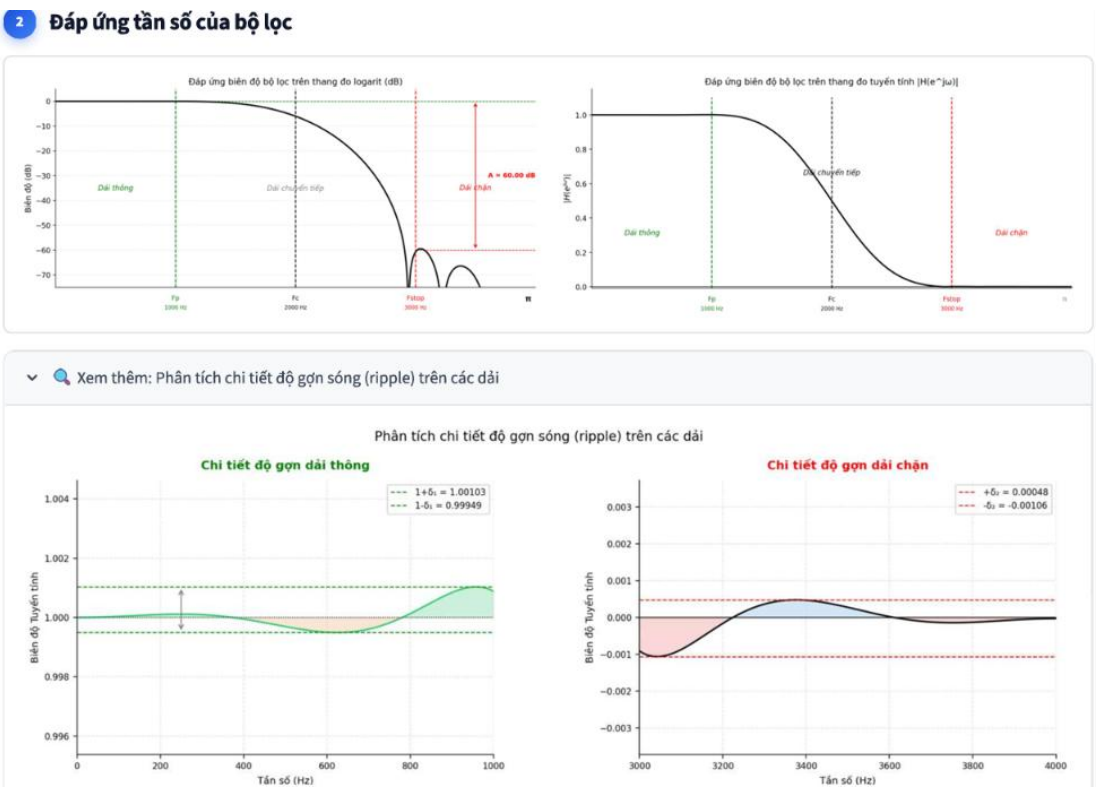
Quy trình kiểm chứng được thực hiện theo các 5 bước tuần tự như sau:

Bước 1. Nhập các thông số thiết kế bộ lọc FIR trên giao diện Streamlit như tần số lấy mẫu, tần số cắt, độ suy hao và số tap.



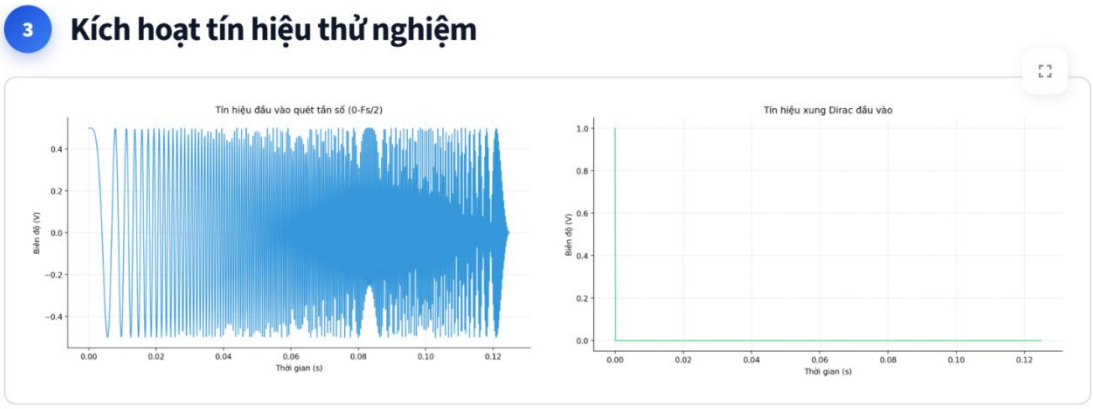
Hình 4.13 Giao diện nhập thông số thiết kế bộ lọc

Bước 2. Từ kết quả ước tính tiến hành mô phỏng bộ lọc bằng Python



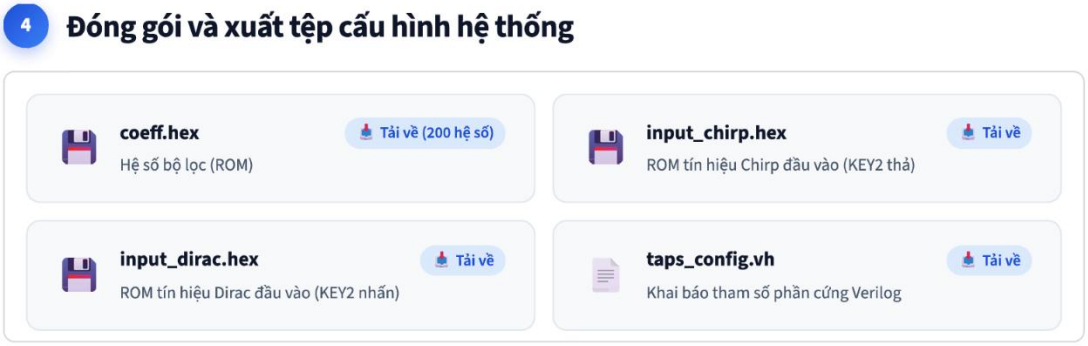
Hình 4.14 Giao diện hiển thị đáp ứng tần số mô phỏng Python

Bước 3. Tạo tín hiệu thử nghiệm



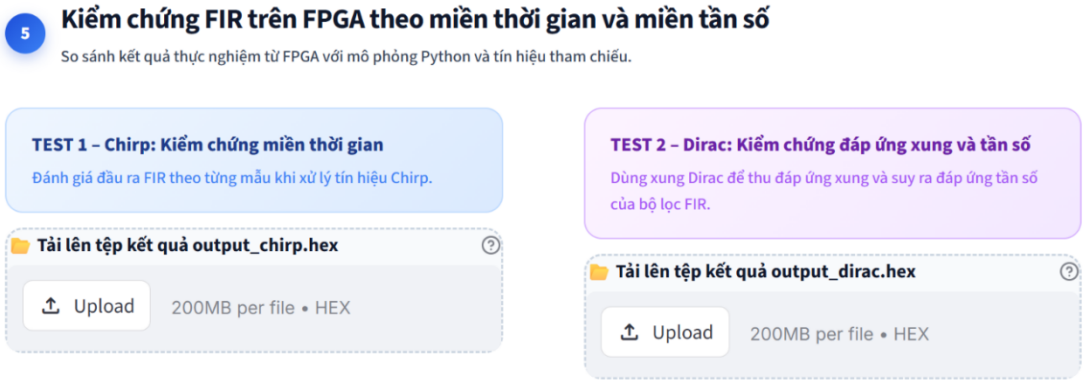
Hình 4.15 Giao diện hiển thị tín hiệu thử nghiệm

Bước 4. đóng gói các tệp để nạp vào kit



Hình 4.16 Giao diện đóng gói các tệp để nạp vào kit

Bước 5. Kiểm chứng kết quả



Hình 4.17 Giao diện kiểm chứng kết quả

Trong quá trình kiểm chứng, ba bộ thông số được lựa chọn để đánh giá hệ thống ở các mức độ khác nhau.

Bảng 4.6 Các trường hợp kiểm chứng bộ lọc FIR trên Streamlit

Case	***Mục tiêu kiểm thử***	***Fs***	***Fpass***	***Fstop***	***As***	***Số tap***	***Trễ nhóm***	***β Kaiser***
1	*Kiểm tra hệ thống với bộ lọc đơn giản, dải chuyển rộng, số tap nhỏ*	*8000 Hz*	*800 Hz*	*1800 Hz*	*40 dB*	*19*	*9 mẫu*	*3.3953*
2	*Kiểm tra cấu hình cân bằng giữa chất lượng lọc, số tap và tài nguyên phần cứng*	*8000 Hz*	*1000 Hz*	*1400 Hz*	*60 dB*	*75*	*37 mẫu*	*5.6533*
3	*Kiểm tra hệ thống với bộ lọc yêu cầu cao, dải chuyển hẹp, suy hao lớn và số tap gần giới hạn 200 tap*	*8000 Hz*	*1000 Hz*	*1200 Hz*	*70 dB*	*175*	*87 mẫu*	*6.7553*

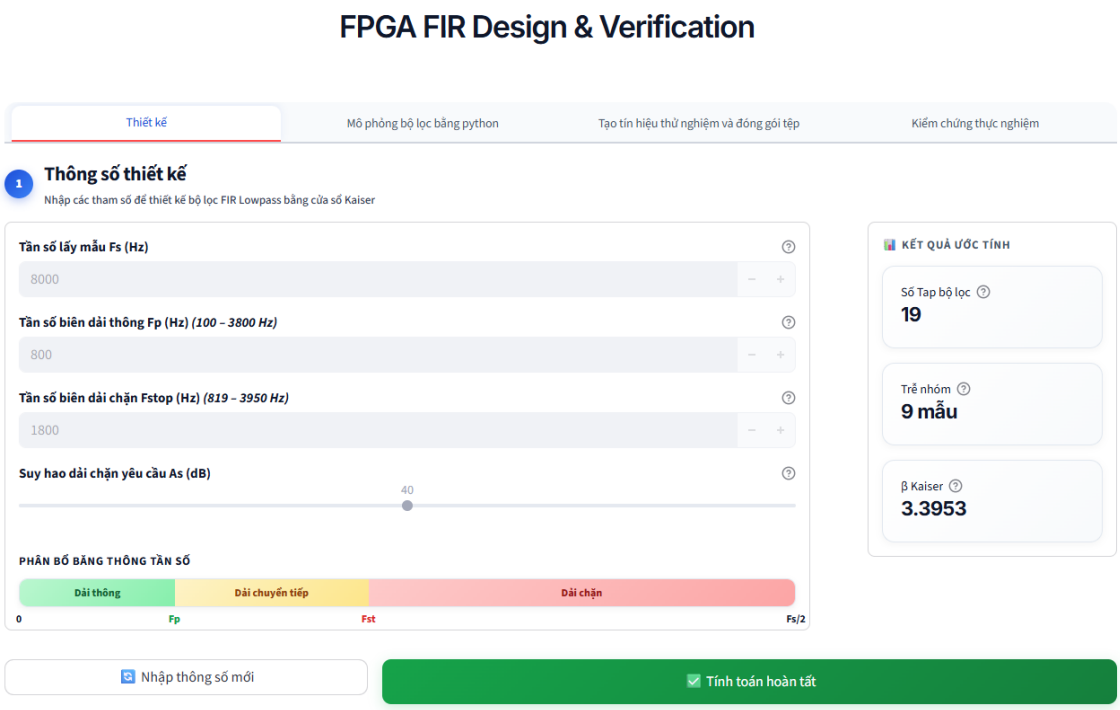
Đối với mỗi case, hệ thống thực hiện hai hướng kiểm chứng chính.

Thứ nhất, tín hiệu đầu ra từ FPGA được so sánh với tín hiệu đầu ra tham chiếu tính bằng Python. Hai tín hiệu được hiển thị trên cùng một đồ thị để quan sát mức độ tương đồng. Các chỉ số MSE và Max Error được sử dụng để đánh giá sai số giữa hai tín hiệu.

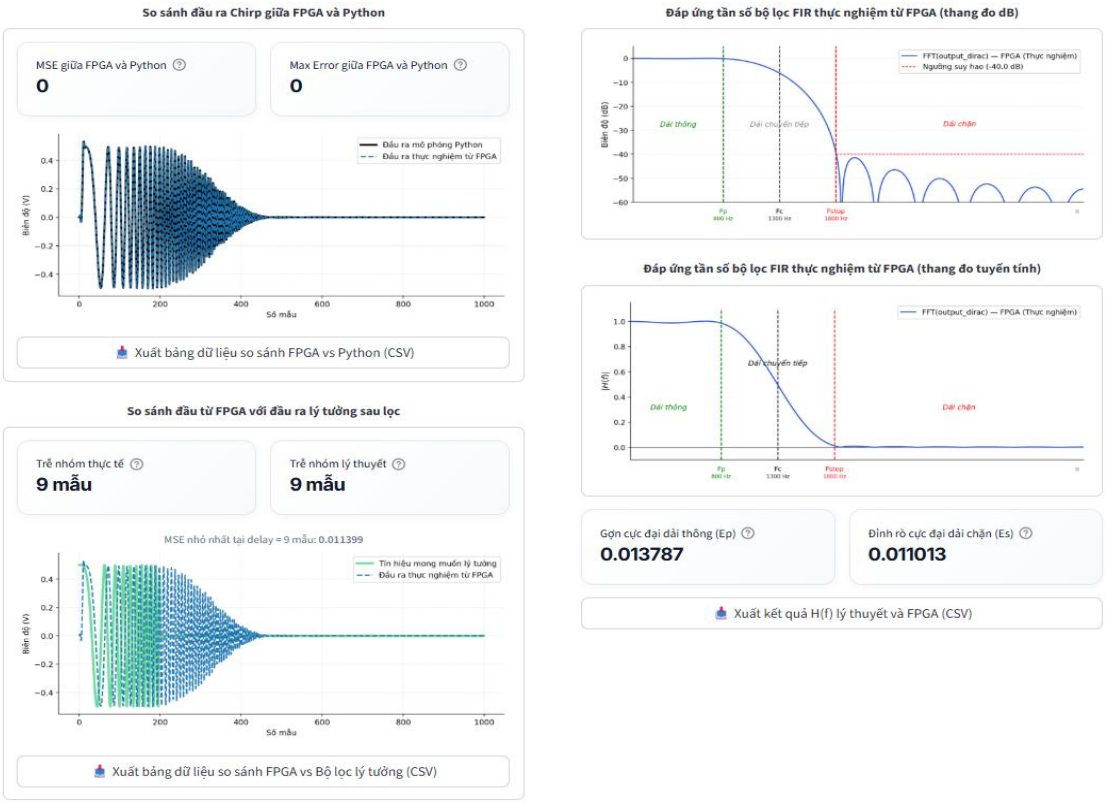
Thứ hai, tín hiệu Dirac được sử dụng để xác định đáp ứng xung của bộ lọc. Khi đầu vào là tín hiệu Dirac, đầu ra của bộ lọc chính là đáp ứng xung. Từ đáp ứng xung này, Streamlit thực hiện FFT để biểu diễn đáp ứng tần số. Kết quả đáp ứng tần số được dùng để đánh giá đặc tính thông thấp, gợn dải thông và mức rò dải chặn của bộ lọc

4.2.3 Kết quả và nhận xét

A.1 Kết quả case 1



Hình 4.18 Thông số thiết kế FIR trong Case 1



Hình 4.19 Kết quả case 1

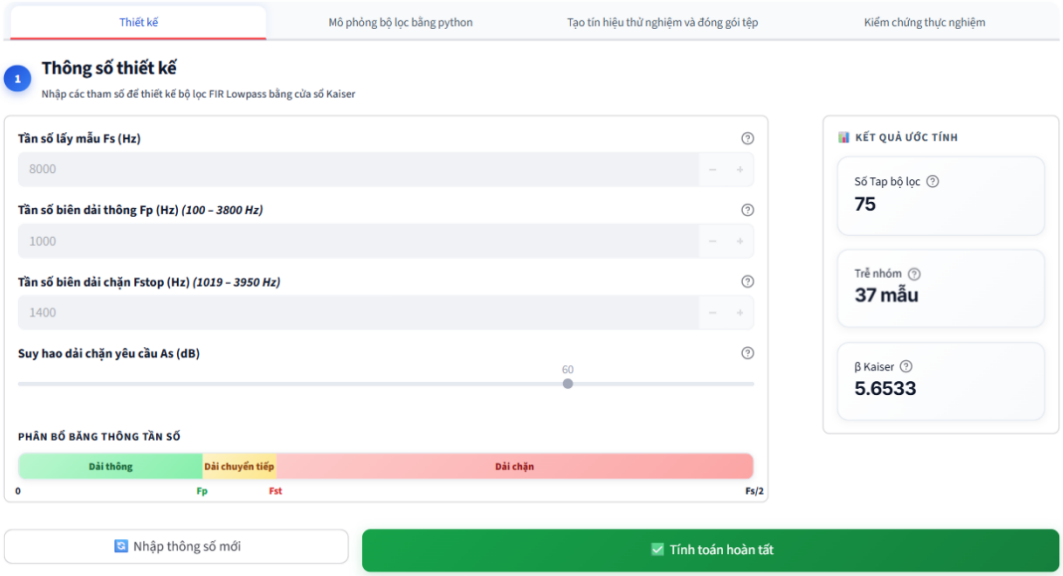
Bảng 4.7 Kết quả case 1

Thông số	Giá trị

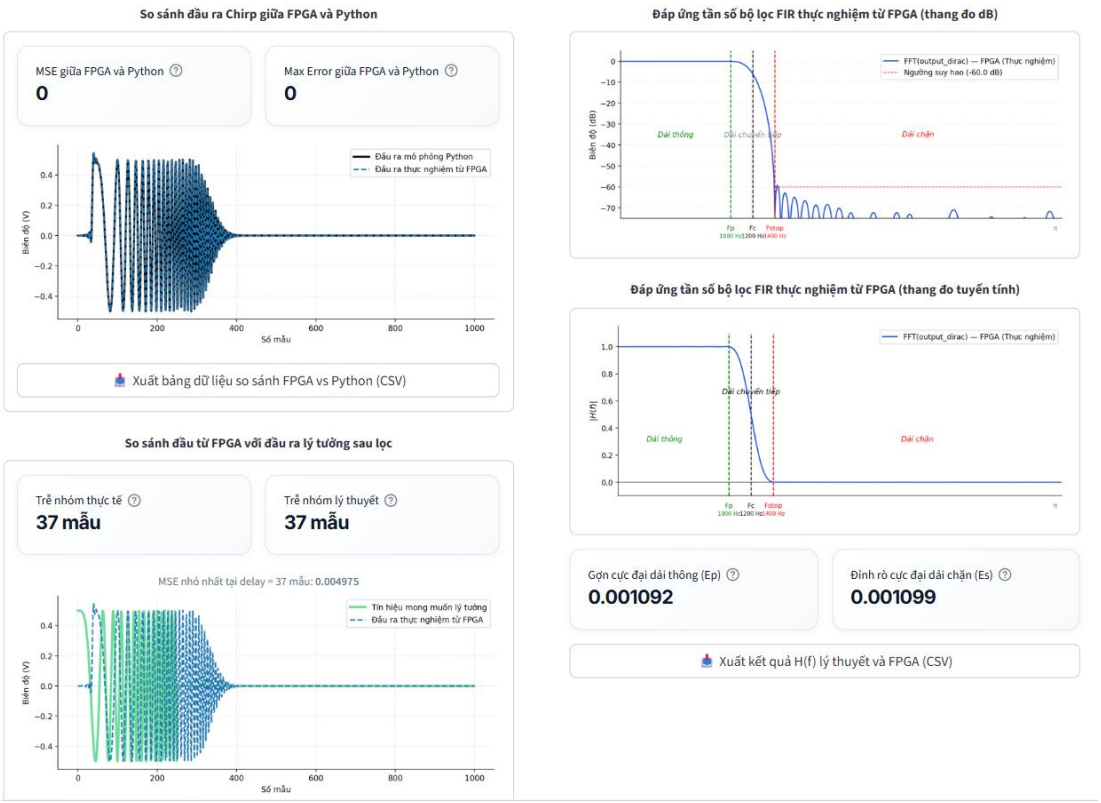
MSE	0
Max Error	0
Ep (gợn cực đại dải thông)	0.013787
Es (đỉnh rò cực đại dải chặn)	0.011013
Trễ nhóm	9

Nhận xét: Với số tap nhỏ, hệ thống có thể thực hiện đầy đủ quá trình thiết kế hệ số, nạp hệ số, lọc tín hiệu và hiển thị kết quả. Tín hiệu đầu ra FPGA có xu hướng tương đồng với kết quả Python. Sai số giữa hai tín hiệu chủ yếu đến từ quá trình lượng tử hóa Q15 và xử lý fixed-point trên FPGA.

A.1 Kết quả case 2



Hình 4.20 Thông số thiết kế FIR trong case 2



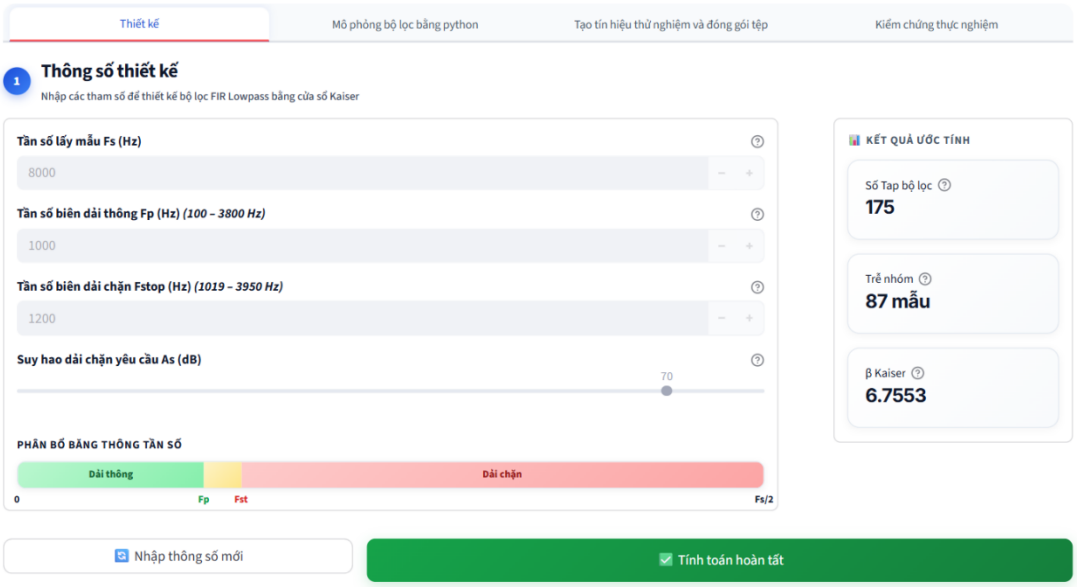
Hình 4.21 Kết quả case 2

Bảng 4.8 Kết quả case 2

Thông số	**Giá trị**
MSE	0
Max Error	0
Ep (gợn cực đại dải thông)	0.001092
Es (đỉnh rò cực đại dải chặn)	0.001099
Trễ nhóm	37

Nhận xét: Kết quả cho thấy đầu ra FPGA bám theo đầu ra tham chiếu từ Python. So với Case 1, bộ lọc trong Case 2 có dải chuyển tiếp hẹp hơn và yêu cầu suy hao dải chặn cao hơn nên số tap tăng lên. Đáp ứng tần số thể hiện rõ đặc tính thông thấp, biên độ trong dải thông được giữ tương đối ổn định và tín hiệu ở dải chặn bị suy giảm rõ rệt.

A.3 Kết quả case 3



Hình 4.22 Thông số thiết kế FIR trong Case 3



Hình 4.23 Kết quả case 3

Bảng 4.9 Kết quả case 3

Thông số	**Giá trị**
MSE	0
Max Error	0
Ep (gợn cực đại dải thông)	0.000409
Es (đỉnh rò cực đại dải chặn)	0.000452
Trễ nhóm	87

Nhận xét: Case 3 cho thấy hệ thống vẫn có thể hoạt động với bộ lọc có số tap lớn. Khi số tap tăng, đáp ứng tần số của bộ lọc trở nên sắc hơn, dải chuyển tiếp hẹp hơn và khả năng suy giảm dải chặn tốt hơn. Tuy nhiên, do FIR core sử dụng kiến trúc MAC tuần tự nên số chu kỳ xử lý cho mỗi mẫu cũng tăng theo số tap.

A.4 Tổng hợp kết quả

Bảng 4.10 Tổng hợp kết quả

Case	Số tap	MSE	Max Error	Ep	Es	Nhận xét
1	19	0	0	0.013787	0.011013	Hoạt động tốt với bộ lọc đơn giản, số tap nhỏ và dải chuyển rộng.
2	75	0	0	0.001092	0.001099	Case chính, cân bằng giữa chất lượng lọc, sai số và tài nguyên phần cứng.
3	175	0	0	0.000409	0.000452	Kiểm tra khả năng hoạt động của hệ thống khi số tap gần giới hạn 200 tap.

Từ các kết quả trên, có thể thấy hệ thống FIR trên FPGA hoạt động đúng với nhiều cấu hình bộ lọc khác nhau. Khi số tap tăng, chất lượng lọc được cải thiện nhưng thời gian xử lý cũng tăng theo do kiến trúc sử dụng MAC tuần tự. Kết quả so sánh với Python cho thấy đầu ra FPGA có sự tương đồng với mô hình tham chiếu, trong khi đáp ứng tần số thu được từ tín hiệu Dirac thể hiện đúng đặc tính của bộ lọc thông thấp.

Nhìn chung, phần kiểm chứng trên Streamlit đã chứng minh được khả năng thiết kế, nạp hệ số, xử lý và đánh giá bộ lọc FIR cấu hình lại trên FPGA.

CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài đã thực hiện thiết kế và kiểm chứng hệ thống xử lý tín hiệu số trên FPGA sử dụng bộ lọc FIR thông thấp có khả năng cấu hình lại hệ số thông qua giao tiếp UART với máy tính. Hệ thống không chỉ truyền nhận dữ liệu mà còn thực hiện trực tiếp phép lọc FIR trên FPGA, trong đó PC/Web đóng vai trò giao diện cấu hình, quan sát và đối chiếu kết quả.

Trên FPGA, bộ lọc FIR được hiện thực theo kiến trúc nhân – cộng tích lũy tuần tự 200 tap nhằm tiết kiệm tài nguyên phần cứng. Các module UART Receiver, Byte Packer, Coeff Loader, ROM Controller, FIR Filter, FIFO Buffer, Unbyte Packer và UART Transmitter đã được tích hợp thành một luồng xử lý hoàn chỉnh từ khâu nạp hệ số đến khâu xuất kết quả sau lọc.

Quá trình kiểm chứng được thực hiện ở hai mức. Ở mức mô phỏng RTL, các dạng sóng cho thấy từng khối và toàn hệ thống hoạt động đúng theo kịch bản kiểm thử. Ở mức hệ thống, kết quả FPGA được so sánh với mô phỏng Python/Streamlit, trong đó trường hợp kiểm chứng trực tiếp cho MSE = 0 và Max Error = 0. Điều này chứng minh FIR core và các khối truyền nhận dữ liệu hoạt động đúng theo mô hình tham chiếu.

Kết quả phân tích bổ sung cho thấy độ trễ tạo kết quả FIR đầu tiên khoảng 7,556 µs, phù hợp với kiến trúc MAC tuần tự trên clock 27 MHz. Trong khi đó, thời gian toàn hệ thống chủ yếu bị chi phối bởi tốc độ UART 9600 baud. Vì vậy, kiến trúc MAC tuần tự là lựa chọn hợp lý đối với FPGA Tang Nano 9K, vừa tiết kiệm tài nguyên vừa đáp ứng yêu cầu kiểm chứng của đề tài.

Nhìn chung, đề tài đã chứng minh được khả năng triển khai bộ lọc FIR thông thấp có thể cấu hình lại hệ số trên FPGA mà không cần nạp lại bitstream. Kết quả đạt được cho thấy hệ thống có tính đúng đắn, có cơ sở đánh giá định lượng và có khả năng mở rộng trong các hướng xử lý tín hiệu số phức tạp hơn.

5.2 Hướng phát triển của đề tài

Trong thời gian tiếp theo, đề tài có thể được phát triển theo một số hướng sau:

Thứ nhất, mở rộng hệ thống để hỗ trợ nhiều loại bộ lọc FIR khác nhau như bộ lọc thông cao, thông dải và chắn dải. Khi đó, người dùng có thể thay đổi đặc tính bộ lọc trực tiếp trên giao diện phần mềm mà không cần chỉnh sửa phần cứng.

Thứ hai, tối ưu kiến trúc FIR trên FPGA theo hướng tăng tốc độ xử lý, chẳng hạn sử dụng kiến trúc song song hoặc pipeline để giảm thời gian tính toán cho mỗi mẫu đầu vào. Việc này giúp hệ thống phù hợp hơn với các ứng dụng xử lý tín hiệu thời gian thực có tốc độ lấy mẫu cao.

Thứ ba, cải tiến giao tiếp truyền dữ liệu giữa máy tính và FPGA. UART có ưu điểm đơn giản, dễ triển khai nhưng tốc độ truyền còn hạn chế. Trong các phiên bản tiếp theo, có

thể nghiên cứu thay thế hoặc bổ sung các chuẩn giao tiếp tốc độ cao hơn như SPI, USB hoặc Ethernet.

Thứ tư, nâng cấp giao diện Streamlit để hỗ trợ trực quan hóa kết quả tốt hơn, tự động tính sai số, hiển thị đáp ứng tần số, đáp ứng thời gian và lưu kết quả kiểm chứng thành file phục vụ phân tích.

Thứ năm, triển khai hệ thống với tín hiệu thực tế từ cảm biến hoặc nguồn tín hiệu ngoài thay vì chỉ sử dụng tín hiệu kiểm thử như Dirac và Chirp. Điều này giúp đánh giá khả năng ứng dụng của bộ lọc FIR trên FPGA trong các bài toán xử lý tín hiệu thực tế.

Thứ sáu, bổ sung bước xuất báo cáo tự động từ Gowin IDE về tài nguyên sử dụng, timing/Fmax và công suất tiêu thụ. Khi có các số liệu này, báo cáo có thể đánh giá sâu hơn hiệu quả phần cứng thay vì chỉ dừng ở kiểm chứng chức năng.

TÀI LIỆU THAM KHẢO

[1] Pong P. Chu, RTL Hardware Design using VHDL, John Wiley & Sons, 2006.

[2] Behrouz A. Forouzan and Sophia Chung Fegan, Data Communications and Networking, 4th ed., McGraw-Hill, 2007.

[3] Hoang Le, Chương I: Tổng quan về nền tảng thiết kế số và công nghệ FPGA, Lecture notes, Học Viện Kỹ Thuật Mật Mã, 2022.

[4] U. Meyer-Baese, Digital Signal Processing with Field Programmable Gate Arrays, 3rd ed., Berlin, Germany: Springer, 2007.

[5] Phạm Thái Ngọc, Hồ Ngọc Bảo, Đỗ Duy Tân, Trương Quang Phúc, "Thiết kế và thi công bộ truyền nhận UART đa kênh dựa trên công nghệ FPGA," Tạp chí Khoa học Giáo dục Kỹ thuật, số 61, 2020.

[6] L.-M. Po, "FIR Filter Design," EE4015 Digital Signal Processing, Department of Electrical Engineering, City University of Hong Kong, lecture slides, 2022. [Online]. Available: https://www.ee.cityu.edu.hk/~lmpo/ee4015/pdf/2022_EE4015_L08A_FIR_Filter_Design.pdf. [Accessed: Jun. 5, 2026].

[7] J. G. Proakis and D. G. Manolakis, Digital Signal Processing: Principles, Algorithms, and Applications, 3rd ed., Prentice Hall, 1996.

[8] S. W. Smith, The Scientist and Engineer's Guide to Digital Signal Processing, California Technical Publishing, 1997.